



MENOUFIA UNIVERSITY
FACULTY OF COMPUTERS AND INFORMATION
DEPARTMENT OF COMPUTER SCIENCE



VESTORA

GRADUATION PROJECT DOCUMENTATION

Vestora — An Equity Crowdfunding Platform for Founders and Investors

Submitted in partial fulfilment of the requirements for the degree of Bachelor of Science

SUPERVISED BY

Dr. Khaled Amin

SEPTEMBER 2026



Project Team

01	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	
02	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	
03	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	
04	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	
05	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	
06	FULL NAME		
	STUDENT ID	ROLE ON THE PROJECT	

SUPERVISOR
Dr. Khaled Amin

SIGNATURE

Declaration

We declare that this document and the software system it describes are our own work, carried out as a graduation project in the Department of Computer Science, Faculty of Computers and Information, Menoufia University.

Where the work of others has been used, it is cited in the text and listed in the references. Third-party libraries, frameworks and services are identified at the point of use and summarised in Section 5. No part of this work has been submitted for any other degree or qualification.

The source code of the platform was written by the project team. Where a generated artefact appears in this document — a schema diagram derived from the data model, an endpoint table derived from the controllers — its origin is stated in the caption.

NAME AND SIGNATURE

NAME AND SIGNATURE

NAME AND SIGNATURE

NAME AND SIGNATURE

NAME AND SIGNATURE

NAME AND SIGNATURE

DATE

Acknowledgment

We are grateful to our supervisor, **Dr. Khaled Amin**, whose guidance shaped this project well beyond its code. The questions he asked at the right moments — about correctness, about what happens when a payment fails, about who is allowed to do what — are visible throughout this document.

We thank the academic and technical staff of the Faculty of Computers and Information for the foundation this work is built on, and for an environment in which a project of this scope was possible.

NAME AND TITLE	NAME AND TITLE

Finally, we thank our families, whose patience across a long build is the reason there was a project to write about at all.

Abstract

Early-stage founders and individual investors are separated by a market that serves neither of them well. Founders without an existing network struggle to reach capital; investors have no structured way to discover, evaluate and track small ventures. Reward-based crowdfunding platforms solved the first half of this problem for consumer products, but they do not model equity, do not support the diligence an investor needs, and end their involvement at the point of payment.

Vestora is a web platform that addresses the full relationship rather than the transaction. It supports the venture from submission through administrative review, publication, funding and post-funding reporting, and it gives investors discovery, saved searches, a watchlist, a document request channel and a portfolio view over their commitments.

The system is built as an ASP.NET Core REST API over SQL Server with Entity Framework Core, and a React front end using Next.js. Access is controlled by JSON Web Tokens with refresh-token rotation, layered over role-based authorisation and an account lifecycle that covers email verification, logout after repeated failed sign-in attempts, and administrative suspension. Payment is handled through a provider abstraction so that funding rules live in the domain rather than in an integration, with signature-verified, idempotent webhook processing. Messaging, presence, typing indicators and read receipts are delivered over SignalR.

Two design commitments shape most of the system. First, funding totals are **derived** rather than stored, so an approved investment and a funded one can never be confused. Second, a venture's administrative **status** is modelled separately from its commercial **stage**, so moderation decisions and fundraising progress cannot overwrite one another. Both are enforced at the database level rather than by convention.

This document records the requirements, the architecture, the decisions and their trade-offs, the security and payment models, the verification strategy, and an evaluation of the result against the objectives set out at the start.

KEYWORDS

Equity crowdfunding · Web application architecture · ASP.NET Core · React · Entity Framework Core · JSON Web Tokens · Role-based access control · Payment integration · Real-time communication · SignalR

الملخص

يفصل بين رواد الأعمال في مراحلهم المبكرة وبين المستثمرين الأفراد سوقٌ لا يخدم أيًا منهما بالشكل الكافي. فرائد الأعمال الذي لا يملك شبكة علاقات قائمة يجد صعوبة في الوصول إلى رأس المال، والمستثمر لا يجد وسيلة منظمة لاكتشاف المشاريع الصغيرة وتقييمها ومتابعتها. وقد عالجت منصات التمويل الجماعي القائمة على المكافآت نصف هذه المشكلة بالنسبة للمنتجات الاستهلاكية، لكنها لا تمثل حصص الملكية، ولا تدعم إجراءات الفحص النافي للجهالة التي يحتاجها المستثمر، وينتهي دورها عند لحظة الدفع. **Vestora** منصة ويب تعالج العلاقة كاملة لا الصفقة وحدها. فهي ترافق المشروع من لحظة تقديمه، مرورًا بالمراجعة الإدارية والنشر والتمويل، وصولًا إلى التقارير اللاحقة للتمويل. وتمنح المستثمر أدوات الاكتشاف وعمليات البحث المحفوظة وقائمة المتابعة وقناة لطلب المستندات، إلى جانب عرض لمحفظة الاستثمارية.

بُني النظام باستخدام واجهة برمجية من نوع REST على منصة ASP.NET Core فوق قاعدة بيانات SQL Server عبر Entity Framework Core، مع واجهة أمامية من React باستخدام Next.js. ويُضبط الوصول عبر رموز JSON Web Tokens مع تدوير رموز التجديد، فوق طبقة تفويض قائمة على الأدوار، ودورة حياة للحساب تشمل توثيق البريد الإلكتروني وإيقاف الدخول بعد تكرار المحاولات الفاشلة والتعليق الإداري. ويُعالج الدفع من خلال طبقة تجريد للمزود، بحيث تبقى قواعد التمويل داخل نطاق النظام لا داخل التكامل الخارجي، مع معالجة للـ webhooks موقّعة ومحصّنة ضد التكرار. أما المراسلة والحضور ومؤشرات الكتابة وإيصالات القراءة فنُنقل عبر SignalR.

ويحكم النظام التزامان تصميميان أساسيان: الأول أن إجماليات التمويل تُشَتَّق ولا تُخزَّن، فلا يختلط استثمارٌ معتمد بآخر ممول فعليًا. والثاني أن الحالة الإدارية للمشروع تُنمذج بمعزل عن مرحلته التجارية، فلا تطمس قرارات المراجعة تقدّم التمويل ولا العكس. وكلا الالتزامين مفروضٌ على مستوى قاعدة البيانات لا بالاصطلاح. توثّق هذه الرسالة المتطلبات والمعمارية والقرارات الهندسية ومقايضاتها، ونموذجي الأمان والدفع، واستراتيجية التحقق، وتقييمًا للنتيجة في ضوء الأهداف الموضوعية في البداية.

Executive Summary

The problem

A founder with a credible early-stage venture and no investor network has few routes to capital. Banks want collateral and trading history; venture capital operates on introductions; reward-based crowdfunding sells products, not equity. On the other side, an individual willing to place a modest amount into an early venture has no dependable way to find one, judge it, or follow what happens to the money afterwards.

What was built

Vestora is a web platform that carries a venture through its whole life rather than to the point of payment. Founders submit a venture, which is reviewed by an administrator before it becomes visible. Investors discover ventures through search and filtering, save them, follow founders, request documents, and commit capital. After funding, founders publish milestones and updates; investors track commitments in a portfolio view. Both sides communicate in real time inside the platform, and administrators moderate content, handle reports and see the activity of the platform as a whole.

How it is engineered

Layer	Choice
Front end	React with Next.js, Tailwind CSS and a component library built on Radix primitives
API	ASP.NET Core, a REST surface across twenty-three controllers
Data	SQL Server through Entity Framework Core, evolved over a versioned migration history
Identity	JSON Web Tokens with rotating refresh tokens, email verification, lockout on repeated failure, and role-based authorisation
Payments	A provider abstraction with funding rules held in the domain; webhooks are signature-verified and idempotent
Real time	SignalR for messaging, presence, typing indicators and read receipts

What distinguishes it

Three properties are worth stating plainly, because each was a decision with a cost.

1. **Funding totals are derived, never stored.** An investment that has been approved is not the same thing as an investment that has been paid, and the system cannot conflate the two. The rule is enforced by database constraints rather than by discipline.
2. **A venture's administrative status is separate from its commercial stage.** Moderation and fundraising progress move independently, so an approval can never silently reset a funding stage, and a funding event can never un-reject a venture.
3. **Payment rules do not live in the payment integration.** The provider interface handles transport only. Switching or adding a provider cannot change what the platform considers a valid commitment.

Status

The platform is implemented and running against a hosted database. Payment runs against a simulated provider by default, with a Stripe-compatible path — including signature-verified webhook handling — available by configuration; Section 11 describes the separation precisely and states what has and has not been exercised against a live provider. Verification, deployment and performance results are reported in Chapters 15 to 17, and the outcome is assessed against the original objectives in Chapter 19.

How to Read This Document

This document is long because the system is. It is organised so that it does not have to be read in order.

If you want	Read
The shortest complete picture	The Abstract, the Executive Summary, and §1.7 System Overview.
Why the system is built this way	Chapter 5 (technology choices), Chapter 6 (architecture, including the Architecture Decision Records in §6.9).
Whether it is safe	Chapter 10 (threat model, authentication, authorisation) and Chapter 11 (payment integrity).
Whether it works	Chapter 15 (testing), Chapter 17 (performance), Chapter 19 (evaluation against the stated objectives).
What it actually does	Chapters 13 and 14, which walk the platform's features as journeys rather than as endpoint listings.
What was hard	Chapter 18 — the problems that cost the most time, and how each was resolved.
Reference material	The appendices: full API listing, schema, traceability matrix, test catalogue, interface gallery.

Conventions

Figures, tables and listings are numbered within their chapter — *Figure 6.2* is the second figure of Chapter 6 — and every one is referred to from the text.

Code appears as text, never as a screenshot, so that it can be searched, copied and read at any zoom level. Listings show the portion that carries the point being made; complete files are in Section 6.

Requirements carry stable identifiers (*FR-14*, *NFR-03*) and are traceable from Chapter 4 through design and implementation to the test cases in Section 4.

Decisions that shaped the architecture are recorded as numbered *Architecture Decision Records* in §6.9, each stating the context, the decision and its consequences.

NOTE

Sections still marked *Draft* are noted as such in place. Nothing in this document is presented as finished when it is not.

Contents

1 Introduction	2
1.1 Background and Motivation	2
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Aim and Objectives	3
1.5 Project Scope	3
1.5.1 In Scope	3
1.5.2 Out of Scope	4
1.6 Contributions	4
1.7 System Overview	5
1.8 Document Structure	6
2 Domain Analysis and Related Work	7
2.1 The Crowdfunding and Equity Investment Domain	7
2.2 Review of Existing Platforms	8
2.2.1 Reward-Based Platforms	8
2.2.2 Equity Platforms	8
2.2.3 Regional Context	8
2.3 Comparative Feature Analysis	9
2.4 Gap Analysis: Where Vestora Differs	9
2.5 Stakeholder Analysis	10
2.6 Business Model and Revenue Strategy	10
2.7 Legal and Regulatory Considerations	10
2.8 Summary	11
3 Methodology and Project Management	12
3.1 Development Approach	12
3.2 Development Phases	12
3.3 Team Structure	13
3.4 Version Control and Collaboration	13
3.5 Tooling	14
3.6 Risk Management	14
3.7 Schedule	15
4 Requirements Engineering	17
4.1 Elicitation Approach	17
4.2 User Personas	17
4.3 Functional Requirements	17
4.3.1 Identity and Access	18
4.3.2 Projects and Ventures	18
4.3.3 Discovery and Engagement	18
4.3.4 Investment and Funding	19
4.3.5 Communication	19
4.3.6 Administration and Moderation	19
4.4 Non-Functional Requirements	19
4.5 Use Case Model	20
4.6 Use Case Specifications	24
4.7 Requirements Traceability	24
5 Technology Stack and Engineering Decisions	26
5.1 Decision Framework and Evaluation Criteria	26
5.2 Frontend Framework: React with Next.js	26
5.3 Case Study: Migrating from Angular to React	27
5.4 Backend Platform: ASP.NET Core	28
5.5 Data Layer: SQL Server with EF Core	28
5.6 Authentication Strategy: JWT with Refresh Tokens	29

5.7	Payment Integration Strategy	30
5.8	Real-Time Transport: SignalR	31
5.9	Transactional Email Delivery	31
5.10	UI Layer: Tailwind CSS with shadcn/ui	32
5.11	Hosting and Infrastructure	32
5.12	Decision Summary	33
6	System Architecture	34
6.1	Architectural Goals and Constraints	34
6.2	Architectural Style and Rationale	34
6.3	System Context and Containers	35
6.4	Component View	35
6.5	Backend Layering	37
6.5.1	Controllers	37
6.5.2	Services	37
6.5.3	Data Access	37
6.6	Frontend Architecture	37
6.6.1	Routing and Segments	37
6.6.2	Data Fetching	37
6.6.3	State Management	37
6.7	Cross-Cutting Concerns	38
6.7.1	Validation	38
6.7.2	Error Handling	38
6.7.3	Logging	38
6.7.4	Time and Serialisation	38
6.8	Design Patterns Applied	38
6.9	Architecture Decision Records	39
6.10	Deployment View	39
7	Database Design	42
7.1	Conceptual Data Model	42
7.2	Entity Relationship Diagram	42
7.3	Logical Schema and Data Dictionary	46
7.3.1	Project	46
7.3.2	Investment and the funding chain	46
7.4	Normalisation Analysis	46
7.5	Indexing Strategy	47
7.6	Constraints and Integrity Rules	49
7.6.1	Why funding totals are derived	49
7.6.2	Why status and stage are separate	49
7.6.3	Referential rules	50
7.7	Migration Strategy	50
7.8	Backup and Recovery	51
8	UI/UX Design System	52
8.1	Design Principles	52
8.2	Design Tokens	52
8.3	Component Library	54
8.4	The System in Use	54
8.4.1	The theme axis	55
8.4.2	The language axis	55
8.4.3	Mobile	56
8.4.4	A designed failure state	57
8.5	From Wireframe to Interface	57
8.6	User Flows	57
8.7	Responsive Strategy	58
8.7.1	Bidirectional layout	58
8.8	Accessibility Compliance	58
8.9	Motion and Interaction	59

9 API Design and Implementation	61
9.1 REST Principles Applied	61
9.2 Resource Modelling and URI Design	61
9.3 Request and Response Contracts	62
9.4 Validation Strategy	62
9.5 Error Handling and Result Shaping	62
9.6 Pagination, Filtering and Sorting	62
9.7 API Documentation	63
10 Security, Authentication and Authorisation	64
10.1 Threat Model	64
10.2 OWASP Top Ten Mitigation Mapping	65
10.3 Registration and Email Verification	65
10.4 One-Time Password Flow	67
10.5 JWT Issuance and Claims	67
10.6 Session Lifecycle: Refresh, Logout, Revocation	67
10.7 Brute-Force Protection and Lockout	67
10.8 Role-Based Access Control	69
10.9 Authorisation Matrix	69
10.10 Account Lifecycle and Suspension	69
10.11 Input Validation and Sanitisation	69
10.12 File Upload Security	70
10.13 Audit Logging and Data Privacy	70
11 Payment Architecture	71
11.1 Money Flow and Domain Model	71
11.2 Provider Abstraction	71
11.3 Checkout Sequence	72
11.4 Webhook Handling and Signature Verification	74
11.5 Idempotency and Replay Safety	74
11.6 Payment State Machine	74
11.7 Funding Integrity Model	75
11.8 Expiry, Refunds and Reconciliation	75
11.9 Environment Separation	76
12 Real-Time Communication	77
12.1 Hub Architecture and Connection Lifecycle	77
12.2 Presence Tracking and Durable Last-Seen	77
12.3 Typing Indicators and Read Receipts	78
12.4 Message Persistence and Delivery	79
12.5 Attachment Handling	81
12.6 Notification Fan-Out	81
12.7 Reconnection and Scaling	81
13 Feature Walkthrough I: Identity, Discovery and Engagement	82
13.1 Walkthrough Method and Notation	82
13.2 User Profile Journey	82
13.3 Browse, Discovery and Search	82
13.4 Saved Projects and Saved Searches	83
13.5 Following and Investor Networks	83
13.6 Reviews, Comments and Replies	83
13.7 Notification Experience	84
14 Feature Walkthrough II: Investment, Governance and Insight	85
14.1 Investment Workflow	85
14.2 Project Lifecycle: Status versus Stage	86
14.3 Milestones and Project Updates	89
14.4 Document Requests and Deal Room	89
14.5 Administrative Workflow: Review and Moderation	89
14.6 Role-Based Dashboards	90

14.6.1 Founder Dashboard	90
14.6.2 Investor Dashboard	90
14.7 Analytics and Reporting	92
15 Testing Strategy	94
15.1 Testing Approach	94
15.2 Unit Testing	95
15.3 Integration Testing	95
15.4 API Testing	96
15.5 End-to-End Testing	96
15.6 Security Testing	96
15.7 Defect Management	97
15.8 Requirements-to-Test Traceability	97
16 Deployment and Operations	98
16.1 Environment Strategy	98
16.2 Hosting Architecture	98
16.3 Database Hosting and Connection Security	98
16.4 Build and Release	98
16.5 Configuration Management	99
16.6 Secrets Management	99
16.7 Logging	100
16.8 Monitoring and Health Checks	100
16.9 Error Reporting	100
16.10 Backup, Rollback and Recovery	100
16.11 Release Checklist	101
17 Performance and Scalability	102
17.1 Performance Budget and Targets	102
17.2 Frontend Performance	102
17.3 API Response Times	103
17.4 Database Query Optimisation	103
17.4.1 Projection instead of materialisation	104
17.4.2 Eliminating repeated queries	104
17.4.3 Indexing	104
17.5 Caching Strategy	104
17.6 Load Testing	104
17.7 Scalability	105
18 Challenges and Solutions	108
18.1 Timestamps That Were Right Everywhere Except Production	108
18.2 Approval Silently Counting as Funding	108
18.3 One Status Column, Three Writers	109
18.4 Rewriting the Frontend Without Rewriting the System	109
18.5 A Shared Database and No Local Fallback	110
18.6 Theme Tokens That Inverted Under a Dark Scope	110
18.7 Testing Against Motion and Smooth Scrolling	110
18.8 Process Challenges	110
18.9 What Generalises	111
19 Evaluation and Results	112
19.1 Objectives Assessment	112
19.2 Research Questions Answered	113
19.3 Requirements Coverage	113
19.4 User Evaluation	114
19.5 Limitations	114
19.6 Threats to Validity	115
20 Conclusion and Future Work	116
20.1 Summary of Contributions	116
20.2 Lessons Learned	116

20.2.1 Technical	116
20.2.2 Process	117
20.2.3 Personal	117
20.3 Future Work	117
20.4 Closing	118
References	120
1 Complete API Reference	123
1.1 Surface Map	124
1.2 Representative Endpoints	124
2 Database Schema and Data Dictionary	126
2.1 Identity and Access	126
2.2 Ventures	127
2.3 Funding	127
2.4 Engagement and Social	128
2.5 Communication	128
2.6 Conventions	128
3 Requirements Traceability Matrix	129
3.1 Functional Requirements	129
3.2 Non-Functional Requirements	130
3.3 Reading the Matrix	131
4 Test Case Catalogue	132
4.1 Domain Unit Cases	132
4.2 Integration Cases	133
4.3 Authorisation Cases	133
4.4 Security Cases	134
4.5 End-to-End Journeys	134
4.6 Summary	134
5 Interface Gallery	135
5.1 Captured Gallery	135
5.1.1 Investor	135
5.1.2 Founder	136
5.1.3 Administrator	137
5.1.4 Authorisation, observed	138
5.1.5 Communication	139
5.2 Public Surfaces	139
5.3 Identity	139
5.4 Investor Surfaces	139
5.5 Founder Surfaces	140
5.6 Communication	140
5.7 Administration	140
5.8 Responsive and Theme	140
6 Key Code Listings	141
6.1 The Payment Provider Boundary	141
6.2 Webhook Verification and Idempotency	141
6.3 Deriving Funding Totals	142
6.4 Independent State Columns	142
6.5 Business Rules as Unique Indexes	143
6.6 Uniform Result Translation	143
7 Installation and Setup Guide	144
7.1 Prerequisites	144
7.2 Configuration	144
7.3 Running the API	144
7.4 Running the Web Application	145
7.5 Verifying the Installation	145

7.6 Common Problems	145
8 User Manual	146
8.1 Getting Started (All Users)	146
8.2 For Investors	146
8.2.1 Finding ventures	146
8.2.2 Assessing a venture	146
8.2.3 Committing capital	146
8.2.4 Tracking	147
8.3 For Founders	147
8.3.1 Submitting a venture	147
8.3.2 Managing funding	147
8.3.3 Keeping investors informed	147
8.3.4 Documents	147
8.4 For Administrators	147
8.4.1 The review queue	147
8.4.2 Reports	147
8.4.3 Accounts	147
8.4.4 Oversight	148

List of Figures

Figure 1	System context: the platform’s internal containers and the external services it depends on.	5
Figure 2	Use case model. Fourteen cases across three human actors and one external one. <code>UC-05 Commit capital</code> and <code>UC-07 Settle a commitment</code> are deliberately separate cases, not two steps of one — which is the same distinction §14.1 draws in the state model.	21
Figure 3	Level-1 data flow. Six processes, five stores, three external actors. The funding chain is a single store because §11.1 treats its four tables as one transactional unit.	23
Figure 4	Component view of the API. Controllers delegate and never query; domain services hold the rules and perform no I/O; only <code>AppDbContext</code> reaches the database.	36
Figure 5	Deployment view. One instance of each container; the payment provider’s callback is the only inbound path the platform does not initiate.	40
Figure 6	The funding chain, derived from the Entity Framework configuration in <code>AppDbContext</code> . Identity and the venture sit at the top; each step down holds a fact the step above cannot.	43
Figure 7	Domain class model. <code>Investor</code> and <code>Innovator</code> specialise <code>User</code> through a discriminator (§7.1); the services at the foot hold rules the entities themselves do not.	45
Figure 8	The palette, printed from the token definitions themselves. The final cell is the contrast pairing that §8.7 depends on.	53
Figure 9	The type scale, set in the faces the product loads.	53
Figure 10	The landing page. Cinzel at display size, the gold accent used once, and an image-led composition — the four principles of §8.1 in one frame.	54
Figure 11	The venture listing under the light and dark token sets. Every colour is a token with the same role in both; nothing is re-chosen per theme (§8.2).	55
Figure 12	The same surface in English and Arabic. The layout mirrors, the display face changes to Aref Ruqaa, and the information is identical.	55
Figure 13	The landing page at 390 px, in English light and Arabic dark — both varying axes at once. The composition rearranges; nothing is removed (§8.6).	56
Figure 14	The same listing captured with the API stopped. What is visible is the error state : a designed surface with a stated cause and a single recovery action, not a blank page or a raw message (§8.8).	57
Figure 15	Registration and email verification. The response is identical whether or not the address is already registered, so the endpoint is not an account-existence oracle. .	66
Figure 16	Sign-in and refresh-token rotation. Presenting an already-rotated token invalidates the whole chain — the replay detection that was observed firing during screenshot capture (Section 5).	68
Figure 17	The funding path end to end. Signature verification precedes every state change, the unique event identifier makes duplicate delivery a no-op, and the total is derived at the end rather than incremented along the way.	73
Figure 18	Payment transaction states.	75
Figure 19	A diligence conversation, seen by the investor. Three mechanisms of this chapter are visible at once: durable last-seen in the header (§12.2), read state on each outgoing message, and the unread divider.	78

Figure 20	The same thread from the founder's side, and from the investor's side in Arabic under the dark token set. The bubbles swap alignment with the sender, and again with the writing direction.	79
Figure 21	A conversation end to end. Presence is derived from the set of a user's connections, so closing one tab of three does not take them offline; messages are persisted before they are pushed.	80
Figure 22	A venture page. The commitment path begins here, and the page leads with evidence — imagery, sector, stage, location and the founder — before it asks for anything.	86
Figure 23	The three state machines of a venture, side by side. They share no transition and no writer, which is why an approval cannot reset funding progress (ADR-05).	88
Figure 24	The investor dashboard for a new account. Funded and Approved commitments are separate tiles — the distinction of §14.1 is not an internal detail, it is the first thing the surface reports.	91
Figure 25	The same dashboard in Arabic under the dark token set. The sidebar, tiles and reading order mirror; the figures and their meaning do not change (§8.6.1).	91
Figure 26	Investor overview: funded and approved as separate tiles (§14.1). Pipeline: commitments by state.	135
Figure 27	Portfolio: settled holdings only. Payments: settlement history.	136
Figure 28	Watchlist (§13.4). Activity log.	136
Figure 29	Saved searches (§13.3). Profile settings, separate from account (§13.2).	136
Figure 30	Founder overview (§14.6). Venture list with moderation state.	136
Figure 31	Funding: committed against settled. Incoming commitments awaiting approval (§14.1).	137
Figure 32	Venture analytics (§14.7). Venture submission form (§14.2).	137
Figure 33	Platform overview with derived analytics (§14.7). Review queue — the primary moderation surface (§14.5).	137
Figure 34	Venture registry across all states. Account administration and suspension (§10.10).	137
Figure 35	Fee revenue, derived from settled transactions (§2.6). Audit log: administrative action attributed to its actor (§10.13).	138
Figure 36	Security events from <code>SecurityLog</code> (§10.13). Platform-wide activity stream.	138
Figure 37	An investor account requesting a surface it is not entitled to. The client redirects to a refusal rather than rendering an empty page — and the API would refuse the underlying call regardless of what the client did (§10.8). This is case AZ-01 of Section 4, observed rather than executed.	138

List of Tables

Table 1	Project objectives and where each is assessed.	3
Table 2	The four crowdfunding models and what distinguishes them.	7
Table 3	Capability comparison. The final two rows are capabilities Vestora does not have. ...	9
Table 4	Stakeholders and how each shaped the design.	10
Table 5	Development phases and what each produced.	13
Table 6	Areas of responsibility. Team members are listed on the roster page; the mapping is recorded there rather than duplicated here.	13
Table 7	Tooling and what each is used for.	14
Table 8	Risk register with actual outcomes. Five of six materialised.	14
Table 9	The three personas the requirements serve.	17
Table 10	Identity and access requirements.	18
Table 11	Venture management requirements.	18
Table 12	Discovery and engagement requirements.	18
Table 13	Investment and funding requirements.	19
Table 14	Communication requirements.	19
Table 15	Administration and moderation requirements.	19
Table 16	Non-functional requirements, each with a section that assesses it.	20
Table 17	Evaluation criteria applied to every technology decision in this chapter.	26
Table 18	Frontend framework evaluation.	27
Table 19	Backend platform evaluation.	28
Table 20	Database evaluation. SQL Server and PostgreSQL tie on the weighted score.	29
Table 21	Authentication lifetimes, all supplied by configuration.	29
Table 22	Email delivery evaluation.	31
Table 23	Summary of technology decisions and the cost accepted for each.	33
Table 24	Container responsibilities and boundaries.	35
Table 25	Patterns used, and the specific problem each solves.	38
Table 26	Index of Architecture Decision Records.	39
Table 27	The Project table. Three independent state columns, three independent writers. ...	46
Table 28	Indexes and the query that motivated each.	48
Table 29	Three state columns, three writers, three questions.	49
Table 30	Delete behaviour by relationship class.	50
Table 31	Colour tokens. A dark-mode set exists in the product with the same role assignments.	52
Table 32	Typographic tokens.	53
Table 33	Accessibility position by criterion, including one that is only partial.	58
Table 34	URI patterns and what each is for.	61
Table 35	STRIDE threat enumeration and the control applied to each.	64
Table 36	OWASP Top Ten mapping. Two rows record gaps rather than controls.	65
Table 37	Access token properties.	67
Table 38	Authorisation matrix. ¹ A founder may not invest in their own venture. ² Administrators moderate reported content, not private conversations.	69
Table 39	The four objects in the funding chain and the distinct fact each holds.	71

Table 40	Where payment responsibility lives.	72
Table 41	Payment transaction states. Only <code>Settled</code> contributes to funding.	74
Table 42	What horizontal scaling of the real-time layer would require.	81
Table 43	Investment states. Only one of them is money.	85
Table 44	The three state columns, their values and who may move them.	87
Table 45	Administrative surfaces and what each is for.	90
Table 46	<code>dotnet test</code> against <code>MyAppApi.Tests</code> . Figures are from the run, not from a plan. ...	94
Table 47	Test layers and the reasoning behind the effort split.	94
Table 48	Domain unit cases and the specific failure each prevents from recurring.	95
Table 49	Coverage by scope. The three rows are the same run measured at three altitudes. . .	95
Table 50	Significant defects and how each was resolved. Note that the two most severe were resolved by changing the model, not the code.	97
Table 51	Environments and their payment configuration.	98
Table 52	Configuration categories. Every value here differs by environment or is a secret. . .	99
Table 53	Performance targets, how each is checked, and its state.	102
Table 54	Frontend timings. Both pages paint content in under half a second.	103
Table 55	Listing latency. NFR-02 requires p95 below 300 ms; it is not met.	103
Table 56	Concurrency sweep. The last two rows measure the harness, not the server — see below.	105
Table 57	Scaling constraints in the order they would bind.	106
Table 58	Objectives assessment. Six met, one partly met.	112
Table 59	Requirements coverage. Every requirement is built; none is verified by an automated test.	113
Table 60	Future work, ordered by dependency and value.	118
Table 61	The API surface by controller. Counts are HTTP action attributes in each controller.	124
Table 62	Endpoints referenced from the main text.	125
Table 63	Identity and access tables.	126
Table 64	Venture tables. All are owned by the venture and cascade with it.	127
Table 65	Funding tables. No aggregate column exists anywhere in this group.	127
Table 66	Engagement tables. Four business rules in this group are unique indexes rather than application checks (§7.5).	128
Table 67	Communication tables.	128
Table 68	Functional requirements traced to design, implementation and test.	129
Table 69	Non-functional requirements traced to design, implementation and assessment. .	130
Table 70	Domain unit cases. All eleven are executed by <code>MyAppApi.Tests</code> and passing; three trace directly to defects in Section 18.	132
Table 71	Integration cases. These verify constraints as constraints rather than as intentions.	133
Table 72	Authorisation cases, one per negative cell of §10.9.	133
Table 73	Security cases, targeted at the threat model in §10.1.	134
Table 74	End-to-end journeys. E2E-03 is the most valuable test in the suite.	134
Table 75	Catalogue summary.	134
Table 76	Axes covered by the captured gallery.	135
Table 77	Public surfaces.	139

Table 78 Identity surfaces, including two failure states. 139

Table 79 Investor surfaces. Entry 16 is captured twice deliberately. 139

Table 80 Founder surfaces. 140

Table 81 Communication surfaces. 140

Table 82 Administrative surfaces. 140

Table 83 Responsive and theme captures. Entry 36 evidences an accessibility claim rather than illustrating a feature. 140

Table 84 Prerequisites. 144

Table 85 Required configuration. A missing value fails startup rather than defaulting. 144

Table 86 Common setup problems and their causes. 145

List of Listings

Listing 1	The provider boundary. It exposes no domain concept — no venture, no investor, no funding target — only the three operations a payment processor performs.	30
Listing 2	The controller-to-service seam. Outcome mapping happens in one place for the whole API.	37
Listing 3	Uniform result translation at the controller seam.	62
Listing 4	Duplicate suppression. The uniqueness constraint, not the check, is what makes this safe under concurrency.	74
Listing 5	<code>IPaymentProvider</code> . Two implementations satisfy it and neither contains a domain rule.	141
Listing 6	Webhook handling. Acknowledging a duplicate with <code>200</code> is deliberate — an error causes the provider to retry indefinitely for an event that will never become processable.	142
Listing 7	Funding derivation. The <code>?? 0m</code> matters: a venture with no transactions has raised zero, not null.	142
Listing 8	Venture state indexing.	142
Listing 9	Constraints carrying business rules.	143
Listing 10	The controller-to-service seam. A controller may not reference <code>AppDbContext</code> (§6.5).	143

Abbreviations

Term	Expansion
ADR	Architecture Decision Record
API	Application Programming Interface
BCrypt	Password hashing function based on the Blowfish cipher
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
EF Core	Entity Framework Core
ERD	Entity Relationship Diagram
FR / NFR	Functional / Non-Functional Requirement
JWT	JSON Web Token
ORM	Object-Relational Mapping
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
REST	Representational State Transfer
RSC	React Server Component
SPA	Single-Page Application
SSR	Server-Side Rendering
TTL	Time To Live
UTC	Coordinated Universal Time
WCAG	Web Content Accessibility Guidelines

PART 1

Foundation

Why the project exists, what already exists in the field, and how the work was organised.

Introduction

1.1 Background and Motivation

Capital and ideas are both plentiful. What is scarce is a reliable way for the two to find each other at the smallest end of the market.

A founder with a working prototype and no investor network has a narrow set of options. Bank lending is priced for collateral and trading history, neither of which an eighteen-month-old company has. Institutional venture capital operates on deal flow that arrives through introductions, so the founder without an introduction is not rejected so much as never seen. What remains is the founder's own network — which is precisely the resource that correlates with prior advantage rather than with the quality of the idea.

The mirror problem exists on the other side. A person willing to place a modest sum into an early-stage venture has no structured place to do it. Public markets offer liquidity but not early-stage exposure. Angel syndicates require capital and connections at a level most people do not have. The individual investor is left choosing between opacity and exclusion.

Reward-based crowdfunding platforms addressed a version of this gap and demonstrated that strangers will fund strangers at scale when the mechanism is trustworthy. But their model is a pre-order: the backer receives a product, not a stake, and the platform's involvement ends when the money moves. Nothing in that model supports the questions an investor actually asks — what is the company worth, what am I entitled to, how do I verify the claims in this pitch, and what happens after I commit.

Vestora exists in the space those two models leave open: a platform that treats funding as the middle of a relationship rather than the end of a transaction.

1.2 Problem Statement

The problem this project addresses can be stated as four concrete failures of the current arrangement.

Discovery is networked, not merit-based A founder's access to investors is determined largely by who they already know. There is no neutral surface on which a venture can be found on the strength of what it is.

Evaluation is unsupported An investor considering an early-stage venture needs the team, the financial ask, supporting documents and a record of progress. Where these exist at all they are scattered across decks, emails and calls, and none of it is verifiable by the platform hosting the pitch.

Commitment is unmodelled Existing consumer platforms model a purchase. An equity or revenue commitment is a different object: it has an approval step, a settlement step, and a status that must never be confused with either.

The relationship ends at payment Once funds move, the platform stops. There is no channel through which a founder reports progress and no view through which an investor tracks it, so accountability depends entirely on the goodwill of the founder.

Each of these is a software problem before it is a market problem, and each appears again later in this document as a requirement, a design decision and a test.

1.3 Research Questions

The work is organised around four questions.

1. **RQ1.** How can a platform present early-stage ventures so that discovery depends on the attributes of the venture rather than on the founder's existing network?
2. **RQ2.** What data model correctly separates the administrative state of a venture from its commercial progress, such that neither can corrupt the other?
3. **RQ3.** How can funding integrity — the guarantee that a recorded commitment reflects money that actually moved — be enforced structurally rather than by application-level convention?
4. **RQ4.** What architecture allows a payment provider to be replaced or simulated without any change to the platform's definition of a valid investment?

RQ2 and RQ3 are answered by the data model in Section 7 and the funding rules in Section 11. RQ4 is answered by the provider abstraction described in §11.2. RQ1 runs through discovery, search and ranking in Section 13.

1.4 Aim and Objectives

Aim. To design, build and evaluate a web platform that connects early-stage founders with individual investors, and that remains correct about money and about state throughout the venture's life.

The aim decomposes into seven objectives. Each is stated so that it can be checked rather than asserted; Section 19 reports the outcome against this list.

	Objective	Verified by
01	Provide a submission-to-publication pipeline in which every venture passes an explicit administrative review before becoming visible.	§14.2, §14.5
02	Model venture state such that administrative status and commercial stage are independent and cannot overwrite one another.	§7.6, §14.2
03	Guarantee that funding totals reported to users are derived from settled payments only, enforced below the application layer.	§7.6, §11.7
04	Isolate payment-provider integration behind an interface so that domain rules are provider-independent.	§11.2
05	Enforce access control by role across every state-changing endpoint, with an account lifecycle covering verification, lockout and suspension.	§10.7–§10.10
06	Support discovery through search, filtering, saved searches and a watchlist, so that a venture can be found without an introduction.	§13.3, §13.4
07	Sustain the relationship after funding through milestones, updates, document requests and in-platform messaging.	§14.3, §14.4

Table 1 Project objectives and where each is assessed.

1.5 Project Scope

1.5.1 In Scope

The delivered system covers the following.

- **Identity and access.** Registration, email verification, one-time password confirmation, sign-in with JSON Web Tokens and rotating refresh tokens, password reset, logout after repeated failures, role-based authorisation, and administrative suspension.
- **Venture management.** Creation and editing of ventures with imagery, documents, team members, funding targets, milestones and progress updates.
- **Administrative review.** A moderation pipeline covering approval, rejection with reason, reporting, and an audit trail of administrative actions.
- **Discovery.** Listing, search, filtering, sorting, saved searches, bookmarks and a following relationship between users.
- **Investment.** Funding requests, an approval step, checkout against a payment provider, settlement recorded from provider callbacks, and portfolio views for the investor.
- **Communication.** Direct messaging with presence, typing indicators, read receipts and image attachments; a notification system with in-app delivery.
- **Insight.** Founder and investor dashboards, and administrative analytics covering activity, revenue and moderation load.

1.5.2 Out of Scope

The following are deliberately excluded, and are revisited in §20.3.

- **Live money movement.** The platform integrates a payment provider behind an abstraction and runs against a simulated provider by default. The Stripe-compatible path, including signature-verified webhook handling, is implemented and configurable, but the system has not been operated against a live merchant account. Section 11 states precisely what has and has not been exercised.
- **Legal execution of equity.** The platform records commitments; it does not generate, execute or register share instruments.
- **Identity verification and anti-money-laundering checks.** Regulatory onboarding is discussed in §2.7 as a constraint on deployment, not implemented.
- **Secondary trading.** There is no mechanism for transferring a commitment between investors.
- **Automated recommendation.** Ranking is deterministic and attribute-based. No machine-learned recommendation is implemented.

1.6 Contributions

This project contributes the following, in decreasing order of generality.

1. **A state model that separates moderation from commerce.** A venture carries an administrative *status* and a commercial *stage* as independent fields with independent transition rules. This prevents an entire class of defect in which an approval resets fundraising progress, or a funding event silently reverses a rejection. The model and its constraints are given in §7.6.
2. **A derived-funding integrity model.** No column stores “amount raised”. Totals are computed from settled payment transactions at read time, and database constraints make the invalid intermediate states unrepresentable. The consequence — that an approved investment is structurally distinct from a funded one — is developed in §11.7.
3. **A provider-independent payment layer.** Domain rules governing what constitutes a valid commitment live in the platform’s own service layer; the provider interface carries transport only. The same rules therefore apply identically to the simulated and the Stripe-compatible provider, which is what makes the simulated path a legitimate test surface rather than a stand-in.
4. **A working platform.** An implemented system of twenty-two API controllers over a relational model of more than forty entities, with a React front end of sixty-five routes, evolved across a versioned migration history.

1.7 System Overview

Before the detail of later chapters, this section gives the whole shape of the system in one page.

Three kinds of user meet on the platform. A **founder** submits and manages a venture. An **investor** discovers ventures, commits capital and tracks the result. An **administrator** reviews submissions, handles reports and oversees platform activity. A single account may act as founder and investor; the administrative role is separate and granted, not self-selected.

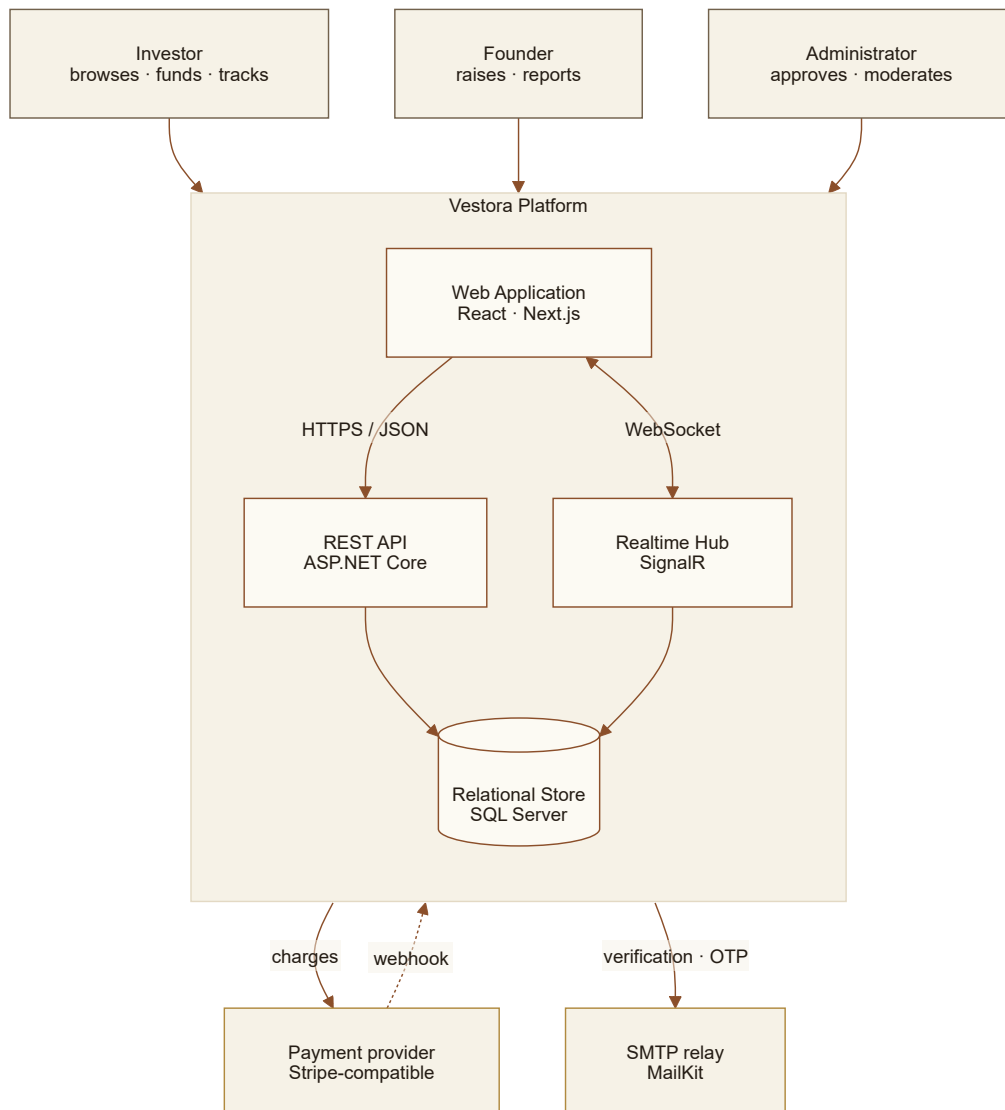


Figure 1 System context: the platform’s internal containers and the external services it depends on.

The browser talks to a Next.js application, which renders the interface and calls a REST API over HTTPS. The same browser holds a persistent WebSocket to a SignalR hub for messaging and presence. The API is the only writer to the relational store; the hub reads and writes the messaging tables through the same data context. Two external dependencies sit outside the boundary: a payment provider, reached through an interface that hides which provider is configured, and an SMTP relay used for verification, password reset and one-time password delivery.

A venture moves through the platform along a fixed path — submitted, reviewed, published, funded, reported on — and the system’s correctness properties are mostly properties of that path. Section 14 follows it end to end.

1.8 Document Structure

The document is in five parts.

Part I — Foundation establishes why the project exists (this chapter), what already exists in the field and where Vestora differs (Section 2), and how the work was organised and managed (Section 3).

Part II — Analysis and Design derives the requirements (Section 4), justifies each technology choice against alternatives (Section 5), presents the architecture and the decisions behind it (Section 6), specifies the data model (Section 7), and documents the interface system (Section 8).

Part III — Implementation covers the API surface (Section 9), the security model (Section 10), the payment subsystem (Section 11), real-time communication (Section 12), and a walkthrough of the platform's features as user journeys (Section 13, Section 14).

Part IV — Quality and Delivery reports verification (Section 15), deployment and operations (Section 16), and performance (Section 17).

Part V — Closure records the problems that cost the most time (Section 18), assesses the result against the objectives above (Section 19), and sets out what comes next (Section 20).

Reference material — the full endpoint listing, the schema, the traceability matrix and the test catalogue — is held in the appendices so that the chapters remain readable.

Domain Analysis and Related Work

NOTE
Claims in this chapter about how the field behaves are attributed to the crowdfunding literature in the bibliography. Claims about **this** platform are attributed to the sections that implement them. No statistic appears here without a source — which is why the chapter argues in structural terms rather than in numbers the project cannot independently verify.

2.1 The Crowdfunding and Equity Investment Domain

Crowdfunding describes raising capital from many small contributors rather than few large ones. The term covers four distinct models, and conflating them is the most common error in discussions of the field.

Model	The contributor receives	Consequence
Donation	Nothing	No return expectation; no regulatory perimeter.
Reward	A product or perk	Effectively pre-ordering. The relationship ends on delivery.
Lending	Repayment with interest	A credit product. Regulated as lending.
Equity	A stake in the venture	A securities transaction. Regulated, and the relationship begins rather than ends at funding.

Table 2 The four crowdfunding models and what distinguishes them.

The last row is where this project sits, and the phrase “the relationship begins” is the operative difference. A reward backer’s involvement is complete when the product ships. An equity holder’s interest continues for as long as they hold the stake, which means the platform’s obligations continue too — reporting, communication, and a durable record of what was committed.

The intermediation problem. Early-stage capital allocation is dominated by networks rather than by information [1], [2]. This is not a market failure in the ordinary sense — networks are an efficient screening mechanism for the party who has one — but it produces an outcome in which the availability of capital correlates with prior access rather than with the quality of the venture. A platform that widens the top of the funnel addresses this only if it also supplies the screening that the network was providing. Discovery without evidence relocates the problem rather than solving it.

2.2 Review of Existing Platforms

2.2.1 Reward-Based Platforms

The established reward platforms demonstrated that strangers will fund strangers at scale when the mechanism is credible [3], [4]. Their contribution to this project is the mechanism, not the model: campaign pages, funding targets, progress visibility and update feeds are all patterns worth inheriting.

Their limitations are structural rather than incidental:

- The contributor receives a product, so there is nothing to model as a holding and nothing to report on afterwards.
- Diligence is unsupported — there is no document channel and no verification, because a pre-order does not require one.
- Platform involvement ends at fulfilment.

2.2.2 Equity Platforms

Equity platforms model the holding correctly and support diligence, but they are built for a different participant. Minimum commitments, accreditation requirements and syndicate structures serve institutional and high-net-worth investors [5]. The individual willing to place a small amount is outside their intended market — not excluded by policy, but by product design.

They also tend to treat the platform as a transaction venue rather than an ongoing relationship: the deal closes, and post-funding reporting is left to the founder and the investor to arrange privately.

2.2.3 Regional Context

For a platform intended to serve the regional market, three conditions apply that do not in the markets the incumbents were designed for [1]: payment infrastructure is less uniform, the regulatory perimeter for equity offerings is still developing, and the population of individual investors with prior investing experience is small. The third condition has a direct design consequence — an interface for this market cannot assume familiarity with investment vocabulary, which shapes §8.5.

2.3 Comparative Feature Analysis

Capability	Reward platforms	Equity platforms	Vestora
Public discovery without an account	Yes	Often no	Yes
Search and filter by attribute	Yes	Limited	Yes
Models a holding rather than a purchase	No	Yes	Yes
Administrative review before publication	Varies	Yes	Yes
Document request and grant channel	No	Yes	Yes
In-platform direct messaging	Limited	Varies	Yes
Milestones and post-funding updates	Partial	Rare	Yes
Approved and funded modelled separately	No	Varies	Yes
Accessible to small individual amounts	Yes	No	Yes
Legal execution of the instrument	N/A	Yes	No
Identity and anti-money-laundering checks	Partial	Yes	No

Table 3 Capability comparison. The final two rows are capabilities Vestora does not have.

The last two rows are stated deliberately. Vestora records commitments; it does not execute share instruments and does not perform regulatory onboarding. Both are in §1.5.2 as out of scope and in §2.7 as constraints on any real deployment. A comparison table in which the author's system wins every row is not a comparison.

2.4 Gap Analysis: Where Vestora Differs

Three gaps emerge from the comparison, and each maps to a contribution in §1.6.

1. **The small individual investor is unserved at the equity end.** Reward platforms accept small amounts but do not model a holding; equity platforms model a holding but exclude small amounts. Vestora occupies the intersection.
2. **The post-funding relationship is nobody's product.** Reward platforms end at fulfilment; equity platforms end at close. Milestones, updates and a durable portfolio view (§14.3, §14.6) are the platform's answer.
3. **Commitment state is modelled loosely across the field.** Where a platform displays a single funding figure, it is usually not distinguishing intention from settlement. The separation in §14.1, enforced structurally in §7.6, is the project's most transferable contribution — it is not specific to crowdfunding.

2.5 Stakeholder Analysis

Stakeholder	Interest	Influence on design
Founders	Reach capital; spend minimum time on the platform.	Submission must be short; reporting must be lightweight.
Investors	Find ventures; assess them; know where their money is.	Discovery, evidence channels, and unambiguous commitment state.
Administrators	Keep the platform credible; act defensibly.	Queues, decisive actions, and an audit trail (§14.5).
Regulators	Investor protection; market integrity.	Recorded consent, auditability, and the scope boundary in §2.7.
The platform	Sustainable operation.	A fee on settled transactions only (§2.6).

Table 4 Stakeholders and how each shaped the design.

2.6 Business Model and Revenue Strategy

The platform earns a percentage fee on **settled** transactions, expressed in basis points and held in configuration (§11.8).

Two properties follow from charging on settlement rather than on listing or on commitment. First, incentives align: the platform earns only when a founder actually raises, so there is no revenue in publishing ventures that will not fund. Second, the fee is computed from the same settled-transaction set that funding totals are derived from (§7.6) — revenue and funding cannot disagree, because they are aggregations over the same rows.

The fee amount is stored per transaction rather than recomputed at read time. This is a deliberate exception to the derive-don't-store rule elsewhere in the system, and the reason is that the fee schedule is **time-varying**: recomputing a historical transaction's fee under today's rate would silently rewrite what was actually charged.

Alternatives considered: a listing fee, rejected because it charges founders before any value is delivered and creates an incentive to publish indiscriminately; and a subscription for investors, rejected because it is a barrier at exactly the point the platform is trying to remove one.

2.7 Legal and Regulatory Considerations

Equity crowdfunding sits inside a securities perimeter in most jurisdictions [2]. A deployment of this platform to real transactions would require, at minimum:

- **Authorisation** appropriate to the jurisdiction for facilitating investment in unlisted securities.
- **Identity verification and anti-money-laundering checks** on both sides of a transaction.
- **Risk disclosure** to investors, with recorded acknowledgement.
- **Custody arrangements** for funds in transit and a defined treatment of failed raises.
- **Data protection** compliance for the personal data the platform holds.

None of the first four is implemented. This is stated as a boundary, not minimised: the project is a software engineering exercise that models the mechanics of the domain, and the regulatory surface is documented so that the distance between this system and a deployable one is visible rather than implied.

The design does not obstruct that distance being closed. Recorded consent, auditability (§10.13) and the separation of commitment from settlement (§14.1) are all preconditions for the compliance work, and were built as if it would happen.

2.8 Summary

Reward platforms solved discovery and abandoned the relationship. Equity platforms modelled the relationship and abandoned the small investor. The capability comparison in §2.3 locates Vestora at the intersection, and the gap analysis in §2.4 states what it adds — with two rows, honestly, where it does less.

Methodology and Project Management

3.1 Development Approach

The project used an iterative, incremental approach with fixed-length cycles — Scrum in structure, adapted where the ceremonies did not fit a part-time student team.

Why not waterfall. A sequential approach requires requirements to be substantially correct before design begins. This project's most important requirements — the separation of approval from settlement (FR-28), and of moderation from commercial stage (FR-14) — were **not** known at the outset. They were discovered when their absence produced defects (§18.2, §18.3). A methodology that treats requirement change as failure would have treated the project's two most valuable findings as failures.

What was adapted. Daily stand-ups were replaced by asynchronous written updates, because a six-person team with different timetables cannot reliably meet daily. Sprint length was set at two weeks — short enough to correct course, long enough to complete something demonstrable around lectures and examinations.

What was kept strictly. A demonstrable increment at the end of every cycle. A sprint that produced only partially-integrated work was treated as not having produced anything, because “nearly done” accumulates invisibly and is discovered all at once.

3.2 Development Phases

Work proceeded in phases, and the phase names survive in the migration history (§7.7) — `Phase2_InvestmentPipeline`, `Phase4ProductionAuthSecurity`, `Phase6_VentureLifecycle` — which makes the schema an accidental record of the project plan.

	Phase	Outcome
1	Foundation: data model, identity, venture CRUD.	A working submission-to-listing path.
2	Investment pipeline.	Commitment and approval — and the discovery that the two were being conflated with settlement (§18.2).
3	Discovery: search, filtering, bookmarks, following.	The public surface.
4	Production authentication and security hardening.	Refresh rotation, lockout, audit tables, upload controls.
5	Payments.	The provider abstraction and the funding chain (Section 11).
6	Venture lifecycle and moderation.	The three-column state model (§18.3).
7	Real-time communication.	Messaging, presence, read state (Section 12).
8	Frontend migration.	Angular to React and Next.js (§5.3).
9	Dashboards, analytics and administration.	The surfaces of §14.5–§14.7.

Table 5 Development phases and what each produced.

The ordering is worth noting for one property: **authentication hardening came after the investment pipeline**, not before. In hindsight this was the wrong order — the security model constrains the shape of endpoints, and retrofitting authorisation across an existing surface cost more than building it in would have. Recorded here because a phase plan that is only presented as it was intended teaches less than one presented as it happened.

3.3 Team Structure

The team is six members. Roles were assigned by capability with deliberate overlap, so that no area had exactly one person who understood it.

Area	Responsibility
Backend and data	API, domain services, schema and migrations.
Frontend	Interface implementation, routing, state, integration.
Design system	Tokens, components, accessibility, visual direction.
Payments and security	Provider abstraction, authentication, audit.
Quality and documentation	Test strategy, defect tracking, this document.

Table 6 Areas of responsibility. Team members are listed on the roster page; the mapping is recorded there rather than duplicated here.

The overlap rule. Every area had a primary and a secondary. This cost some efficiency and bought the ability to continue when someone was unavailable — which, on a student timetable with examination periods, was not a hypothetical.

3.4 Version Control and Collaboration

Branching. Work happens on branches named for the change, merged to the main branch through review. The main branch is expected to build.

Review. Every merge is reviewed by someone other than its author. On a team this size the value is less defect-catching than **knowledge distribution** — the overlap rule above is maintained by review more than by assignment.

Commit discipline. Commits describe why rather than what. The diff already shows what changed.

The migration exception. Schema changes are announced before they are applied, because the team shares one database (§18.5). This is the only workflow rule that exists because of an infrastructure constraint rather than a preference, and it is the one most often forgotten.

3.5 Tooling

Purpose		Tool and role
Source control		Git, with review-gated merges to main.
Backend development	develop-	Visual Studio and the .NET CLI.
Frontend development	develop-	Visual Studio Code with the framework toolchain.
Database		SQL Server Management Studio for inspection; EF Core migrations for all changes (§7.7).
API exploration		The generated OpenAPI browser in development (§9.8).
Design		Figma for structure and system passes (§8.4).
Documentation		This document is typeset from plain-text sources in the repository, with diagrams generated from text as well — so both are reviewable in a diff.

Table 7 Tooling and what each is used for.

The last row is a methodology decision rather than a tooling one. Diagrams and prose that live in version control alongside the code can be reviewed, diffed and corrected in the same way as the code — and a diagram that is regenerated from a source file cannot silently drift from the system it describes.

3.6 Risk Management

Risks were identified at the outset and reviewed at phase boundaries. The table below records what was anticipated and what actually happened, which is more useful than a register of risks that never materialised.

Risk	Rating	Outcome
Payment integration proves infeasible without a merchant account.	High	Materialised. Mitigated by the provider abstraction (ADR-02), which made the simulated path a legitimate test surface rather than a stand-in.
Scope exceeds the schedule.	High	Materialised. Managed by explicit prioritisation (§4.4) and a written out-of-scope list (§1.5.2).
A team member becomes unavailable during examinations.	Medium	Materialised. Absorbed by the primary/secondary overlap rule.
Shared database blocks work or breaks under a bad migration.	Medium	Materialised. Managed by process, not solved (§18.5).
Framework migration consumes the remaining schedule.	High	Materialised by choice. Survivable because the API boundary was real (§18.4).
Data loss.	Low	Did not materialise. Provider backups untested by us — a residual risk stated in §7.8.

Table 8 Risk register with actual outcomes. Five of six materialised.

Five of six anticipated risks occurred. That is not a failure of planning — it is what a risk register looks like when the risks were identified honestly rather than chosen for being unlikely.

3.7 Schedule

Work spanned the academic year in the nine phases of §3.2. Two observations about the schedule are worth recording, because both would change how a similar project is planned.

First, **the frontend migration was not in the original plan**. It was decided mid-project on evidence that accumulated after the original plan was written (§5.3), and it consumed schedule that had been allocated to features. Some of those features are in §20.3 as a direct consequence.

Second, **this document was started late**. Architecture Decision Records were reconstructed from commit history and memory rather than written when the decisions were made (§18.8). They are accurate, but they cost several times what they would have cost written at the time — and the reconstruction is only possible because the migration history and commit messages were disciplined enough to reconstruct from.

PART 2

Analysis and Design

From elicited requirements to the technology choices, the architecture, the data model and the interface system built on top of them.

Requirements Engineering

4.1 Elicitation Approach

Requirements came from three sources, weighted differently because they are differently reliable.

Domain analysis The comparative study in Section 2 established what existing platforms do and — more usefully — what they omit. The gaps in §2.4 are the origin of most functional requirements in this chapter.

Structured interviews Conversations with people on both sides of the problem: individuals who had attempted to raise early-stage capital, and individuals who had considered placing money into small ventures. These produced the requirements around **evidence** — documents, milestones, progress — which no amount of competitor analysis would have surfaced, because the competitors do not have them.

Supervisory review Requirements were reviewed against the questions a domain expert asks: what happens when a payment fails, who is allowed to do this, what happens if two people do it at once. Several non-functional requirements exist because of a question of that shape.

Requirements are identified as **FR-*nn*** and **NFR-*nn***, and the identifiers are stable — a requirement that is dropped leaves its number retired rather than reused, so a reference in an older document never silently means something else.

4.2 User Personas

Persona	Situation	What they need from the platform
The founder	Has a working product and no investor network. Time-poor; the platform competes with building the product.	To be found on merit; to answer diligence questions once rather than twenty times; to report progress without assembling a document.
The investor	Willing to place a modest amount early. Cannot access syndicates. Wary of opacity.	To discover ventures by attribute; to see evidence rather than claims; to know what state their money is in; to follow what happens next.
The administrator	Responsible for what appears on the platform and accountable for it.	A queue rather than a firehose; the ability to act decisively; a record of what was done and by whom.

Table 9 The three personas the requirements serve.

4.3 Functional Requirements

The full catalogue is in Section 3. This section presents the requirements grouped by area, with priority expressed as MoSCoW.

4.3.1 Identity and Access

ID	Requirement	Pri.	Realised in
FR-01	A visitor shall register with an email address and password.	M	§10.3
FR-02	The system shall refuse sign-in until the address is verified.	M	§10.3
FR-03	The system shall lock an account after a configured number of consecutive failed sign-in attempts.	M	§10.7
FR-04	A user shall reset a forgotten password through a time-boxed, single-use link.	M	§10.4
FR-05	The system shall issue an access token and a rotating refresh token on successful sign-in.	M	§10.5, §10.6
FR-06	A user shall maintain a public profile.	M	§13.2
FR-07	An administrator shall suspend and reinstate an account.	M	§10.10

Table 10 Identity and access requirements.

4.3.2 Projects and Ventures

ID	Requirement	Pri.	Realised in
FR-08	A founder shall create a venture with a title, description, funding target and imagery.	M	§14.2
FR-09	A venture shall not be publicly visible until an administrator approves it.	M	§14.2, §14.5
FR-10	An administrator shall reject a venture with a stated reason.	M	§14.5
FR-11	A founder shall attach documents with open or restricted visibility.	S	§14.4
FR-12	A founder shall define milestones and publish progress updates.	S	§14.3
FR-13	A founder shall list team members on a venture.	S	§14.2
FR-14	A venture's moderation status, lifecycle status and commercial stage shall be independently maintained.	M	§7.6, §14.2

Table 11 Venture management requirements.

4.3.3 Discovery and Engagement

ID	Requirement	Pri.	Realised in
FR-15	A visitor shall browse approved, active ventures without an account.	M	§13.3
FR-16	A user shall search and filter ventures by attribute.	M	§13.3
FR-17	A user shall save a filter combination as a named search.	S	§13.3
FR-18	A user shall bookmark a venture, at most once.	M	§13.4
FR-19	A user shall follow another user and receive their updates.	S	§13.5
FR-20	A user shall comment on a venture and reply to comments.	S	§13.6
FR-21	An investor shall review a venture at most once.	S	§13.6
FR-22	A user shall report a venture for administrative attention.	M	§13.6

Table 12 Discovery and engagement requirements.

4.3.4 Investment and Funding

ID	Requirement	Pri.	Realised in
FR-23	An investor shall commit an amount to an approved venture.	M	\$14.1
FR-24	A founder or administrator shall approve or decline a commitment.	M	\$14.1
FR-25	An approved commitment shall be settled through a payment provider.	M	\$11.3
FR-26	The system shall record settlement only on verified provider confirmation.	M	\$11.4
FR-27	The system shall process a duplicate provider callback exactly once.	M	\$11.5
FR-28	Funding totals shall be derived from settled transactions only.	M	\$7.6, \$11.7
FR-29	An unresolved checkout shall expire after a configured interval.	M	\$11.8
FR-30	A founder shall not invest in their own venture.	M	\$10.9
FR-31	An investor shall view their pipeline, settlements and portfolio.	M	\$14.6

Table 13 Investment and funding requirements.

4.3.5 Communication

ID	Requirement	Pri.	Realised in
FR-32	Participants shall exchange direct messages.	M	\$12.4
FR-33	The system shall show presence and typing indication.	S	\$12.2, \$12.3
FR-34	The system shall show read state to the sender.	S	\$12.3
FR-35	A participant shall attach an image to a message.	C	\$12.5
FR-36	The system shall notify users of relevant domain events.	M	\$13.7
FR-37	An investor shall request a restricted document; the founder shall grant or refuse.	S	\$14.4

Table 14 Communication requirements.

4.3.6 Administration and Moderation

ID	Requirement	Pri.	Realised in
FR-38	An administrator shall work a queue of ventures awaiting review.	M	\$14.5
FR-39	An administrator shall work a queue of user reports.	M	\$14.5
FR-40	Every administrative action shall be recorded against the acting administrator.	M	\$10.13, \$14.5
FR-41	An administrator shall view platform activity, security events and fee revenue.	S	\$14.5, \$14.7
FR-42	A founder shall view engagement analytics for their ventures.	S	\$14.7

Table 15 Administration and moderation requirements.

4.4 Non-Functional Requirements

Non-functional requirements are stated with a number wherever a number is meaningful. “The system shall be fast” is not a requirement; it is a wish.

ID	Requirement	Assessed in
NFR-01	Public venture pages shall be server-rendered so that primary content is present in the first response.	§17.2
NFR-02	The venture listing shall return within 300 ms at the 95th percentile under expected load.	§17.3
NFR-03	No collection endpoint shall return an unbounded result set.	§9.7
NFR-04	Passwords shall be stored only as salted hashes.	§10.2
NFR-05	Refresh tokens shall be stored only as hashes and rotated on use.	§10.6
NFR-06	All timestamps shall be stored and transmitted in UTC.	§6.7
NFR-07	Monetary values shall use exact decimal representation.	§7.3
NFR-08	Every state-changing endpoint shall enforce authorisation server-side.	§10.8
NFR-09	Error responses shall disclose no internal detail.	§6.7
NFR-10	The application shall refuse to start when a required secret is absent.	§16.6
NFR-11	Uploads shall be limited by size and restricted to an allow-list of verified content types.	§10.12
NFR-12	Security-relevant events shall be persisted to queryable storage, not only to logs.	§10.13
NFR-13	The interface shall meet WCAG 2.1 AA.	§8.7
NFR-14	The interface shall be usable from 360 px to 1920 px wide.	§8.6
NFR-15	Schema changes shall be applied only through versioned migrations.	§7.7
NFR-16	Notification delivery shall not block the request that generated it.	§12.6

Table 16 Non-functional requirements, each with a section that assesses it.

4.5 Use Case Model

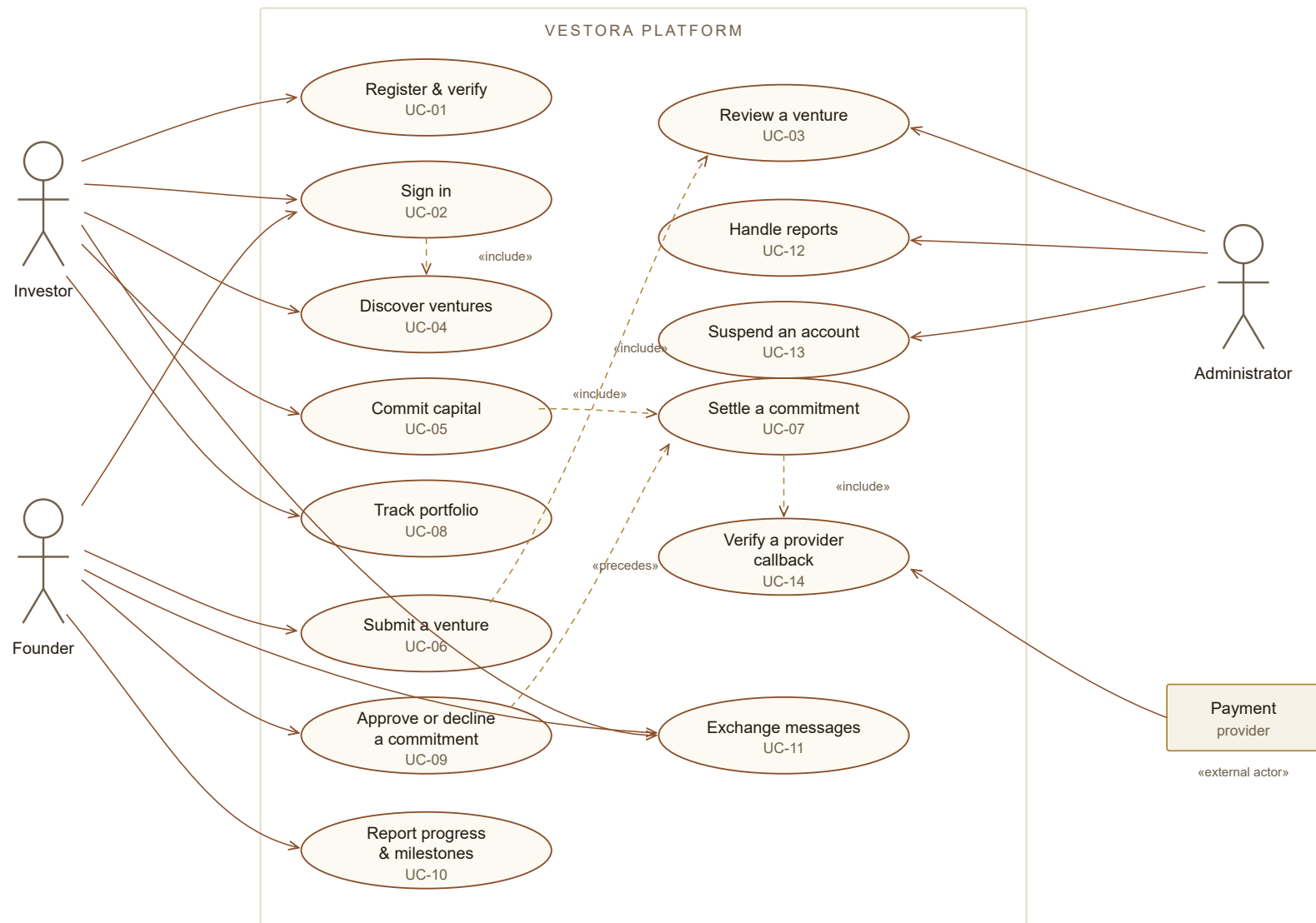


Figure 2 Use case model. Fourteen cases across three human actors and one external one. **UC-05 Commit capital** and **UC-07 Settle a commitment** are deliberately separate cases, not two steps of one — which is the same distinction §14.1 draws in the state model.

The actors are the three personas of §4.2 plus the payment provider, which is a non-human actor that initiates the settlement callback. Modelling it explicitly matters: the callback is an inbound interaction the platform does not control the timing of, and treating it as an actor rather than as an internal step is what surfaces the requirements FR-26 and FR-27.

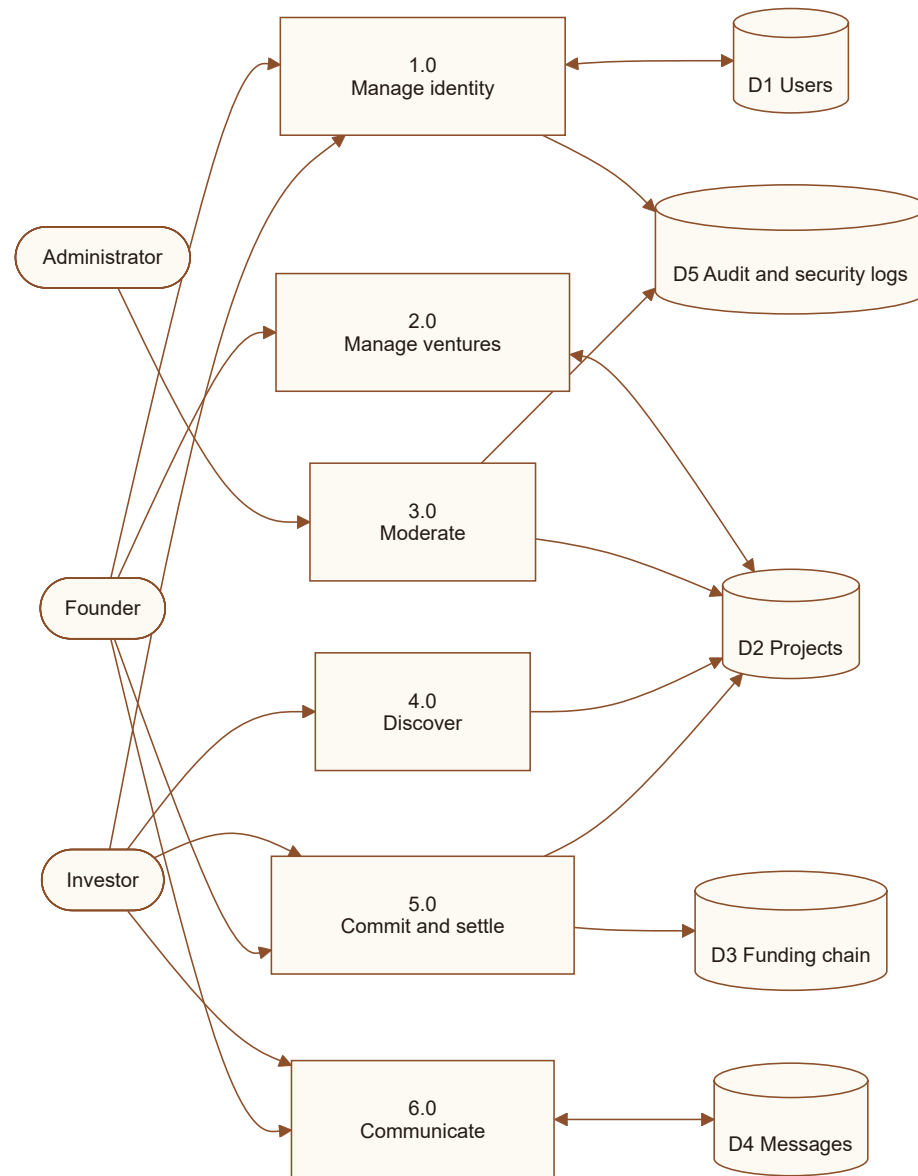


Figure 3 Level-1 data flow. Six processes, five stores, three external actors. The funding chain is a single store because \$11.1 treats its four tables as one transactional unit.

The use case diagram and its full specifications are held in Section 3 with the requirement catalogue, so that a reader checking a requirement finds its use case beside it rather than in a different chapter.

4.6 Use Case Specifications

Two specifications are given here because their alternate flows are where the system's difficulty lives. The remainder are in Section 3.

UC-07 — SETTLE AN APPROVED COMMITMENT

Actors. Investor (primary), payment provider (secondary).

Preconditions. The commitment exists and is approved; the venture is approved and active; no settled transaction exists for this commitment.

Main flow.

1. The investor opens checkout.
2. The system validates the preconditions and creates a funding request with an expiry.
3. The system requests a checkout session from the configured provider.
4. The investor completes payment on the provider's surface.
5. The provider posts a callback.
6. The system verifies the signature, records the event, and marks the transaction settled.
7. Funding totals recompute (§11.7).

Alternate flows.

- **3a.** The provider is unavailable — the funding request is abandoned and the commitment remains approved.
- **5a.** No callback arrives before expiry — the sweeper lapses the request (§11.8).
- **6a.** Signature verification fails — the payload is rejected and logged; no state changes.
- **6b.** The event has already been recorded — the callback is acknowledged and no state changes (FR-27).

Postcondition. Either exactly one settled transaction exists for the commitment, or none does.

UC-03 — REVIEW A SUBMITTED VENTURE

Actor. Administrator.

Preconditions. The venture's moderation status is pending.

Main flow.

1. The administrator opens the review queue and selects a venture.
2. The administrator approves it.
3. Moderation status becomes approved; lifecycle status and commercial stage are unchanged.
4. The action is written to the audit log against the administrator.
5. The venture becomes publicly visible if its lifecycle status is active.

Alternate flows.

- **2a.** The administrator rejects with a reason; the founder is notified and may resubmit.
- **3a.** A resubmission returns moderation status to pending **without** affecting commercial stage (FR-14) — the defect this separation exists to prevent is described in §14.2.

Postcondition. The venture's moderation status is approved or rejected, and the decision is attributable.

4.7 Requirements Traceability

Every requirement in this chapter carries a “realised in” column, and every one appears again in the traceability matrix in Section 3, mapped forward to its design section, its implementation and its test case.

The purpose is checkability. A requirement with no design section is unimplemented; one with no test is unverified. The matrix makes both visible rather than requiring a reader to take the claim on trust — and §19.2 reports what it shows, including where it shows a gap.

Technology Stack and Engineering Decisions

5.1 Decision Framework and Evaluation Criteria

A technology chapter that lists what was used explains nothing. The list is visible in the project file; what is not visible is why each item is there rather than its alternative, and what was given up in choosing it. This chapter records the second thing.

Every decision below is evaluated against the same five criteria, weighted once and applied consistently. The weights come from the requirements in Section 4, not from preference.

Criterion	Weight	Why it carries this weight
Fit to requirement	30%	A choice that does not serve a stated requirement is not a trade-off, it is a mistake. Weighted highest by definition.
Team capability	25%	A small undergraduate team on a fixed timetable. A technically superior option the team cannot use well is not superior.
Ecosystem maturity	20%	Documentation, community answers and library availability determine how much of the schedule is spent on the problem rather than on the tool.
Operational cost	15%	Hosting, licensing and the effort of running the thing after it is written.
Exit cost	10%	What it would take to replace this choice later. Low exit cost is worth paying a little for.

Table 17 Evaluation criteria applied to every technology decision in this chapter.

Scores are on a one-to-five scale. Where a score is close, the narrative explains the tie-break; where an option lost decisively, that is stated plainly rather than softened.

NOTE

Two decisions in this chapter were revised during the project rather than settled at the start: the frontend framework (§5.3) and the payment integration strategy (§5.7). Both are documented as they actually happened, including the cost of the change.

5.2 Frontend Framework: React with Next.js

Requirement pressure. The platform has public pages that must be indexable and fast on first load — the venture listing and the venture detail page are its shop window — and authenticated application surfaces where first-load cost matters less than interaction quality. A framework that serves only one of those two well would force a compromise on the other.

Criterion	React + Next.js	Angular	Vue + Nuxt
Fit to requirement (30%)	5	3	4
Team capability (25%)	4	3	2
Ecosystem maturity (20%)	5	4	3
Operational cost (15%)	3	4	4
Exit cost (10%)	3	2	3
Weighted total	4.20	3.25	3.20

Table 18 Frontend framework evaluation.

Angular’s loss is not about quality. It is a coherent, opinionated framework with first-class dependency injection and a structural discipline that suits large teams — and it was, in fact, what the first version of this project was built on. It scored lower here on **fit to requirement**: its server-rendering story requires additional infrastructure, and the component ecosystem available to us was Angular Material, whose visual opinions run against the interface direction described in Section 8.

Vue with Nuxt scored well on fit but poorly on team capability: nobody on the team had used it, and §5.1 weights that at a quarter of the decision.

What was given up. React is a library, not a framework, so structural discipline is not enforced by the tool. Angular would have supplied it for free. The mitigation is described in §6.5: an enforced module layout and lint rules that make the intended boundaries mechanical rather than cultural.

5.3 Case Study: Migrating from Angular to React

This project has an advantage over most framework comparisons: it did not choose between the two on paper. It shipped one, then moved to the other.

The starting position. The initial implementation used Angular with Angular Material and a conventional client-rendered single-page application. It worked. The decision to move was not driven by a defect.

What forced the reassessment. Three pressures accumulated.

1. **The public surface was slow to first paint.** Venture listing and detail pages are the discovery surface. A client-rendered application fetches the shell, then the framework, then the data — so the content a first-time visitor came for arrives last.
2. **The component library fought the design direction.** Angular Material encodes a specific visual language. Overriding it comprehensively costs more than building on unstyled primitives, and produces a result that reads as “Material with the corners filed off”.
3. **Iteration speed on the interface was the bottleneck.** The interface quality bar for this project is high (Section 8), which means many rounds of visual revision. The cost per round mattered more than any single technical attribute.

The decision. Migrate the frontend to React with Next.js, retaining the existing API unchanged. The API boundary made this possible: the backend had no knowledge of the frontend framework, so the migration replaced one container in the sense of Section 6, not the system.

ADR-01

Replace the Angular frontend with React and Next.js

CONTEXT The public discovery surface was client-rendered and slow to first paint; the component library resisted the intended visual direction; interface iteration was the schedule bottleneck.

DECISION Rewrite the frontend as a Next.js application using React, with Tailwind CSS and unstyled Radix-based primitives. Leave the REST API untouched.

CONSEQUENCES First paint on public pages improves and those pages become server-renderable. Interface iteration accelerates. In exchange the team gives up Angular's enforced structure and absorbs a full frontend rewrite mid-project; structural discipline moves from the framework into lint rules and an enforced module layout.

What it cost. A complete reimplementing of every screen, and a period during which the team was learning a new rendering model — server components, the boundary between server and client code, and data fetching that no longer looked like a service injected into a component. The clean API boundary limited the blast radius, but the cost was real and is reported in Section 18 rather than presented as a free upgrade.

What it bought. Public pages render on the server. The interface is built on primitives that impose no visual opinion. And the team ended the project able to speak to both frameworks from experience rather than from documentation — which is the reason this section exists at all.

5.4 Backend Platform: ASP.NET Core

Requirement pressure. The backend carries money-adjacent invariants, role-based authorisation across every state-changing endpoint, and a real-time transport. Static typing and a mature authorisation story were therefore weighted heavily inside **fit to requirement**.

Criterion	ASP.NET Core	Node.js + Ex- press	Spring Boot
Fit to requirement (30%)	5	3	5
Team capability (25%)	5	3	2
Ecosystem maturity (20%)	4	5	5
Operational cost (15%)	4	4	3
Exit cost (10%)	3	4	3
Weighted total	4.45	3.65	3.75

Table 19 Backend platform evaluation.

Three properties decided it. First, the type system is enforced at compile time, and the invariants in Section 11 are the kind that benefit from being unrepresentable rather than merely tested. Second, authorisation, model validation, dependency injection and configuration are first-party concerns rather than assembled from packages, which matters for a team that cannot afford to spend its schedule integrating middleware. Third, SignalR (§5.8) belongs to the same platform, so the real-time transport did not introduce a second runtime.

Spring Boot scored equally on fit and higher on ecosystem, and lost only on team capability — the single largest avoidable risk on a fixed timetable. Node.js with Express scored well on ecosystem and exit cost but poorly on fit: the guarantees this system needs would have to be reconstructed at runtime rather than being available at compile time.

5.5 Data Layer: SQL Server with EF Core

The data is relational and the invariants are relational. A venture has investments; an investment belongs to exactly one investor and one venture; a payment transaction settles exactly one funding request. The integrity rules in §7.6 are foreign keys, check constraints and unique indexes — the

native vocabulary of a relational database. A document store would require these to be enforced in application code, which is precisely the arrangement this project set out to avoid.

Criterion	SQL Server	PostgreSQL	MongoDB
Fit to requirement (30%)	5	5	2
Team capability (25%)	5	3	3
Ecosystem maturity (20%)	5	5	4
Operational cost (15%)	3	5	4
Exit cost (10%)	3	4	2
Weighted total	4.40	4.40	2.90

Table 20 Database evaluation. SQL Server and PostgreSQL tie on the weighted score.

The tie-break. SQL Server and PostgreSQL score identically. PostgreSQL is cheaper to host and has a lower exit cost; SQL Server is more familiar to the team and integrates more smoothly with the .NET tooling the rest of the project already uses. The decision went to SQL Server on **hosting availability**: the deployment target (§16.2) offers managed SQL Server, and eliminating a separately administered database server removed an operational risk the team was not staffed to carry.

This is worth stating clearly because it is the weakest-justified choice in the chapter. Had hosting been neutral, PostgreSQL would have been at least as good a decision.

EF Core was chosen over Dapper and raw ADO.NET for the migration history alone. The schema evolved across more than twenty migrations (§7.7); reproducing that as hand-written, hand-ordered SQL would have consumed schedule with no compensating benefit. The cost is the usual one — generated queries need supervision, and §17.4 documents the places where a generated query had to be replaced by an explicit projection.

5.6 Authentication Strategy: JWT with Refresh Tokens

Requirement pressure. The frontend and the API are separate origins, and a persistent WebSocket must authenticate against the same identity as the REST calls. A cookie-session model tied to a single origin fits neither cleanly.

The chosen design issues a short-lived access token carrying the user’s identity and role, paired with a longer-lived refresh token that is rotated on use. The lifetimes are configuration, not constants in code:

Setting	Value	Reasoning
Access token lifetime	60 minutes	Short enough that a leaked token expires within a session; long enough to avoid refresh churn.
Refresh token lifetime	14 days	A fortnight of inactivity ends the session. Rotated on every use, so a replayed refresh token is detectable.
Email verification token	60 minutes	Single-use and time-boxed.
Password reset token	10 minutes	Deliberately shorter: a reset link sitting in an inbox is a higher-value target than a verification link.
Lockout threshold	5 attempts / 15 minutes	Blunts credential stuffing without locking out a legitimate user who mistypes.

Table 21 Authentication lifetimes, all supplied by configuration.

What was given up. A stateless access token cannot be revoked before it expires. An account suspended one minute into a token's life retains API access until minute sixty unless something else intervenes. The mitigation — checking account state on the paths where it matters rather than trusting the token alone — is described in §10.9, and the residual exposure is stated there rather than hidden here.

5.7 Payment Integration Strategy

This section documents a decision that is frequently misdescribed, so it is stated precisely.

The requirement is not “integrate a payment provider”. It is: the platform's definition of a valid commitment must not be a property of whichever provider happens to be configured. If the rules for what counts as a funded investment live inside an integration, then changing provider changes the domain — and testing the domain requires the provider.

The decision was therefore to introduce a provider interface that carries transport only, and to keep every domain rule in the platform's own service layer.

```

1 public interface IPaymentProvider
2 {
3     /// "stripe" or "simulated" - stored on every transaction it handles.
4     string Name { get; }
5
6     Task<ProviderCheckout> CreateCheckoutAsync(
7         ProviderCheckoutRequest request, CancellationToken ct = default);
8
9     Task<ProviderPaymentResult> GetSessionResultAsync(
10        string sessionId, CancellationToken ct = default);
11
12    Task<ProviderRefundResult> RefundAsync(
13        string providerPaymentId, decimal amount, string currency,
14        CancellationToken ct = default);
15 }

```

Listing 1 The provider boundary. It exposes no domain concept — no venture, no investor, no funding target — only the three operations a payment processor performs.

Two implementations satisfy it. A **simulated** provider, which is the default and resolves checkouts locally. And a **Stripe-compatible** provider, which speaks HTTP to Stripe's API and whose webhook callbacks are signature-verified by a hand-implemented verifier rather than by a vendor SDK. Selection is by configuration at startup.

ADR-02

Payment rules live in the domain, not in the provider integration

CONTEXT Funding integrity is the system's most important invariant. If the rules governing it live inside a provider integration, they cannot be tested without that provider, and they change when the provider changes.

DECISION Define `IPaymentProvider` with transport operations only. Hold every rule — what may be committed, when a commitment becomes funded, how totals are derived — in the platform's own service layer. Select the implementation by configuration.

CONSEQUENCES The simulated path exercises the same domain rules as the Stripe path, so it is a legitimate test surface rather than a stand-in. Adding a provider cannot alter the meaning of a funded investment. The cost is an extra indirection and a set of provider-neutral result types that must be mapped on both sides.

NOTE

Stated plainly: the platform runs against the simulated provider by default and has not been operated against a live merchant account. The Stripe-compatible implementation, including webhook signature verification and idempotent event handling, is written and configurable. Section 11 sets out exactly which paths have been exercised and which have not. This document does not claim a live payment integration.

Why Stripe as the target provider. Among providers with documented sandbox environments, Stripe’s webhook signature scheme and session model are the best documented and the most widely reproduced — which matters when the verifier is implemented directly rather than taken from an SDK. PayPal’s flow is harder to model behind a narrow interface. Regional gateways were considered for eventual deployment (§2.7), but none offers a sandbox that supports development at this stage.

5.8 Real-Time Transport: SignalR

Messaging, presence, typing indicators and read receipts all require the server to push. The alternatives were polling, raw WebSockets, and server-sent events.

Polling was rejected on cost: presence and typing indicators are only useful at sub-second resolution, and polling at that interval is wasteful at any user count. Server-sent events are unidirectional, which covers notifications but not typing indicators. Raw WebSockets would have required implementing connection lifecycle, reconnection with backoff, and transport fallback by hand.

SignalR provides those, negotiates transport automatically, and — decisively — shares the ASP.NET Core authentication pipeline, so a hub connection authenticates against the same token as a REST call with no second mechanism. Section 12 documents the hub design that follows.

5.9 Transactional Email Delivery

The platform sends four kinds of message: email verification, one-time passwords, password reset links, and notification mail. None is marketing email; all are transactional, low-volume and latency-sensitive at the point of sign-up.

Criterion	SMTP via MailKit	Managed email API	Platform built-in
Fit to requirement (30%)	4	5	2
Team capability (25%)	5	3	3
Ecosystem maturity (20%)	5	4	3
Operational cost (15%)	5	2	4
Exit cost (10%)	5	3	2
Weighted total	4.65	3.75	2.70

Table 22 Email delivery evaluation.

The implementation sends over SMTP using MailKit, behind an `IMailService` interface so the transport can be replaced without touching the call sites. Host, port, credentials and sender identity are configuration; nothing is compiled in.

A managed transactional email API scores higher on **fit** — it provides deliverability reporting, bounce handling and domain authentication that raw SMTP does not — and would be the correct choice for production at volume. It lost here on operational and exit cost: it requires an account, a verified sending domain and a billing relationship, none of which a university project can rely on having, and it introduces a dependency that cannot be run offline.

ADR-03**Send transactional mail over SMTP behind an interface**

CONTEXT Four transactional message types are required. Managed email APIs offer better deliverability but require a verified domain and a billing relationship the project cannot guarantee.

DECISION Implement `IMailService` over SMTP using MailKit, with all connection details in configuration.

CONSEQUENCES No external account is required and development can run against a local relay. Deliverability reporting and bounce handling are unavailable, and a production deployment at volume would need to swap the implementation — which the interface reduces to a single-class change.

5.10 UI Layer: Tailwind CSS with shadcn/ui

The interface direction in Section 8 requires a specific visual system: a warm, editorial palette, a display face used sparingly, and a chamfered plate motif that exists in no component library.

Libraries with strong visual opinions — Angular Material, MUI, Bootstrap — start from a design language and require it to be overridden. The chosen approach inverts this. Radix primitives supply behaviour and accessibility — focus management, keyboard interaction, ARIA semantics — with no styling at all, and Tailwind applies the project's own tokens on top. shadcn/ui provides these as source files copied into the repository rather than as a dependency, so a component can be modified directly instead of fought with.

The cost is that the project owns its component code, including its bugs. The benefit is that accessibility comes from the primitives rather than being retrofitted — which is what makes the compliance claims in §8.7 defensible.

5.11 Hosting and Infrastructure

The deployment target must run an ASP.NET Core application and a SQL Server database, be affordable indefinitely rather than for a trial period, and be operable by a team with no infrastructure engineer.

A managed cloud platform would score higher on capability and considerably lower on operational cost and team capability: it introduces identity management, networking and billing surfaces that are themselves a project. The chosen host provides managed ASP.NET Core and SQL Server together, which removes database administration entirely. The trade — no autoscaling, no managed CI, limited observability — is accepted, and its consequences are documented in Section 16 and §17.7 rather than described as a non-issue.

5.12 Decision Summary

Layer	Chosen	Principal trade-off accepted
Frontend	React with Next.js	No framework-enforced structure; replaced by lint rules and an enforced module layout.
Backend	ASP.NET Core	Narrower hosting market than Node.js.
Database	SQL Server with EF Core	Higher hosting cost than PostgreSQL; generated queries need supervision.
Identity	JWT with rotating refresh tokens	Access tokens cannot be revoked before expiry.
Payments	Provider abstraction, simulated by default	An extra indirection and provider-neutral result types on both sides.
Real time	SignalR	Ties the real-time layer to the ASP.NET Core runtime.
Email	SMTP via MailKit	No deliverability reporting or bounce handling.
UI	Tailwind with Radix primitives	The project owns its component code.
Hosting	Managed ASP.NET Core and SQL Server	No autoscaling, no managed CI, limited observability.

Table 23 Summary of technology decisions and the cost accepted for each.

Every row of that table has a cost column, and none of them is empty. A decision with no stated cost has not been examined.

System Architecture

6.1 Architectural Goals and Constraints

The architecture serves five goals, in this order of precedence. Where two conflict, the earlier wins.

1. **Correctness about money and state.** The invariants in Section 7 and Section 11 must be structurally difficult to violate, not merely tested.
2. **A single writer per piece of state.** Every field with rules attached has one component responsible for changing it.
3. **Substitutability at the edges.** External services — payment, email — must be replaceable without touching domain code.
4. **Comprehensibility by a small team.** An architecture nobody can hold in their head produces defects faster than it prevents them.
5. **Delivery within a fixed timetable.**

Three constraints bounded the design. The team is small and undergraduate. The hosting target offers no autoscaling and no managed queue (§16.2). And the frontend was rewritten mid-project (§5.3), so the API boundary had to be genuinely independent of the client rather than nominally so.

6.2 Architectural Style and Rationale

The system is a **layered monolith behind a REST API**, with a separate rendering application for the client.

This deserves defending, because a distributed architecture is the more fashionable answer. It was rejected on all five goals above. Splitting a system of this size into services would distribute the funding invariant across a network boundary, which converts a database constraint into a consistency problem. It would multiply the operational surface on a host that provides no service mesh, no managed queue and no distributed tracing. And it would spend the team's schedule on infrastructure rather than on the domain.

ADR-06

Build a layered monolith rather than a service-oriented system

CONTEXT The system's most important invariants are relational and transactional. The hosting target offers no queue, no service discovery and no distributed tracing. The team has no infrastructure engineer.

DECISION Build a single deployable API in layers — controllers, services, data — with the client as a separate rendering application over HTTP.

CONSEQUENCES Funding integrity stays inside one transactional boundary and can be enforced by database constraints. Deployment is one artefact. The cost is that the API scales as a unit and a fault in one area shares a process with every other; §17.7 discusses what would have to change to split it later.

The monolith is **layered**, not merely single-process. The layering is what makes the eventual extraction of a service possible, and it is enforced by dependency direction: controllers depend on services, services depend on the data context, and nothing depends upward.

6.3 System Context and Containers

The container view in §1.7 shows four internal containers and two external dependencies. Their responsibilities are:

Container	Responsibility and boundary
Web application	Renders the interface. Holds no domain rules. Calls the API for every read and write, and opens a WebSocket to the hub for live updates.
REST API	The only writer to the relational store. Owns all domain rules, authorisation and validation.
Real-time hub	Pushes messages, presence, typing and read state. Shares the API's authentication and its data context.
Relational store	Holds state and enforces the integrity rules in §7.6.
Payment provider	External. Reached only through <code>IPaymentProvider</code> (§11.2).
SMTP relay	External. Reached only through <code>IMailService</code> (§5.9).

Table 24 Container responsibilities and boundaries.

The important property of this table is what is **absent** from the web application's row. It holds no domain rules. A funding total is never computed in the browser; an authorisation decision is never made there. The client renders decisions the API has already made, which is why a complete frontend rewrite (§5.3) did not touch a single business rule.

6.4 Component View

Within the API, components are grouped by responsibility rather than by entity. The grouping below is the actual shape of the codebase.

Controllers (22) One per resource area — authentication, projects, investments, payments, messages, notifications, follows, bookmarks, reports, admin moderation, admin revenue, founder and investor dashboards, insights, and so on. Controllers parse, authorise and delegate. They contain no rules.

Domain services The rules. `AuthService` owns identity; `PaymentService` owns the payment lifecycle; `FundingMath` owns the definition of what a venture has raised; `PipelineStages` owns valid stage transitions; `AccountRules` owns what a given account is permitted to be.

Infrastructure services `FileUploadSecurityService` validates uploads; `PresenceTracker` holds connection state; `NotificationFanOutQueue` and `NotificationFanOutWorker` move notification delivery off the request path; `MailKitEmailService` sends mail.

Data `AppDbContext` and the entity model. The only component that talks to the database.

The separation between the first two service groups matters. A domain service can be reasoned about without any infrastructure: `FundingMath` is arithmetic over rows, and `PipelineStages` is a transition table. Neither performs I/O, which is what makes them testable without a database.

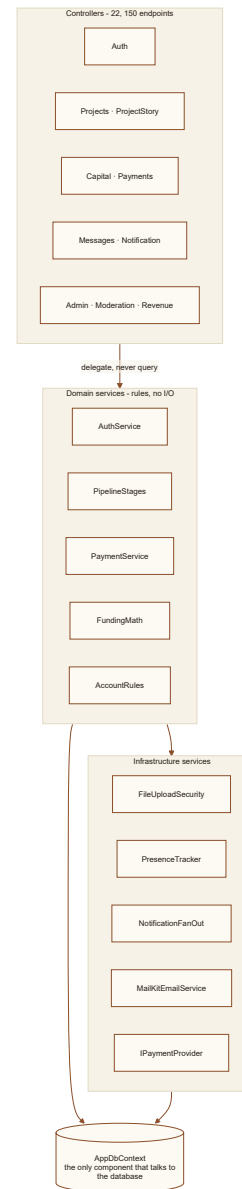


Figure 4 Component view of the API. Controllers delegate and never query; domain services hold the rules and perform no I/O; only **AppDbContext** reaches the database.

6.5 Backend Layering

6.5.1 Controllers

A controller does four things and nothing else: bind the request, check authorisation, call a service, and shape the result. The last of these is handled uniformly rather than per-controller — a shared extension translates a service result into an HTTP response, so the mapping from “not found” to 404 exists once rather than twenty-two times.

This is the discipline that the old version of this project lacked, and its absence is what produced controllers that talked to the data context directly. The rule now is mechanical: **a controller may not reference `AppDbContext`**.

6.5.2 Services

Services return a result object rather than throwing for expected outcomes. An investment that cannot be created because the venture is closed is not an exceptional condition — it is an ordinary answer, and modelling it as an exception both costs performance and obscures the control flow.

```
1 // A service reports outcome in its return type; the controller maps it.
2 var result = await _payments.OpenCheckoutAsync(investmentId, userId, ct);
3 return result.ToActionResult(this);
```

Listing 2 The controller-to-service seam. Outcome mapping happens in one place for the whole API.

6.5.3 Data Access

The data context is reached only from services. Queries that feed list pages project directly into DTOs rather than materialising entities, so a listing query returns the eight columns the card needs rather than the full row and its navigation properties. §17.4 reports what this measured.

6.6 Frontend Architecture

6.6.1 Routing and Segments

The client uses file-based routing over sixty-five routes, grouped into segments that correspond to access level rather than to feature: public pages (landing, venture listing, venture detail, legal), authentication pages, and an authenticated application group containing the dashboards, investment surfaces, messages, settings and the administrative area.

Grouping by access level rather than by feature means the authorisation boundary is visible in the directory structure. An administrative page cannot be added in the wrong place without it being obvious in review.

6.6.2 Data Fetching

Public pages render on the server so that their content is present in the first response (§5.2). Authenticated surfaces fetch on the client, because their content is user-specific and not cacheable, and because they are behind a navigation the user has already paid the load cost for.

6.6.3 State Management

There is no global client state store. This is deliberate. Server state — the list of ventures, a portfolio, a conversation — belongs to the server and is fetched, not mirrored. The state that genuinely lives on the client is small: form state, the open or closed state of overlays, and the live connection status. Introducing a global store would create a second copy of the truth, with the synchronisation problem that follows.

6.7 Cross-Cutting Concerns

6.7.1 Validation

Request validation is declarative and runs before a controller body executes, so a service can assume its input is structurally valid. This separates **structural** validation — a required field is present, an amount is positive — from **domain** validation, which is the service's responsibility and which depends on state the validator cannot see.

6.7.2 Error Handling

Expected outcomes are results (§6.5). Unexpected ones are exceptions, handled centrally, logged, and returned to the client as a generic failure. The client never receives a stack trace or an internal message, because an error response is an information disclosure surface (§10.11).

6.7.3 Logging

Logging is structured and levelled. Security-relevant events — failed sign-in, logout, password reset, administrative action — are additionally written to durable tables (`SecurityLog` , `AdminAuditLog`) rather than to the log stream alone, because a log stream is not queryable evidence.

6.7.4 Time and Serialisation

Every timestamp is stored and transmitted in UTC. This sounds like a convention until it is violated: a serialiser that emits a local-time string without an offset produces a value the client silently re-interprets in its own zone, and the resulting bug is invisible in any environment where the two zones agree.

The system applies an explicit UTC converter at the serialisation boundary so that the format is a property of the API rather than of whichever machine serialised the response. Section 18 describes the incident that made this necessary.

ADR-07

Force UTC at the serialisation boundary rather than by convention

CONTEXT A timestamp serialised without an explicit offset is re-interpreted by the client in its own timezone.

The defect is invisible wherever server and client zones agree, which includes every developer machine.

DECISION Apply an explicit UTC converter in the serialisation pipeline so every emitted timestamp carries its offset, independent of host configuration.

CONSEQUENCES Timestamps are unambiguous across environments and the defect class cannot recur.

Every stored value must be UTC at the point of write, which is a discipline the data layer must maintain.

6.8 Design Patterns Applied

Pattern	Where, and what it buys
Strategy	<code>IPaymentProvider</code> (§11.2). The payment algorithm varies by configuration without the caller knowing which is in use.
Data transfer object	Every API boundary. Entities are never serialised directly, so a schema change cannot silently alter the public contract.
Result object	The service seam (§6.5). Expected failure is a value, not an exception.
Producer-consumer	<code>NotificationFanOutQueue</code> with a background worker. Notification delivery leaves the request path, so a slow fan-out cannot slow a write.
Repository-like projection	List queries project to DTOs inside the service rather than returning entities for the controller to map.

Table 25 Patterns used, and the specific problem each solves.

The table deliberately omits patterns the system does not use. There is no mediator, no event bus and no generic repository abstraction over EF Core — each was considered and rejected as indirection without a corresponding problem. EF Core’s `DbSet` is already a repository; wrapping it produces a second, worse one.

6.9 Architecture Decision Records

Ten decisions are recorded in this document, each at the point where its consequences are discussed rather than collected into a chapter of their own.

ADR	Decision	Section
01	Replace the Angular frontend with React and Next.js	§5.3
02	Payment rules live in the domain, not in the provider integration	§5.7
03	Send transactional mail over SMTP behind an interface	§5.9
04	Funding totals are derived, never stored	§7.6
05	Model moderation, lifecycle and commercial stage as independent columns	§7.6
06	Build a layered monolith rather than a service-oriented system	§6.2
07	Force UTC at the serialisation boundary rather than by convention	§6.7
08	Hold presence in memory, persist last-seen	§12.2
09	Move notification fan-out off the request path	§12.6
10	Fail to start on a missing secret rather than falling back	§16.6

Table 26 Index of Architecture Decision Records.

Each record states its context, the decision, and its consequences — including the cost. A record whose consequences are entirely positive has not been written honestly.

6.10 Deployment View

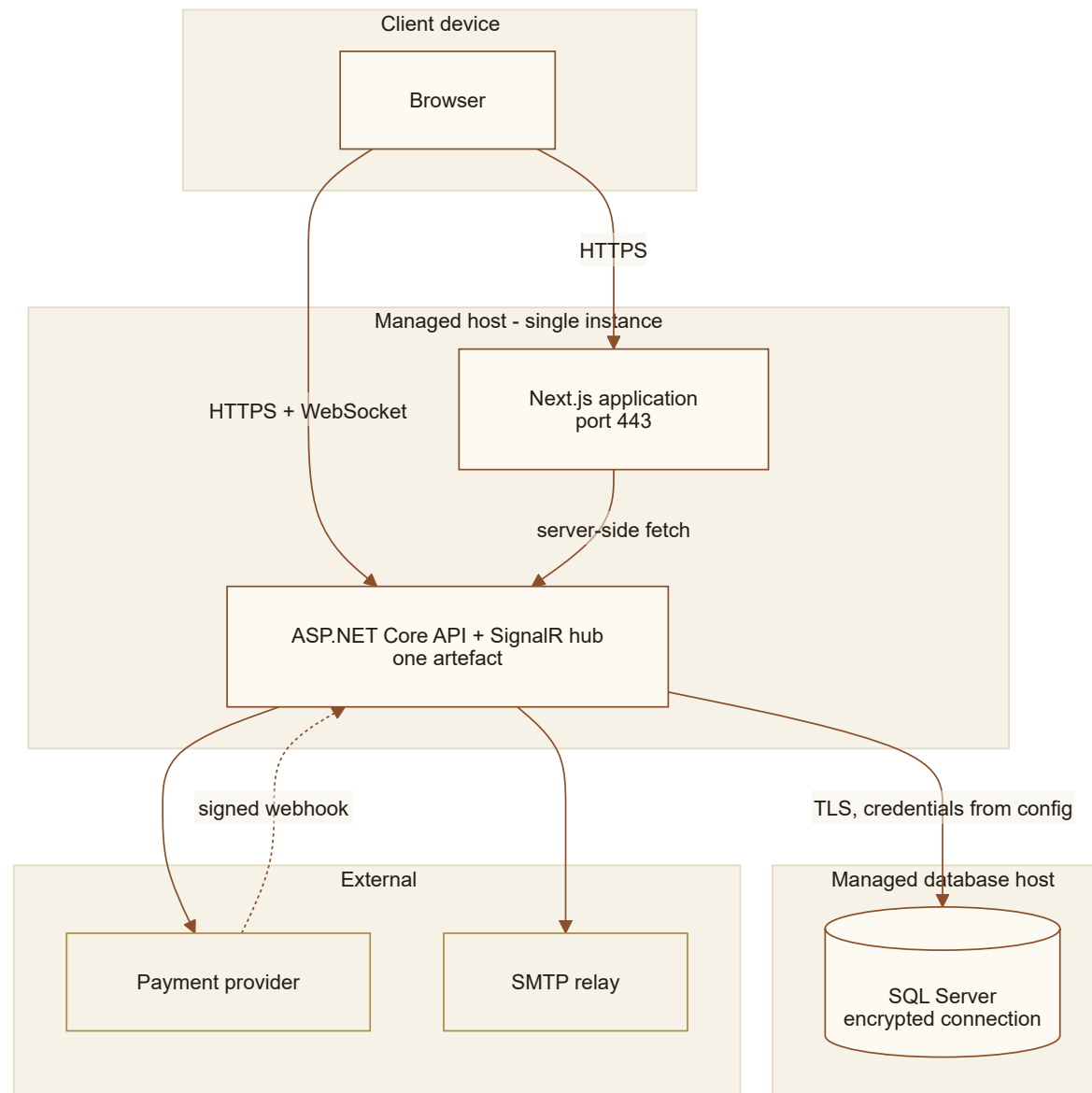


Figure 5 Deployment view. One instance of each container; the payment provider's callback is the only inbound path the platform does not initiate.

The API and the hub deploy as one artefact to a managed ASP.NET Core host. The web application deploys separately. The database is a managed SQL Server instance reached over an encrypted connection whose credentials are supplied by environment configuration and never committed (§16.6). There is exactly one production instance of each. There is no load balancer, no autoscaling group and no read replica — and stating that plainly is more useful than describing an architecture the project does not have. §17.7 sets out which of those would be needed first, and at what point.

Database Design

7.1 Conceptual Data Model

Four nouns carry the platform. Everything else hangs off one of them.

User An account. A user may own ventures, commit capital, or both. The administrative role is separate and granted rather than self-selected.

Project A venture. It is the unit that is submitted, moderated, published, discovered and funded, and it owns the material that describes it — images, documents, team members, milestones and updates.

Investment A commitment by a user to a venture. It is deliberately **not** a payment: it is an intention that has been recorded and may or may not have been approved, and may or may not have settled.

Payment transaction Evidence that money moved. It settles a funding request, which in turn belongs to an investment.

The separation between the last two is the most consequential modelling decision in the system, and §7.6 explains why it is enforced structurally rather than by convention.

Inheritance. `Investor` and `Innovator` are specialisations of `User`, persisted as table-per-hierarchy with a `UserType` discriminator column. The alternative — separate tables joined to a shared identity — was rejected because the two specialisations share almost every column and differ mainly in navigation properties. Table-per-hierarchy keeps authentication, which does not care which specialisation an account is, to a single-table read.

7.2 Entity Relationship Diagram

The diagram below shows the core of the model: identity, the venture and its funding chain. Peripheral tables — notifications, moderation audit records, saved searches, analytics counters — are omitted here for legibility and are listed in full in Section 2.

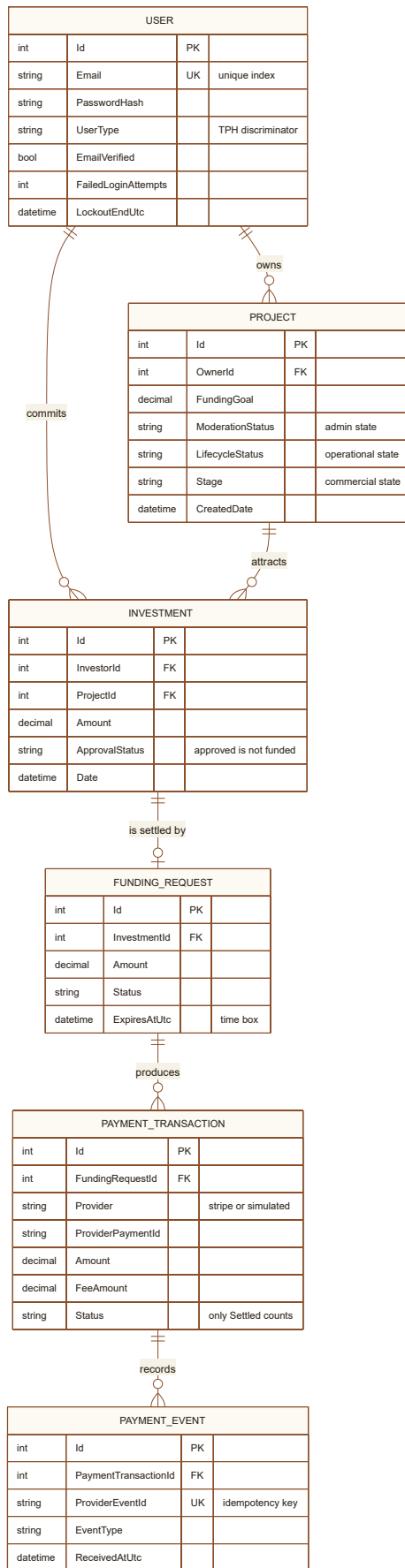


Figure 6 The funding chain, derived from the Entity Framework configuration in `AppDbContext`. Identity and the venture sit at the top; each step down holds a fact the step above cannot.

The entities that **hang off** a venture or a user — images, documents, team members, milestones, updates, reviews, reports, bookmarks, follows, messages and their attachments — are deliberately not drawn here. Rendered as a diagram they form a flat fan of eighteen edges that conveys only “these belong to that”, which the delete-behaviour table in §7.6 states more precisely and the full schema in Section 2 states completely. A diagram that a table replaces has not earned its page.

Two shapes in that diagram are worth reading carefully.

First, the chain `Investment → FundingRequest → PaymentTransaction → PaymentEvent` is four hops rather than one

column on the investment. Each hop exists because it holds information the previous one cannot: a funding request has its own expiry, a transaction has a provider and a fee, and an event has a provider event identifier that makes replay detectable.

Second, `Project` carries **three** state columns — `ModerationStatus`, `LifecycleStatus` and `Stage` — not one. §7.6 explains why collapsing them would be a defect rather than a simplification.

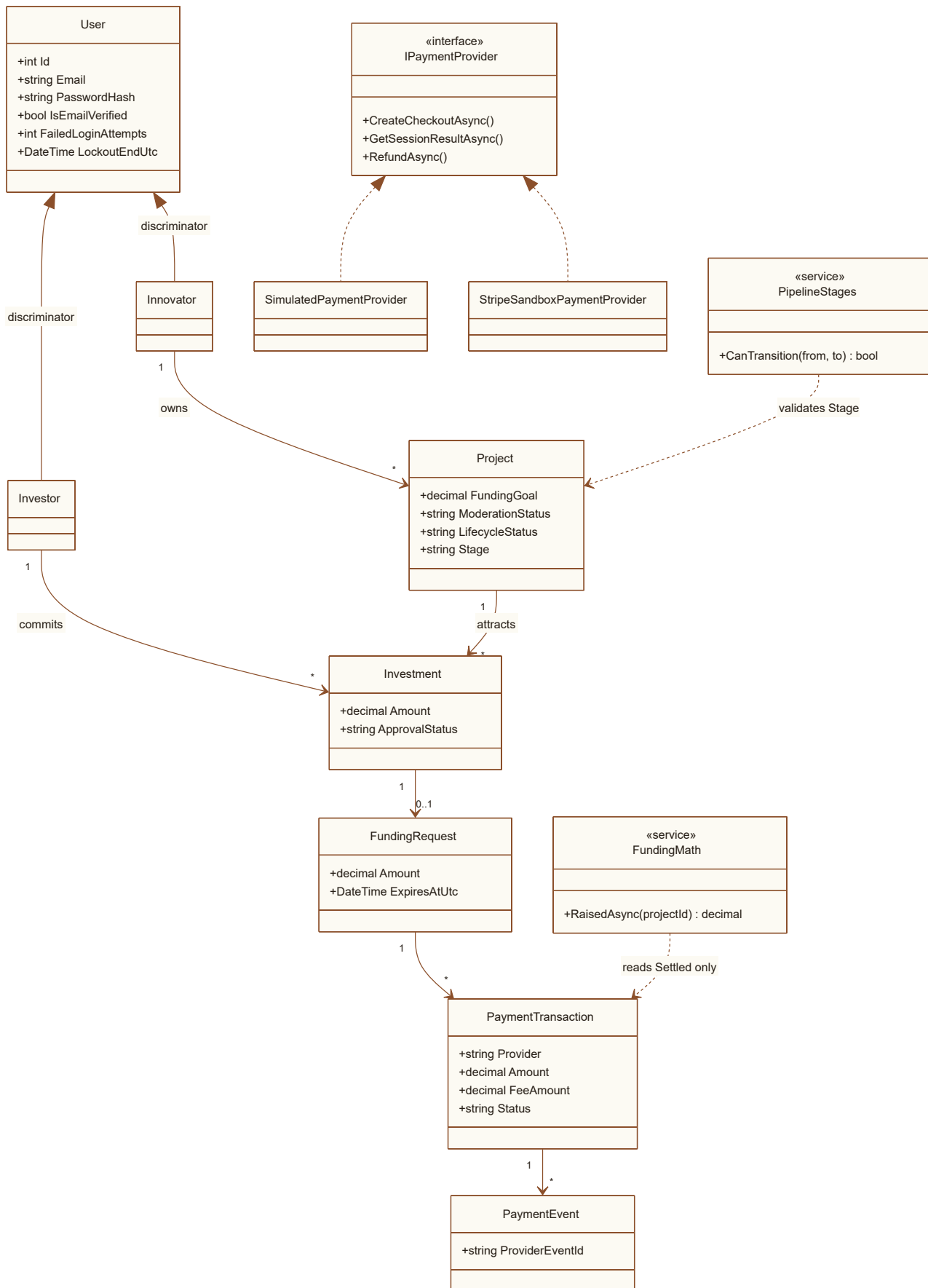


Figure 7 Domain class model. **Investor** and **Innovator** specialise **User** through a discriminator (§7.1); the services at the foot hold rules the entities themselves do not.

7.3 Logical Schema and Data Dictionary

The full dictionary is in Section 2. This section documents the four tables whose columns encode rules rather than facts.

7.3.1 Project

Column	Type	Meaning and rule
<code>Id</code>	<code>int</code> , PK	Surrogate key.
<code>OwnerId</code>	<code>int</code> , FK	The founder. Cascade delete: a venture cannot outlive its owner.
<code>Title</code>	<code>nvarchar</code>	Displayed name.
<code>FundingGoal</code>	<code>decimal</code>	Target amount. Stored with explicit precision; see the note on money types below.
<code>ModerationStatus</code>	<code>nvarchar</code>	Administrative state — pending, approved, rejected. Written only by an administrator.
<code>LifecycleStatus</code>	<code>nvarchar</code>	Operational state — active, paused, closed. Written by the owner or by the system.
<code>Stage</code>	<code>nvarchar</code>	Commercial state — where the venture stands in its funding progression. Written by the funding pipeline.
<code>CreatedDate</code>	<code>datetime2</code>	Stored in UTC. See §6.7.

Table 27 The `Project` table. Three independent state columns, three independent writers.

7.3.2 Investment and the funding chain

`Investment` records intent and its approval; `FundingRequest` records an attempt to settle that intent within a time box; `PaymentTransaction` records what a provider actually did; `PaymentEvent` records each provider callback individually so that duplicates can be recognised.

`PaymentTransaction.Provider` stores the string `"stripe"` or `"simulated"` on every row. This is not diagnostic decoration: it means a transaction always carries the identity of the system that produced it, so a data set can never become ambiguous about which rows represent real settlement.

NOTE

Money is `decimal`, never `float`. Binary floating point cannot represent most decimal fractions exactly, and an accumulated rounding error in a funding total is precisely the class of defect this chapter exists to prevent. Every monetary column is `decimal` with explicit precision and scale, configured in `AppDbContext` rather than left to convention.

7.4 Normalisation Analysis

The schema is in third normal form throughout, with two deliberate, locally-justified departures.

Third normal form. Every non-key column depends on the key, the whole key and nothing but the key. Repeating groups are separate tables — a venture’s images, documents, team members, milestones and updates are each their own table with a foreign key back to the venture, rather than serialised columns.

Departure 1: denormalised analytics counters. `ProjectView` records individual view events, and aggregate view counts are computed from it. Where a count is needed on a listing page for many ventures at once, computing it per row is a query cost paid on the hottest read path in the system. §17.4 documents the projection used and the measurement that justified it.

Departure 2: the discriminator column. Table-per-hierarchy is by definition not fully normalised — columns belonging to one specialisation are null for the other. The alternative was a join on every authentication call. The trade is documented in §7.1 and accepted.

What is not denormalised: funding totals. There is no `AmountRaised` column anywhere in the schema. This is the single most obvious denormalisation available, and it is deliberately not taken. §7.6 explains why.

7.5 Indexing Strategy

Indexes are added for measured access patterns, not speculatively. Each index below exists because a specific query needed it.

Table	Index	Purpose
User	Email, unique	Login by email; the uniqueness constraint is the account-identity rule, not an optimisation.
RefreshToken	TokenHash, unique	Refresh lookup on every token rotation. Unique because two live tokens may never share a hash.
SecurityLog	(UserId, CreatedAtUtc)	Per-user audit queries, which are always time-ordered.
Project	(ModerationStatus, LifecycleStatus, CreatedDate)	The public listing query filters on both state columns and orders by creation date. This composite covers it in one seek.
Project	Stage	Stage-filtered discovery.
Bookmark	(UserId, ProjectId), unique	Enforces one bookmark per user per venture and serves the “is this saved” check.
Follow	(FollowerId, FollowedId), unique	Enforces one follow edge per pair.
Follow	FollowedId	Follower-count and follower-list queries.
Review	(ProjectId, InvestorId), unique	One review per investor per venture — a business rule expressed as an index.
Report	(ProjectId, Status)	The moderation queue filters exactly on this pair.
ProjectView	(ProjectId, CreatedAt)	View aggregation over a time window.

Table 28 Indexes and the query that motivated each.

Four of these are **unique** indexes doing double duty. `Bookmark`, `Follow` and `Review` each express a business rule — save once, follow once, review once — as a constraint the database enforces. An application-level check would leave a race between two concurrent requests; the unique index does not.

7.6 Constraints and Integrity Rules

This is the section the rest of the chapter exists to support.

7.6.1 Why funding totals are derived

The obvious design stores a running total on the venture and increments it when a payment settles. It is faster to read and it is wrong, for two reasons.

First, it can drift. Any code path that writes a payment without updating the total, or updates the total twice for one payment, leaves a number that no longer corresponds to reality — and because it is stored, nothing detects this. Second, and more seriously, it invites the conflation of **approved** with **funded**. Once a single column holds “the amount”, the temptation to increment it at approval time is structural.

The chosen design has no such column. Totals are computed from settled payment transactions at read time, in one place, by `FundingMath`. There is exactly one definition of what a venture has raised, and it is arithmetic over rows that represent money that actually moved.

ADR-04

Funding totals are derived, never stored

CONTEXT A stored running total can drift silently and structurally invites incrementing on approval rather than on settlement — conflating an intention with a payment.

DECISION Store no aggregate funding column. Compute totals from settled payment transactions at read time through a single service, `FundingMath`.

CONSEQUENCES One source of truth, and no drift is possible because there is nothing to drift from. Approved and funded are structurally distinct. The cost is a read-time aggregation on venture pages, which §17.4 measures and §17.5 addresses.

7.6.2 Why status and stage are separate

A venture has three orthogonal kinds of state, written by three different actors:

Column	Answers	Written by
<code>ModerationStatus</code>	Has an administrator allowed this to be public?	Administrator
<code>LifecycleStatus</code>	Is the owner currently running this venture?	Owner or system
<code>Stage</code>	How far has the venture progressed commercially?	Funding pipeline

Table 29 Three state columns, three writers, three questions.

Collapsing these into one column produces defects that are hard to see and easy to ship. An administrator approving a venture would write the same field the funding pipeline writes, so an approval arriving after a funding event would reset commercial progress. A venture paused by its owner would be indistinguishable from one rejected by an administrator, and restoring it would require guessing which it had been.

Keeping them separate means each transition is validated against its own rules and the three writers never contend for the same column. The composite index in §7.5 exists because the public listing query then has to filter on two of them at once — which is the cost of the decision, and a cheap one.

ADR-05

Model moderation, lifecycle and commercial stage as independent columns

CONTEXT A single state column is written by the administrator, the owner and the funding pipeline, so any two of them can overwrite each other's decision.

DECISION Model `ModerationStatus`, `LifecycleStatus` and `Stage` as separate columns with independent transition rules and a single writer each.

CONSEQUENCES An approval can never reset funding progress and a funding event can never un-reject a venture. Listing queries must filter on two columns, which the composite index covers. The state space is larger and must be documented — which §14.2 does.

7.6.3 Referential rules

Delete behaviour is chosen per relationship rather than set globally, and the choice encodes whether the child is **owned** by the parent or merely **refers** to it.

Behaviour	Applied to	Reasoning
<code>Cascade</code>	Project images, documents, team members, milestones, updates, views, reviews, reports, bookmarks; refresh tokens	These have no meaning without their parent. A venture's images are part of the venture.
<code>Restrict</code>	Investments, messages, comments, user-project interactions, notification references	These are records of something that happened. Deleting a user must not silently erase the history of a commitment or a conversation.
<code>SetNull</code>	Security log user reference	The audit record must survive the account it describes.

Table 30 Delete behaviour by relationship class.

The `Restrict` rows are the interesting ones. They mean an account with investment history cannot simply be deleted — which is the correct outcome for a system that records financial commitments, and which is why account **suspension** (§10.10) exists as a distinct operation from deletion.

7.7 Migration Strategy

The schema was not designed once and implemented. It evolved across more than twenty Entity Framework migrations, each named for the change it carries: `Phase4ProductionAuthSecurity`, `AddInvestmentApprovalStatus`, `Phase2_InvestmentPipeline`, `Phase6_VentureLifecycle` and so on.

Three rules governed migrations.

1. **Every schema change is a migration.** No schema was ever altered directly against a database. The migration history is therefore a complete and replayable record.
2. **Migrations are additive where possible.** Adding a nullable column and backfilling is preferred over altering a column in place, because the former is reversible and the latter frequently is not.
3. **A migration is applied to the shared database only after it applies cleanly locally.** The project runs against a hosted database (§16.3), which means a bad migration affects the whole team rather than one machine.

The names above are worth noting for what they reveal: `AddInvestmentApprovalStatus` and `Phase2_InvestmentPipeline` are the migrations in which the separation between an approved and a funded investment was introduced. That separation was not in the first design. It was added when its absence caused a defect, and Section 18 describes what that defect looked like.

7.8 Backup and Recovery

The database is hosted rather than self-administered (§16.3), so backup is a provider responsibility, and the project's obligation is to know what that covers and what it does not.

Covered by the provider Scheduled full backups with point-in-time restore within the provider's retention window.

Covered by the project The schema itself, which is fully reproducible from the migration history in version control. A database can be recreated from an empty instance by applying migrations in order.

Not covered Uploaded files — venture images, documents and message attachments — which live outside the database. This is a real gap and it is recorded as a limitation in §19.4 rather than left implicit.

UI/UX Design System

8.1 Design Principles

A platform asking strangers to exchange money must look like it can be trusted before anyone reads a word of it. Interface quality here is not decoration; it is the first evidence a visitor has, and it is doing work that no feature can do.

Four principles govern every decision in this chapter.

Restraint over decoration Financial interfaces lose credibility through excess, not through plainness. The system uses one display face, one accent, and a great deal of space.

One motif, used sparingly A single structural idea — the chamfered plate — carries the identity. It appears where structure genuinely changes, not on every surface. A motif applied everywhere stops being a motif.

Evidence over assertion The interface shows numbers, dates and documents rather than adjectives. Where two facts differ — committed and settled (§14.1) — it shows both rather than reconciling them into one comfortable figure.

Accessible by construction Behaviour comes from primitives that are accessible before they are styled (§5.10), so compliance is the starting state rather than a retrofit.

8.2 Design Tokens

The system is defined as tokens, not as decisions made per component. Every value below has exactly one definition.

Token	Value	Role
--color-ink	#241C14	Body text. A warm near-black; pure black reads as harsh against a warm ground.
--color-text-secondary	#71614C	Supporting text, labels.
--color-bronze	#8B4F2A	Primary accent: structure, rules, emphasis.
--color-gold	#B08A3F	Secondary accent, used rarely. Gold at volume looks cheap; at low volume it looks expensive.
--color-bg	#F6F2E7	Page ground. Warm cream rather than white.
--color-surface	#FCFAF3	Raised surfaces — cards, plates.
--color-border	#E3D9C4	Hairlines and dividers.

Table 31 Colour tokens. A dark-mode set exists in the product with the same role assignments.

Token	Family	Role
<code>--font-heading-en</code>	Cinzel	Display only: page titles and the wordmark. Never in running text — it is an inscriptional capital face and unreadable at paragraph size.
<code>--font-body-en</code>	Karla	Interface text, labels, controls.
<code>--font-numerals</code>	Spectral	Figures and long-form reading.
<code>--font-heading-ar</code>	Aref Ruqaa	Arabic display.
<code>--font-body-ar</code>	Cairo	Arabic interface text.

Table 32 Typographic tokens.

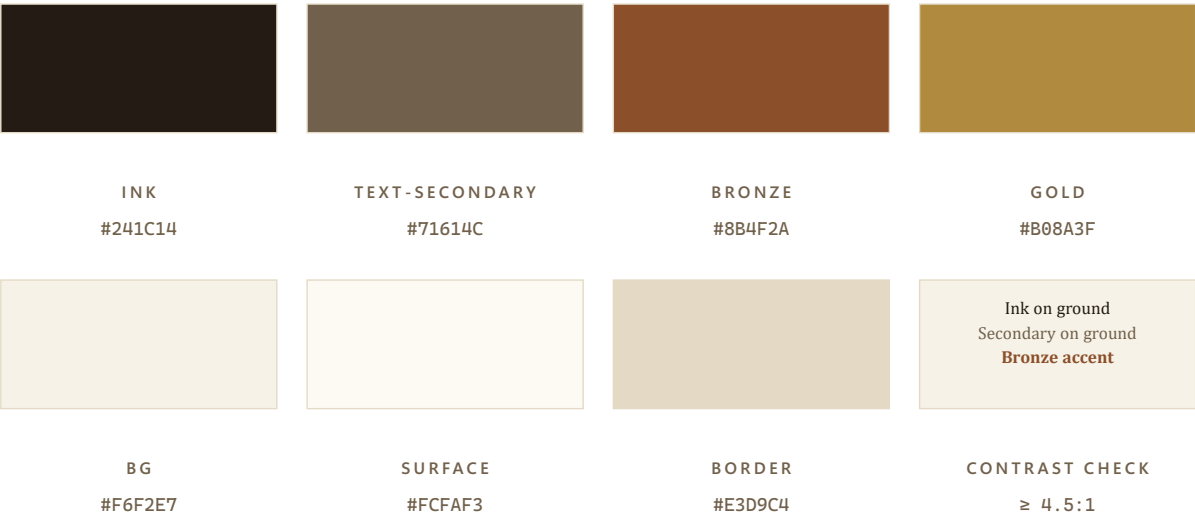


Figure 8 The palette, printed from the token definitions themselves. The final cell is the contrast pairing that §8.7 depends on.

DISPLAY / CINZEL

Ventures seeking capital

HEADING / KARLA

Funding totals are derived

BODY / SPECTRAL

An approved commitment is not a funded one — the platform shows both figures.

LABEL / KARLA

MODERATION STATUS

MONO / CODE

FundingMath.RaisedAsync()

Figure 9 The type scale, set in the faces the product loads.

NOTE

Both figures above are **rendered from the tokens**, not pictures of them. The swatches are filled with the same colour values the product uses and the specimen is set in the same Cinzel, Karla and Spectral the product loads. A design system that cannot be applied outside its original medium has not been abstracted properly — so this book is the test of its own claim.

Spacing and radii. Spacing follows a 4-pixel base scale. Radii are small and uniform; the chamfer, not rounding, is what carries character.

8.3 Component Library

Components come from shadcn/ui: source files copied into the repository rather than a dependency (§5.10). Each is composed of Radix primitives that supply behaviour and are styled entirely with the tokens above.

The consequence worth noting is ownership. A component that misbehaves is modified directly rather than overridden from outside, which means the code that governs the interface is code the project can read. The cost is maintenance: an upstream fix does not arrive automatically.

Composition over configuration. Components are built to be composed rather than to accept a large option surface. A card that takes fifteen props to cover every use eventually covers none of them well; a card that composes from a header, a body and a footer covers all of them.

8.4 The System in Use

A design system is only a system if the same tokens hold across every axis the product varies on. This one varies on two: **theme** and **language**. Both are shown below on the same surface, so the comparison is of the system rather than of two screens.

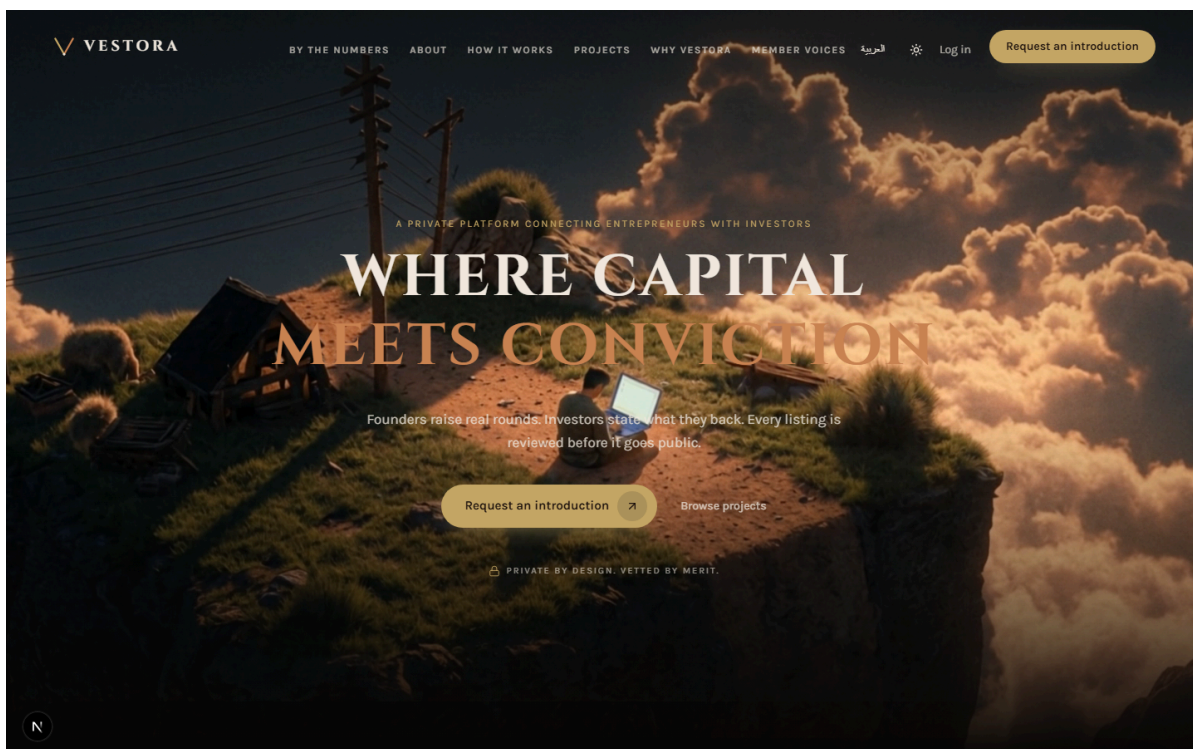


Figure 10 The landing page. Cinzel at display size, the gold accent used once, and an image-led composition — the four principles of §8.1 in one frame.

8.4.1 The theme axis

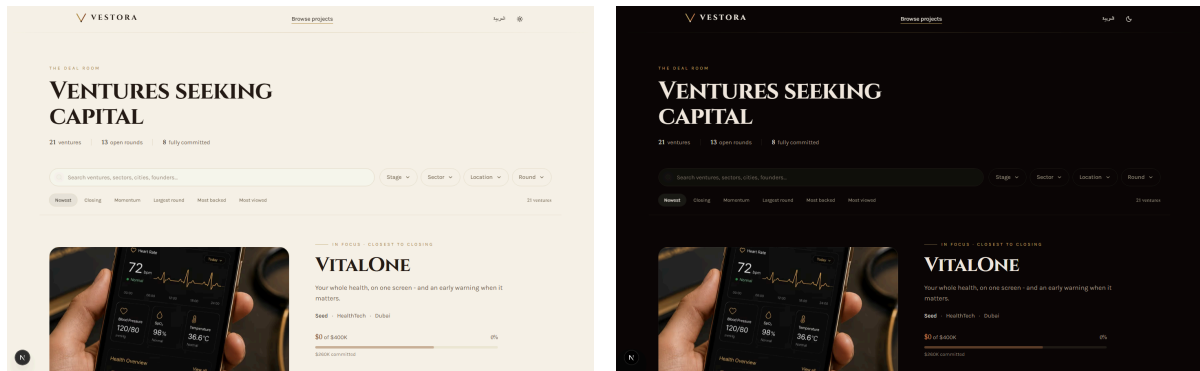


Figure 11 The venture listing under the light and dark token sets. Every colour is a token with the same **role** in both; nothing is re-chosen per theme (§8.2).

The two are not separately designed screens. Each colour resolves through the same semantic token — ground, surface, ink, accent — against a different value set. This is what §18.6 records going wrong once: a token resolved against the root rather than its surface inverted, and text disappeared into its own background.

8.4.2 The language axis

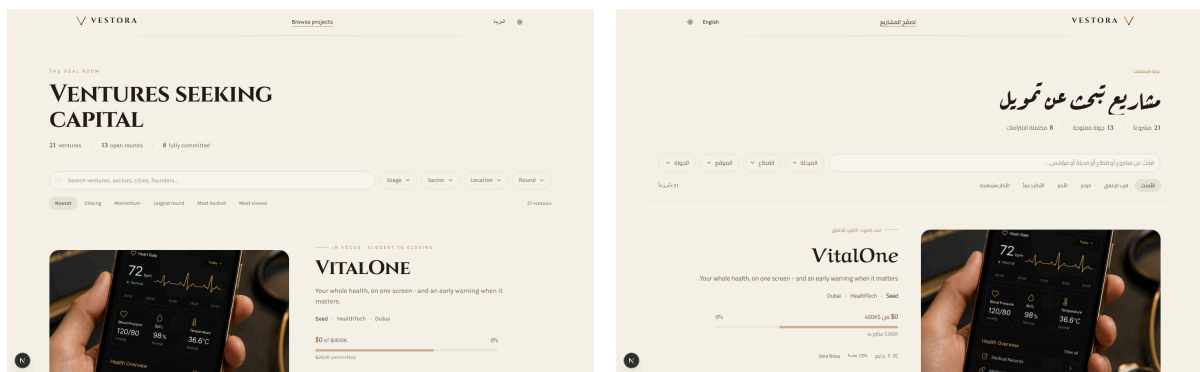


Figure 12 The same surface in English and Arabic. The layout mirrors, the display face changes to Aref Ruqaa, and the information is identical.

Three things change and one does not. The writing direction flips, so the layout mirrors — navigation, filters, progress bars and the whole grid. The display face changes to the Arabic pair in §8.2, because Cinzel has no Arabic. Numerals and Latin proper nouns stay left-to-right inside right-to-left text, which the layout has to tolerate rather than fight.

What does not change is the information. Both captures show the same twenty-one ventures, the same filters and the same funding figures.

8.4.3 Mobile



Figure 13 The landing page at 390 px, in English light and Arabic dark — both varying axes at once. The composition rearranges; nothing is removed (§8.6).

8.4.4 A designed failure state

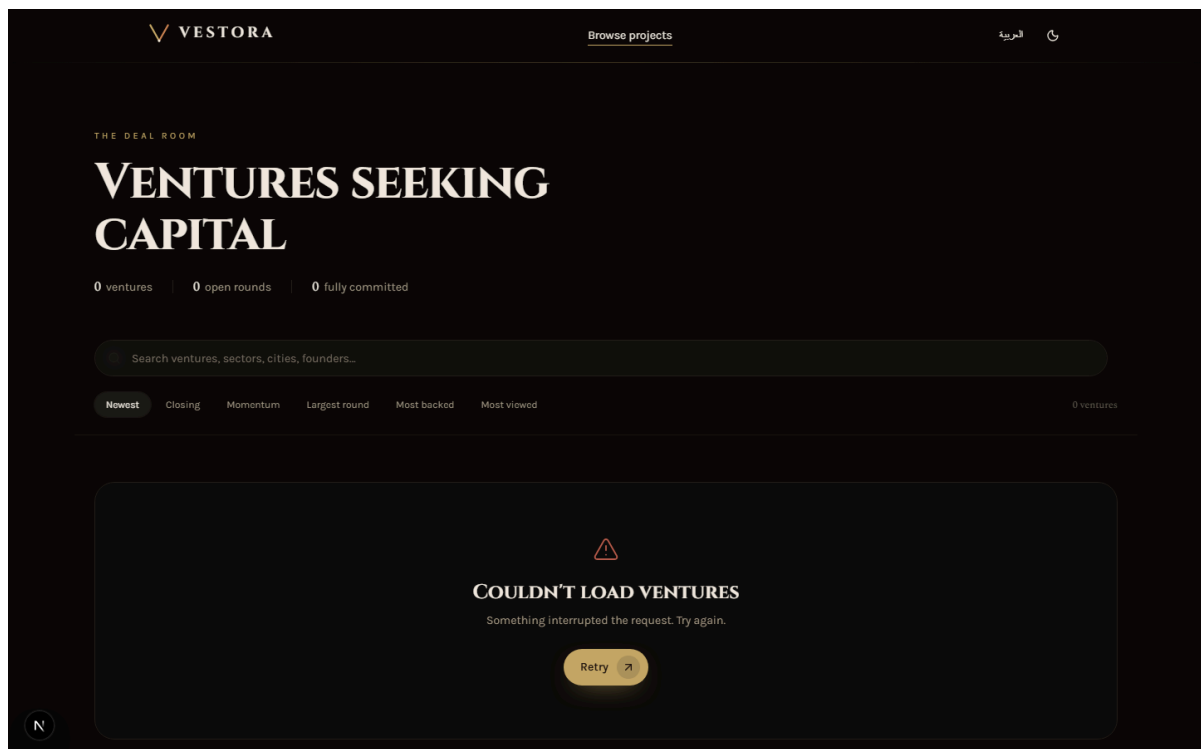


Figure 14 The same listing captured with the API stopped. What is visible is the **error state**: a designed surface with a stated cause and a single recovery action, not a blank page or a raw message (\$8.8).

This capture is deliberate. It is the surface a user meets when something has gone wrong — which is exactly when a platform handling money is being judged — and it is the state most interface galleries omit, because producing it is inconvenient.

8.5 From Wireframe to Interface

Design proceeded in three passes, and the middle one is the one usually skipped.

1. **Structure.** Low-fidelity layouts establishing what is on each screen and in what order of importance, with no colour and no type decisions. Most hierarchy problems are visible and cheap to fix at this stage.
2. **System.** The tokens above, decided once against representative screens rather than negotiated per screen. This is the pass that prevents a system from becoming a collection.
3. **Interface.** High-fidelity screens composed from the system.

Working in this order means visual revision is revision of **tokens**, which propagates, rather than revision of screens, which does not.

8.6 User Flows

Three flows received explicit design attention because each is a point where a user can be lost.

Registration to first meaningful action Registration, verification and onboarding are separate steps, and onboarding asks only for what makes the platform useful immediately. Every additional field before first value is a point of abandonment.

Discovery to commitment Listing, detail, commitment, approval, checkout. The state of a commitment is legible at every step, because a user who cannot tell whether their money has moved will assume the worst.

Submission to publication Draft, submit, review, published — with the venture’s moderation state visible to its founder throughout. A submission that disappears into a queue with no feedback produces support requests.

8.7 Responsive Strategy

The interface is designed from 360 px to 1920 px (NFR-14), mobile-first: the narrow layout is the base and wider layouts add, rather than the reverse.

The distinction that matters is between **responsive** and **adaptive**. The layout responds continuously; the information does not change by breakpoint. A narrow-screen user sees the same facts as a wide-screen one, differently arranged. Hiding information on small screens is a decision that a mobile user needs less — which for a user checking whether their investment settled is plainly false.

Data-dense surfaces — dashboards, administrative tables — are where this is hardest. Wide tables scroll within their own container rather than forcing the page to scroll horizontally, which keeps navigation and page structure in place.

8.7.1 Bidirectional layout

Language is a second responsive axis, and it is the one more often forgotten. Selecting Arabic sets the document direction, and the entire layout mirrors: navigation, filter rows, progress fills, icon placement and the reading order of every grid.

Three rules make this work rather than merely function:

1. **Layout is expressed in logical properties, not physical ones.** Spacing is declared as start and end rather than left and right, so mirroring is a property of the direction rather than a second set of rules.
2. **The type stack changes with the language.** Cinzel has no Arabic coverage, so the display role resolves to Aref Ruqaa and body text to Cairo (§8.2). The **role** is constant; the face is not.
3. **Mixed direction is tolerated, not fought.** Numerals, currency and Latin proper nouns remain left-to-right inside right-to-left text. A funding figure reading \$0 of \$400K inside an Arabic sentence is correct, and the layout must not try to reverse it.

The figures in §8.3 show the same surface on both sides of this axis.

8.8 Accessibility Compliance

The target is WCAG 2.1 AA (NFR-13). Compliance is claimed here only where it was checked.

Criterion	Position	State
Colour contrast	Ink on ground and secondary text on ground both exceed 4.5:1. The gold accent is not used for text at small sizes, where it would not pass.	Verified
Keyboard operation	All interactive elements are reachable and operable by keyboard; focus order follows document order.	Verified
Focus visibility	A visible focus ring is never removed. It is restyled to match the system, not suppressed.	Verified
Semantic structure	Headings are hierarchical; landmarks are present; controls are real controls rather than styled containers.	Verified
Names, roles and values	Supplied by the Radix primitives rather than added by hand.	Inherited
Motion sensitivity	Animation respects the reduced-motion preference.	Verified
Screen reader testing	Spot-checked on primary flows; not exhaustive across every surface.	Partial

Table 33 Accessibility position by criterion, including one that is only partial.

The final row is the honest one. A blanket claim of AA conformance across sixty-five routes would not be supportable, and §19.4 records this as a limitation rather than absorbing it into a claim.

8.9 Motion and Interaction

Motion has one job: to explain what changed. Every transition in the system answers “where did this come from” or “what just happened”, and anything that does not is removed.

Three rules apply. Durations are short — long enough to be perceived, short enough never to be waited on. Easing decelerates, because motion that stops abruptly reads as mechanical. And motion respects the reduced-motion preference, which is a correctness requirement rather than a nicety: for some users, animation is not decoration but a symptom trigger.

States are designed, not defaulted. Loading, empty and error states are part of the design system rather than afterthoughts. An empty watchlist explains what a watchlist is for; a failed payment says what happened and what to do. These are the surfaces a user meets when something has gone wrong, which is exactly when a platform handling money is being judged.

PART 3

Implementation

The API surface, the security model, the payment and real-time subsystems, and a walkthrough of the platform's features as they were built.

API Design and Implementation

9.1 REST Principles Applied

The API is REST in the sense that matters practically: resources have stable addresses, HTTP verbs carry meaning, and status codes are used for their defined purpose rather than as decoration on a body that always says `success: true`.

It is not REST in the hypermedia sense. Responses do not carry links describing available transitions, because the only client is one the team also writes and the indirection would buy nothing.

Four rules are applied consistently:

1. **Verbs mean what they mean.** `GET` never changes state. `POST` creates or performs an action. `PUT` and `PATCH` update. `DELETE` removes.
2. **Status codes are load-bearing.** `200` succeeded, `201` created, `204` succeeded with nothing to say, `400` the request was malformed, `401` no valid identity, `403` valid identity without permission, `404` no such resource, `409` a state conflict, `422` semantically invalid.
3. **The distinction between `401` and `403` is preserved.** Conflating them destroys the client's ability to decide between re-authenticating and showing a refusal.
4. **Resources are nouns, actions are sub-resources.** An action that is not a create or an update on the resource itself becomes a named sub-path rather than a verb in a query string.

9.2 Resource Modelling and URI Design

Resources follow the domain, not the tables. `FundingRequest` and `PaymentTransaction` are separate entities (§11.1) but are not separately addressable: a client opens a checkout on an **investment**, and the chain behind it is an implementation detail.

Pattern	Use
<code>/projects</code>	Collection: list, filter, search, create.
<code>/projects/{id}</code>	A single venture.
<code>/projects/{id}/updates</code>	A sub-collection owned by the venture.
<code>/projects/{id}/support</code>	An action on the venture that is not a create on it.
<code>/payments/webhook/stripe</code>	A provider callback. Deliberately outside the resource tree — it is not a client-facing resource.

Table 34 URI patterns and what each is for.

Twenty-two controllers group these by area — authentication, projects, investors, capital, payments, messages, notifications, follows, bookmarks, engagement, deal room, reports, feed, signals, insights, founder and investor dashboards, and three administrative areas covering moderation, revenue and general administration. The full endpoint listing is in Section 1.

9.3 Request and Response Contracts

No entity is ever serialised directly. Every boundary uses a data transfer object, and this is a rule with no exceptions in the codebase.

Two reasons. First, a schema change would otherwise silently alter the public contract — adding a column would add a field to every response that touches that table. Second, entities carry navigation properties, and serialising one either drags an object graph into the response or throws on a cycle.

DTOs are also where over-fetching is prevented. A venture card needs eight fields; the entity has forty. The listing query projects into the card DTO in the database rather than materialising entities and mapping them in memory (§6.6).

9.4 Validation Strategy

Validation has two layers with a clear division:

Structural, at the boundary Is a required field present, is an amount positive, is an email shaped like an email. Declarative, runs before the controller body, and produces a `400` with a field-level error map. A service never sees a structurally invalid request.

Domain, in the service Is this venture open to investment, is this transition permitted from the current stage, is this investor allowed to commit here. These depend on state a validator cannot see, and produce a `409` or `422` rather than a `400` — the request was well-formed, the world disagreed.

Collapsing these two would push database reads into the validation layer and make error semantics ambiguous.

9.5 Error Handling and Result Shaping

Every failure returns the same shape, so a client has one parser rather than one per endpoint. Expected failures are carried by the service result object (§6.5) and translated by a single shared extension; unexpected ones are caught centrally, logged with detail, and returned without it.

```
1 // Outcome-to-HTTP mapping exists once for the whole API rather than
2 // being re-decided in each of the twenty-two controllers.
3 var result = await _projects.PublishAsync(id, userId, ct);
4 return result.ToActionResult(this);
```

Listing 3 Uniform result translation at the controller seam.

The internal detail of an exception never reaches the client. An error response is an information disclosure surface (§10.11), and a stack trace in a response body tells an attacker the framework, the version and the call path.

9.6 Pagination, Filtering and Sorting

Every collection endpoint is paged. There is no endpoint that returns an unbounded list, because such an endpoint is a denial-of-service surface that grows with the platform's success.

Paged responses carry the items, the total count, the page and the page size, so a client can render a pager without a second request. The page size is capped server-side — a client asking for ten thousand rows receives the maximum, not ten thousand.

Filtering and sorting are allow-listed. A sort field arriving from a client is matched against a permitted set rather than interpolated into a query, which closes the injection surface that dynamic ordering otherwise opens.

9.7 API Documentation

The API is described by an OpenAPI document generated from the code and served through an interactive browser in development. Generation from source rather than hand-maintenance is the point: a hand-written specification is wrong the moment a signature changes, and a specification nobody trusts is worse than none.

The document is disabled outside development. An interactive schema browser is a complete map of the attack surface, and there is no reason to publish one.

The full endpoint listing in Section 1 is derived from the same source, so the appendix and the running system cannot disagree.

Security, Authentication and Authorisation

A platform that records financial commitments is a target. This chapter states what was defended against, how, and — where a defence is partial — what remains exposed.

10.1 Threat Model

Threats are enumerated using STRIDE against the system's four trust boundaries: browser to API, API to database, API to payment provider, and API to SMTP relay.

Threat	Concrete scenario	Control
Spoofing	An attacker signs in as another user through credential stuffing or a stolen token.	BCrypt password hashing, logout after five failures, short access-token lifetime, refresh-token rotation (§10.5–§10.7).
Tampering	A client submits an investment amount or a venture status it is not entitled to set.	Server-side validation of every mutation; state transitions validated against <code>PipelineStages</code> ; no client-supplied identity is trusted.
Repudiation	An administrator denies having rejected a venture; a user denies a sign-in.	<code>AdminAuditLog</code> and <code>SecurityLog</code> as durable tables, not log lines (§10.13).
Information disclosure	Error responses leak internal structure; one user reads another's messages or documents.	Generic error responses (§6.7); ownership checked on every read of user-scoped data.
Denial of service	Repeated sign-in or password-reset requests exhaust resources or spam a mailbox.	Logout thresholds and per-action cooldowns on verification and reset (§10.3, §10.4).
Elevation of privilege	A standard account reaches an administrative endpoint.	Role-based authorisation on every administrative route; the administrative role cannot be self-assigned (§10.8).

Table 35 STRIDE threat enumeration and the control applied to each.

10.2 OWASP Top Ten Mitigation Mapping

Risk	Position in this system
Broken access control	Authorisation is checked server-side on every state-changing endpoint and on every read of user-scoped data. The matrix is in §10.9.
Cryptographic failures	Passwords are BCrypt-hashed with a per-password salt. Refresh tokens are stored as hashes, never in plaintext. Transport is TLS.
Injection	All data access goes through parameterised EF Core queries. There is no string-concatenated SQL in the codebase.
Insecure design	The funding invariant is enforced by database constraints rather than by application discipline (§7.6).
Security misconfiguration	The application refuses to start when a required secret is missing rather than falling back to a default (§16.6).
Vulnerable components	Dependencies are pinned and updated deliberately. This is the weakest area: there is no automated dependency scanning. Recorded in §19.4.
Authentication failures	Lockout, verified email required for sign-in, time-boxed single-use tokens, rotation on refresh.
Data integrity failures	Webhook payloads are signature-verified before any state change (§11.4).
Logging and monitoring failures	Security events are written to durable, queryable tables. Alerting is limited; see §16.9.
Server-side request forgery	The API makes outbound requests only to configured provider endpoints; no user-supplied URL is fetched.

Table 36 OWASP Top Ten mapping. Two rows record gaps rather than controls.

NOTE

Two rows above admit a weakness rather than claiming a control. A mapping table in which every row is satisfied is a table that has not been checked.

10.3 Registration and Email Verification

Registration creates an unverified account and sends a single-use token valid for sixty minutes. Sign-in is refused until the address is confirmed, so an account cannot be created against an address the registrant does not control.

Two details matter. First, resend is rate-limited by a cooldown, so the endpoint cannot be used to flood a third party's mailbox. Second, registration responds identically whether or not the address is already registered — a differing response would turn the endpoint into an account-existence oracle.

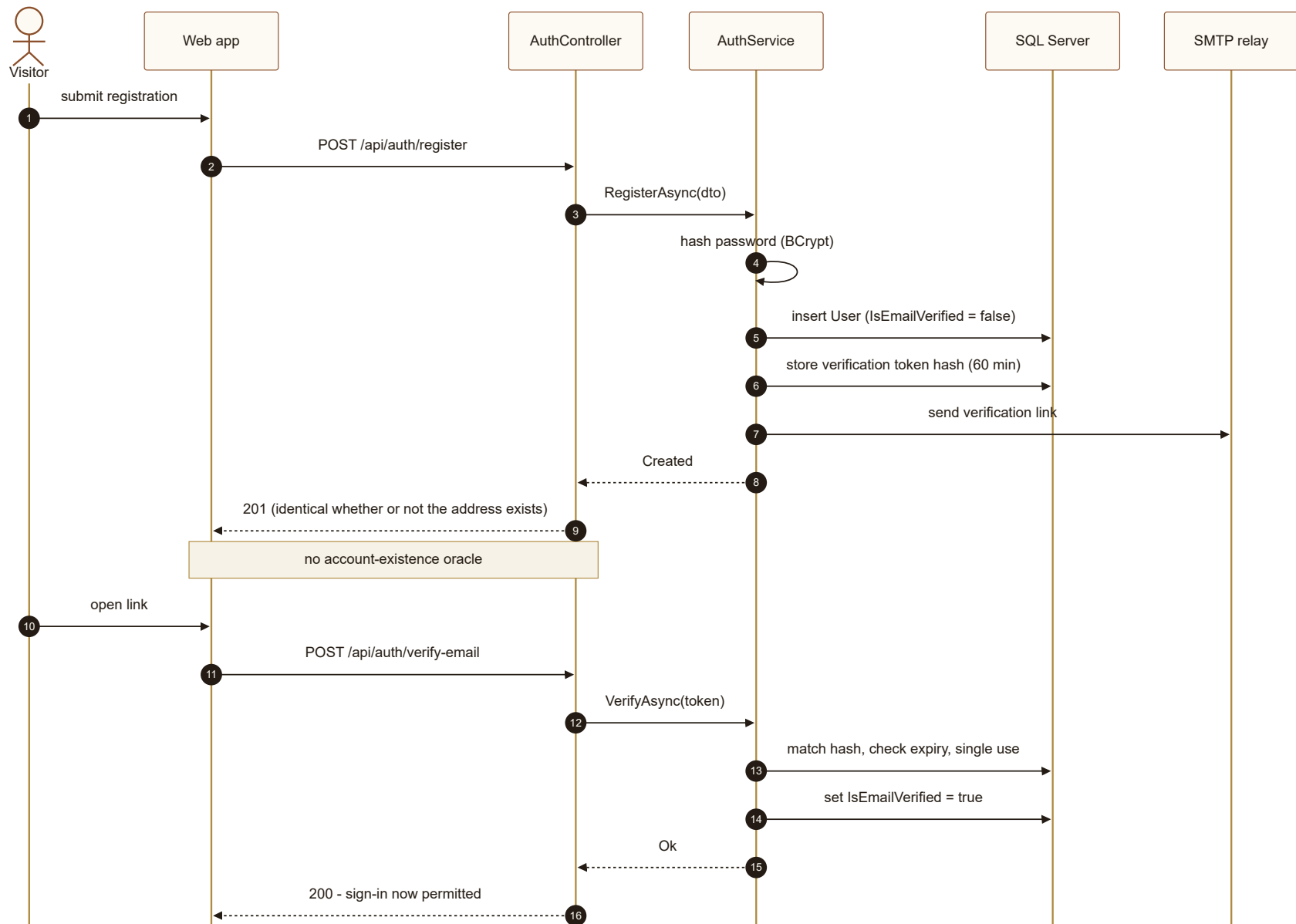


Figure 15 Registration and email verification. The response is identical whether or not the address is already registered, so the endpoint is not an account-existence oracle.

10.4 One-Time Password Flow

Where a confirmation code is used in place of a link, the code is short-lived, single-use, and bound to the account that requested it. Verification failures are counted, and the attempt counter is what limits guessing — a six-digit code is trivially brute-forced if attempts are unbounded, and the code length is not what provides the security.

10.5 JWT Issuance and Claims

On successful sign-in the API issues a signed access token carrying the user identifier, the account type and the role. The token is a bearer credential: possession is sufficient, so its lifetime is the primary control on the damage a leaked token can do.

Property	Value and reasoning
Algorithm	HMAC with a symmetric key supplied by configuration.
Lifetime	60 minutes. Long enough to avoid refresh churn, short enough to bound exposure.
Claims	Identifier, account type, role. No personal data — a token travels in headers and is logged by intermediaries.
Key source	Configuration only. The application refuses to start without it (§16.6).

Table 37 Access token properties.

10.6 Session Lifecycle: Refresh, Logout, Revocation

Refresh tokens are the part of the design that carries the real security weight, because they are long-lived.

- They are stored **hashed**. A database disclosure does not yield usable tokens.
- The hash column carries a unique index, so two live tokens cannot collide.
- They **rotate**: every refresh issues a new token and invalidates the old one.
- Rotation makes replay detectable. Presentation of an already-rotated token indicates the token was captured, and the response is to invalidate the chain rather than to issue a new pair.

Logout invalidates the refresh token. It cannot invalidate an already-issued access token — see the limitation in §10.10.

10.7 Brute-Force Protection and Lockout

Failed sign-in attempts are counted per account. Five failures within the window lock the account for fifteen minutes. The response to a locked account does not differ from the response to a wrong password, so the mechanism does not itself confirm that an account exists.

The threshold is a trade. Lower values frustrate legitimate users who mistype; higher values widen the guessing window. Five and fifteen were chosen as configuration values precisely so they can be tuned against observed behaviour rather than argued about in code review.

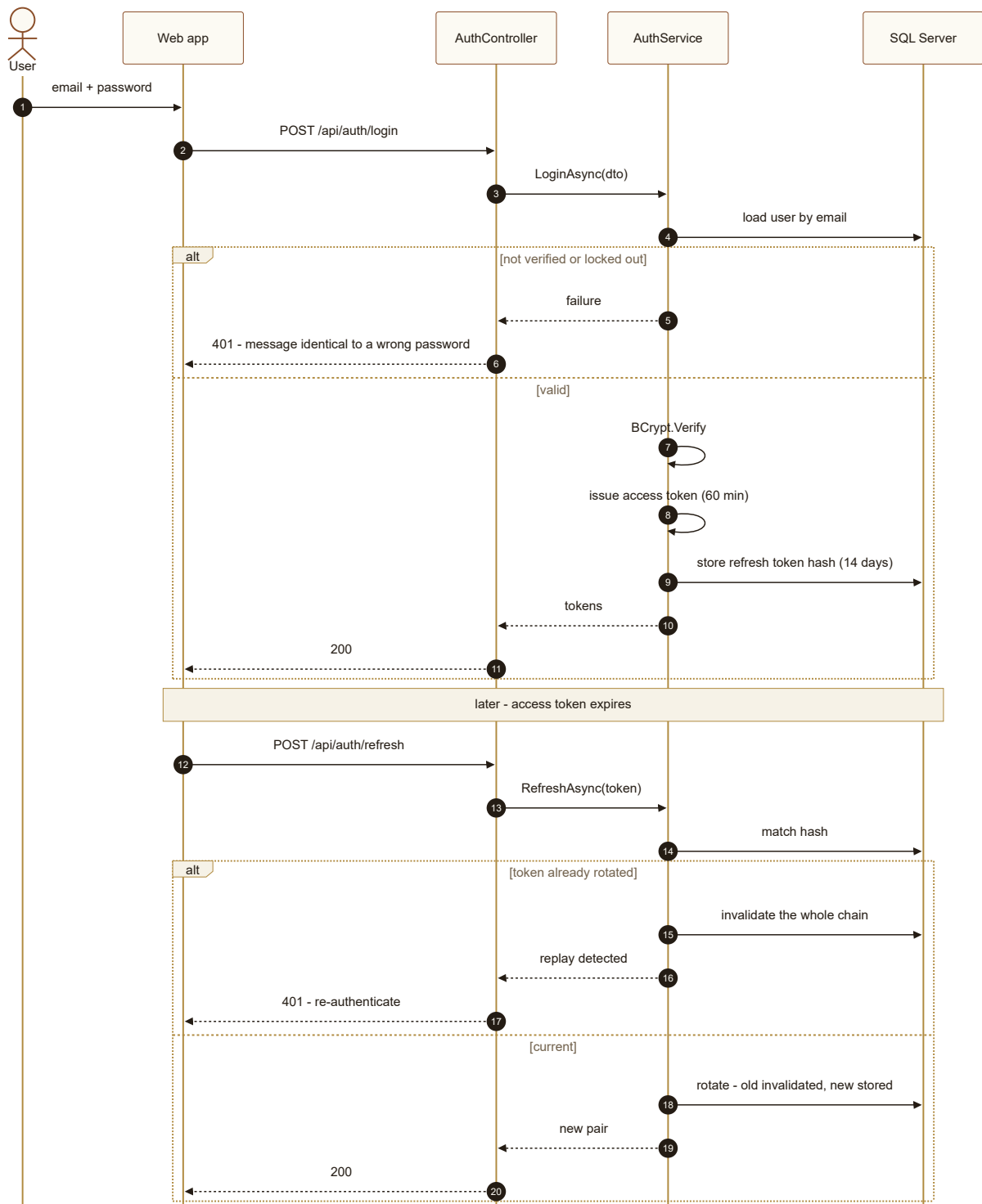


Figure 16 Sign-in and refresh-token rotation. Presenting an already-rotated token invalidates the whole chain — the replay detection that was observed firing during screenshot capture (Section 5).

10.8 Role-Based Access Control

Three roles exist: standard user, administrator, and the system itself for background work. The distinction that matters is that **account type** — whether a user acts as founder or investor — is not a security role. It determines which surfaces are shown; it does not determine what the API permits. Authorisation is enforced at the endpoint, not in the client. A hidden button is a usability affordance, not a control.

10.9 Authorisation Matrix

Action	Owner	Other user	Admin
View a published venture	Yes	Yes	Yes
View an unpublished venture	Yes	No	Yes
Create or edit a venture	Yes	No	No
Approve or reject a venture	No	No	Yes
Commit an investment	No ¹	Yes	No
Approve an investment	Yes	No	Yes
Read a conversation	Participants only	No	No ²
Download a requested document	Yes	If granted	Yes
Suspend an account	No	No	Yes
Read the audit log	No	No	Yes

Table 38 Authorisation matrix. ¹ A founder may not invest in their own venture. ² Administrators moderate reported content, not private conversations.

The two footnotes are the interesting rows. A founder investing in their own venture would let a venture inflate its own funding progress, so the rule is enforced server-side. And administrators deliberately cannot read private conversations: moderation operates on reported content, which is a narrower power than reading everything.

10.10 Account Lifecycle and Suspension

An account moves through registered, verified, active, suspended and closed. Suspension exists as a first-class state because deletion is not available for accounts with financial history (§7.6) — the referential rules prevent it, and correctly so.

NOTE

Stated limitation. A stateless access token cannot be revoked before it expires. An account suspended at minute one of a token's sixty-minute life retains API access until it expires, unless the endpoint checks account state on each call. State-changing and money-adjacent endpoints perform that check; read-only endpoints largely do not, because the per-request cost is paid on every read. The residual exposure is up to sixty minutes of read access for a suspended account, and it is accepted rather than hidden.

10.11 Input Validation and Sanitisation

Validation is server-side and declarative (§6.7). Client-side validation exists for feedback only and is never trusted.

Injection Parameterised queries throughout; no dynamic SQL.

Cross-site scripting The client escapes by default. Rendering raw HTML from user input is not done anywhere in the codebase.

Cross-origin requests The allowed origin list is configuration, not a wildcard.

Mass assignment Requests bind to DTOs that expose only the fields a caller may set. An entity is never bound directly from a request body, so a field such as a venture's moderation status cannot be set by including it in the payload.

The last of these is worth emphasising because it is the most commonly missed. Binding directly to an entity means every column is writable by anyone who knows its name.

10.12 File Upload Security

Uploads are the highest-risk input the platform accepts. Four controls apply, and all four are configuration rather than constants:

- A maximum size of two megabytes per image.
- An allow-list of content types — JPEG, PNG, GIF and BMP. An allow-list, not a deny-list: anything unrecognised is refused.
- Content-type verification against the file's actual bytes rather than its declared header or its extension, both of which are attacker-controlled.
- Stored filenames are generated, never taken from the upload, so a crafted filename cannot traverse a path or collide with an existing file.

10.13 Audit Logging and Data Privacy

Two durable tables carry evidence. `SecurityLog` records authentication events against a user and a timestamp, indexed on that pair because audit queries are always per-user and time-ordered. `AdminAuditLog` records administrative actions — approvals, rejections, suspensions — with the acting administrator.

Both are tables rather than log lines for one reason: a log stream is not evidence you can query, retain reliably, or present.

On privacy: the platform stores the minimum it needs. It holds no payment instrument data at any point — card details are entered on the provider's surface and never traverse the API, which keeps the platform outside the scope that would otherwise apply. Personal data is limited to what a profile displays and what authentication requires.

Payment Architecture

NOTE
Scope of this chapter, stated first. The platform runs against a **simulated** payment provider by default. A Stripe-compatible provider — including signature-verified, idempotent webhook handling — is implemented and selected by configuration, but the system has not been operated against a live merchant account. §11.9 states precisely which paths have been exercised and which have not. Nothing in this chapter claims a live integration.

11.1 Money Flow and Domain Model

A commitment passes through four objects, and the reason there are four rather than one is that each records something the previous cannot.

Object	What it records, and why it is separate
Investment	An intention, and whether it has been approved. It exists before any money is involved and may never be settled.
FundingRequest	An attempt to settle a specific investment, within a time box. It has its own expiry because a checkout that is never completed must not hold a claim indefinitely.
PaymentTransaction	What a provider actually did: amount, fee, provider name, provider reference, status. This is the only object that constitutes evidence of settlement.
PaymentEvent	One row per provider callback, carrying the provider’s event identifier. Its existence is what makes duplicate delivery detectable.

Table 39 The four objects in the funding chain and the distinct fact each holds.

The chain collapses easily and wrongly. A single `paid` flag on the investment would remove three objects and, with them, the ability to answer: when did this expire, which provider handled it, what fee was taken, and have we already processed this callback.

11.2 Provider Abstraction

The interface was given in §5.7. Its defining property is negative: it mentions no domain concept. It knows nothing of ventures, investors or funding targets. It performs three operations — create a checkout, read a session result, refund — and returns provider-neutral types.

Everything that constitutes a rule lives above it:

Component	Owns
<code>PaymentService</code>	The lifecycle: what may be checked out, what a callback is permitted to change, when a transaction becomes settled.
<code>FundingMath</code>	The definition of what a venture has raised. One implementation, one answer.
<code>IPaymentProvider</code>	Transport only. Two implementations, neither of which contains a rule.
<code>PaymentExpirySweeper</code>	Expiry of abandoned checkouts, running outside the request path.

Table 40 Where payment responsibility lives.

The consequence is the point of the design: because the rules sit above the interface, the simulated provider exercises **the same rules** as the Stripe provider. It is a test surface, not a stand-in — and that distinction is what makes the verification in Section 15 meaningful without a merchant account.

11.3 Checkout Sequence

1. An investor commits to a venture. An `Investment` is created in a pending state. No money is involved yet.
2. The founder or an administrator approves the investment. It is now approved — and still not funded.
3. The investor opens checkout. `PaymentService` validates that the investment is approved, unexpired and not already settled, then creates a `FundingRequest` with an expiry and asks the configured provider for a checkout session.
4. The provider returns a session identifier and a URL. A `PaymentTransaction` is written in a pending state, stamped with the provider's name.
5. The investor completes payment on the provider's surface. No card data touches the platform.
6. The provider calls back. The callback is verified (§11.4), deduplicated (§11.5), and — only then — the transaction is marked settled.
7. Funding totals change. Not because a counter was incremented, but because the set of settled transactions that `FundingMath` reads over is now different.

Step seven is the whole design in one sentence. There is no moment at which something is “added to the total”, because there is no total to add to.

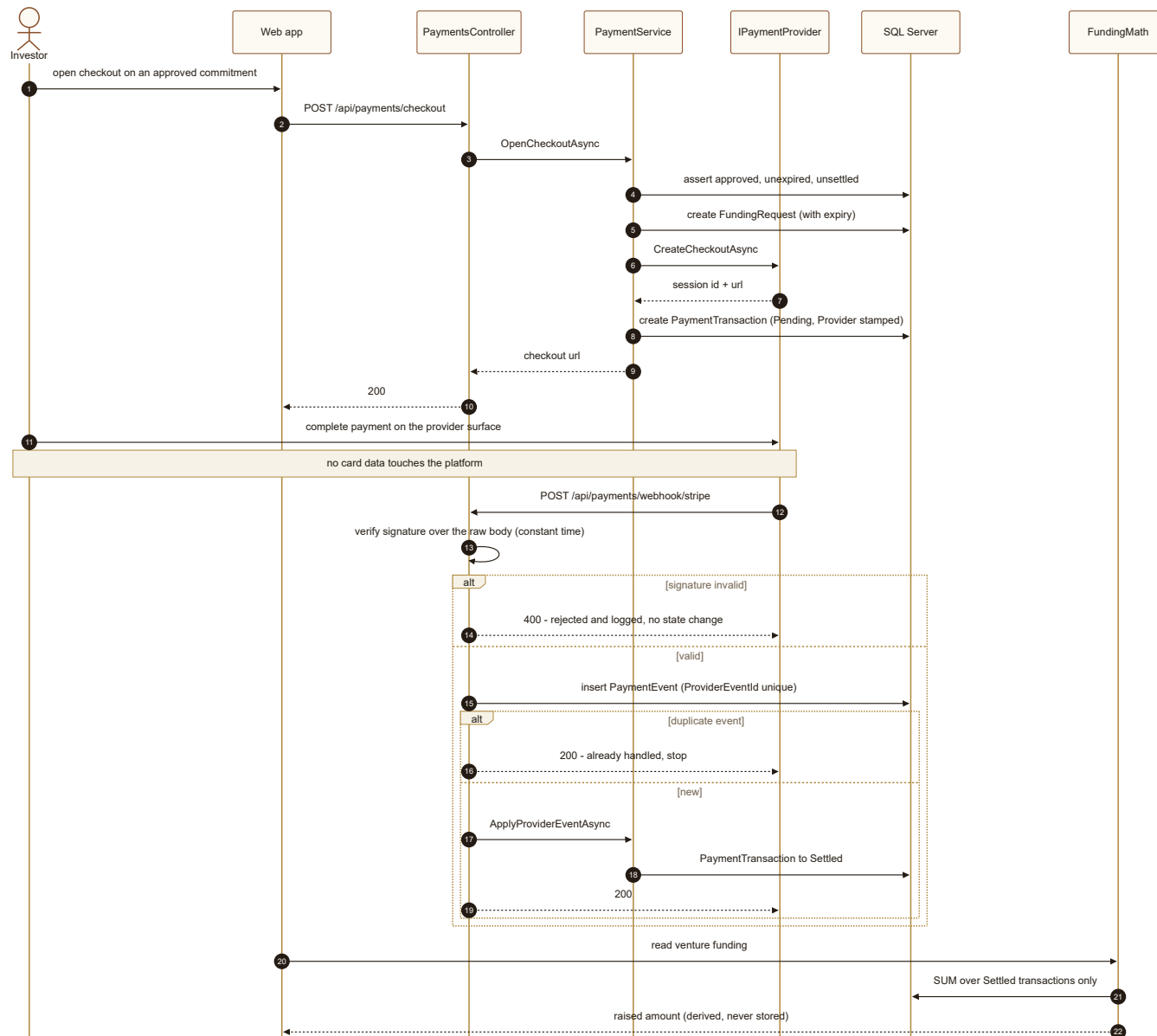


Figure 17 The funding path end to end. Signature verification precedes every state change, the unique event identifier makes duplicate delivery a no-op, and the total is derived at the end rather than incremented along the way.

11.4 Webhook Handling and Signature Verification

A webhook endpoint is unauthenticated by necessity — the provider holds no token — which makes it the most exposed surface in the system. Anyone can post to it.

The defence is signature verification. The provider signs each payload with a shared secret; the endpoint recomputes the signature over the raw body and compares. Three details make this correct rather than merely present:

1. **Verification happens over the raw request body**, before any parsing. Deserialising and re-serialising changes bytes, and the signature is over bytes.
2. **Verification precedes every state change**. An unverified payload is rejected and logged, and no lookup is performed on its contents.
3. **The comparison is constant-time**. A short-circuiting comparison leaks signature bytes through timing.

The verifier is implemented directly rather than taken from a vendor SDK. This was a deliberate choice with a real cost — an SDK would have been fewer lines and less risk — and a real benefit: the mechanism is understood and documented rather than trusted, which is the difference between a system the team can defend in a viva and one it cannot.

11.5 Idempotency and Replay Safety

Providers retry. A callback that receives no response, or a slow response, will arrive again — and a payment system that processes the same event twice has created money.

`PaymentEvent` carries the provider's event identifier under a uniqueness constraint. Processing is therefore: attempt to record the event; if it already exists, acknowledge and stop.

```
1 // The event row is the idempotency key. Recording it and acting on it
2 // happen in one transaction, so a crash between the two is impossible.
3 if (await _db.PaymentEvents.AnyAsync(e => e.ProviderEventId == evt.Id, ct))
4     return Ok(); // already handled - acknowledge and stop
```

Listing 4 Duplicate suppression. The uniqueness constraint, not the check, is what makes this safe under concurrency.

The check alone would leave a race: two concurrent deliveries could both pass it before either wrote. The constraint closes that race — the second insert fails, and the failure is the correct outcome rather than an error.

Acknowledging an unknown or already-collected event with `200` rather than an error is also deliberate: an error response causes the provider to retry indefinitely for an event that will never become processable.

11.6 Payment State Machine

State	Meaning	Exits to
<code>Pending</code>	Checkout created, provider not yet resolved.	<code>Settled</code> , <code>Failed</code> , <code>Expired</code>
<code>Settled</code>	Provider confirmed payment. Counts toward funding.	<code>Refunded</code>
<code>Failed</code>	Provider declined. Does not count.	— (a new request may be opened)
<code>Expired</code>	The time box elapsed with no resolution.	—
<code>Refunded</code>	Settled then reversed. No longer counts.	—

Table 41 Payment transaction states. Only `Settled` contributes to funding.

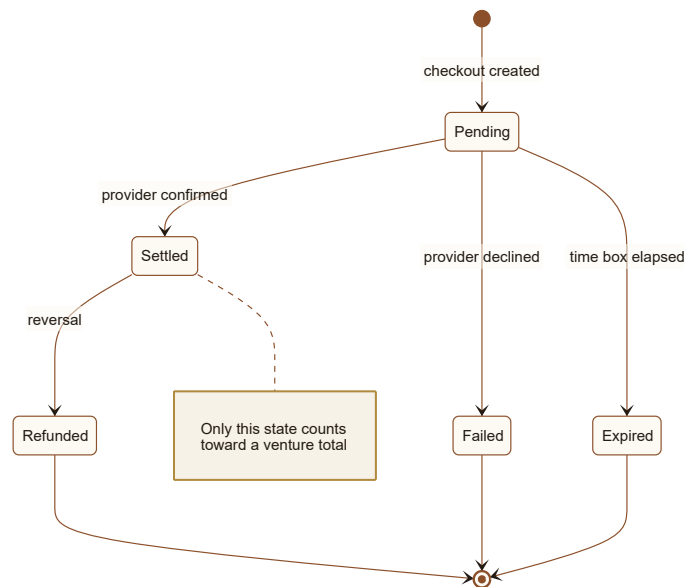


Figure 18 Payment transaction states.

Exactly one state contributes to a funding total. That is the entire integrity rule, and it is expressible in one sentence because the model was built to make it so.

11.7 Funding Integrity Model

Three properties hold, and each is enforced structurally.

Approved is not funded They are different fields on different objects. There is no code path that can conflate them because there is no shared field to conflate.

Totals cannot drift There is nothing to drift. A total is a query over settled transactions, computed in one place. Two callers cannot disagree because there is one implementation.

Duplicate settlement is impossible The uniqueness constraint on the provider event identifier prevents a second processing of the same event, at the database rather than in application logic.

The cost is a read-time aggregation on every page that displays funding progress — which is the hottest read path in the platform. §17.4 measures it and §17.5 describes the caching applied. That cost was accepted knowingly, and it is the correct trade: a stale cached number is recoverable, a drifted stored number is not.

11.8 Expiry, Refunds and Reconciliation

Expiry. Checkouts hold a time box, and abandoned ones are swept by a background component rather than left to be noticed on the next read. Without this, an abandoned checkout would hold an unresolved claim indefinitely.

Fees. The platform fee is configuration expressed in basis points, and it is stored per transaction rather than recomputed. A transaction must remain interpretable after the fee schedule changes; recomputing it would silently rewrite history.

Refunds. A refund is a transition, not a deletion. The transaction stays and changes state, so the record of what happened survives.

Reconciliation. Because every transaction carries its provider name and the provider's own reference, a settled set can be compared line by line against a provider's report. This is the property that makes the model auditable, and it is why **Provider** is stored on every row rather than inferred from configuration — configuration changes, history must not.

11.9 Environment Separation

Provider selection is a startup decision driven by configuration. The default is the simulated provider; setting the provider to Stripe and supplying the secrets selects the Stripe-compatible implementation. Secrets are never committed, and the application refuses to start if a required secret is absent rather than falling back to a placeholder (§16.6).

What has been exercised. The full domain path — commitment, approval, checkout creation, callback verification, deduplication, settlement, expiry sweeping, refund transition and funding recomputation — has been exercised end to end against the simulated provider. Because the rules sit above `IPaymentProvider` (§11.2), that path is the same path the Stripe provider would take.

What has not. Live network interaction with Stripe’s API, real card authorisation, provider-side dispute handling, and settlement timing under production conditions. These require a merchant account the project does not hold. They are listed as limitations in §19.4 and as the first item of future work in §20.3 — not described as complete.

Real-Time Communication

Messaging on this platform is not a chat feature bolted to a marketplace. It is where diligence happens — an investor asks about a number in a pitch, a founder answers, a document is requested. The quality of that exchange is a product requirement, which is why presence, typing indication and read state are treated as first-class rather than as decoration.

12.1 Hub Architecture and Connection Lifecycle

A single hub carries all live traffic. It shares the API's authentication pipeline (§5.8), so a connection is authenticated by the same token as a REST call — there is no second credential and no second identity.

The lifecycle is:

1. The client negotiates a connection, presenting its access token.
2. On connect, the hub records the connection against the user and marks them present.
3. The client joins the conversations it participates in. Membership is checked server-side; a client cannot join a conversation by asking.
4. On disconnect — deliberate or dropped — the connection is removed and, if it was the user's last, presence is updated and a last-seen timestamp is written.

The fourth step contains the subtlety that shapes the whole design: **a user is not a connection**. One person may have three tabs open. Presence is a property of the user, derived from the set of their live connections, and treating a single disconnect as “the user left” produces a status that flickers as tabs close.

12.2 Presence Tracking and Durable Last-Seen

Presence is held in memory, keyed by user, holding a set of connection identifiers. A user is online when that set is non-empty. This is the correct place for it — presence is true only for the lifetime of a process that holds the connections, and persisting it would create a value that outlives its truth.

Last-seen is the opposite and is persisted on the user row. It must survive a restart, because “last seen three hours ago” is exactly the information a user needs when the other party is not online.

ADR-08

Hold presence in memory, persist last-seen

CONTEXT Presence and last-seen look like the same fact but have opposite lifetimes. Presence is only true while connections exist; last-seen must outlive them.

DECISION Track presence in a process-local map keyed by user, holding a set of connection identifiers. Write last-seen to the user row on final disconnect.

CONSEQUENCES Presence is accurate and cheap, and cannot be stale because it cannot outlive the process. Last-seen survives restarts. The cost is that presence does not survive scale-out to more than one instance — a backplane would be required, which §12.7 discusses.

12.3 Typing Indicators and Read Receipts

Typing is transient and is never stored. It is broadcast to the other participant and expires on its own. Persisting a typing indicator would mean writing to the database on every keystroke to record something true for two seconds.

Read receipts are the opposite: a message's read state is a durable property of the message. The design detail that matters is that reading is a server-side event, not a client claim. When a participant opens a conversation, the server marks the relevant messages read and pushes that state to the sender — so both parties see the same truth, and a client cannot assert that it read something it did not.

The result is that a sender sees delivery and read state change live, which is the behaviour users expect from messaging and which is only achievable with a push transport.

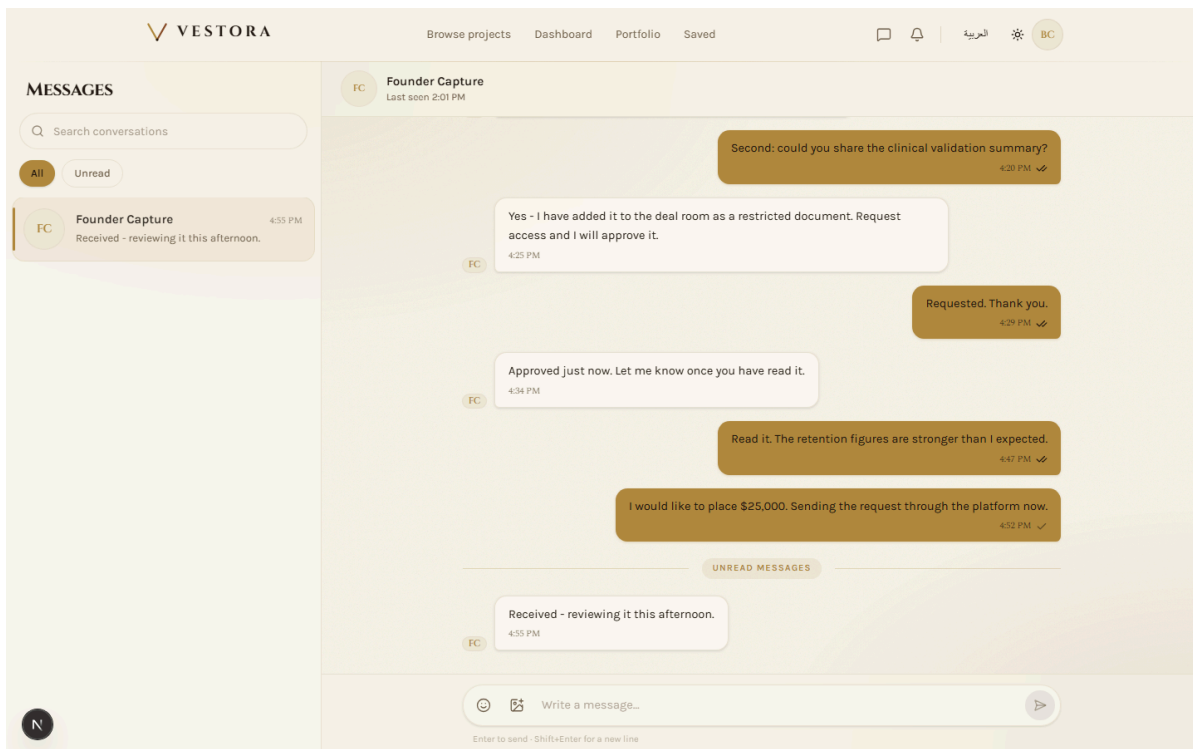


Figure 19 A diligence conversation, seen by the investor. Three mechanisms of this chapter are visible at once: durable last-seen in the header (§12.2), read state on each outgoing message, and the unread divider.

That capture repays a close look at the tick marks. Every outgoing message carries two, except the last, which carries one — the recipient has **received** it and has not **opened** it. The distinction is not cosmetic: the single tick is the server saying **stored and delivered**, and the double tick is the server saying **the other participant opened the conversation**. A client cannot assert either; both are written server-side (§12.4).

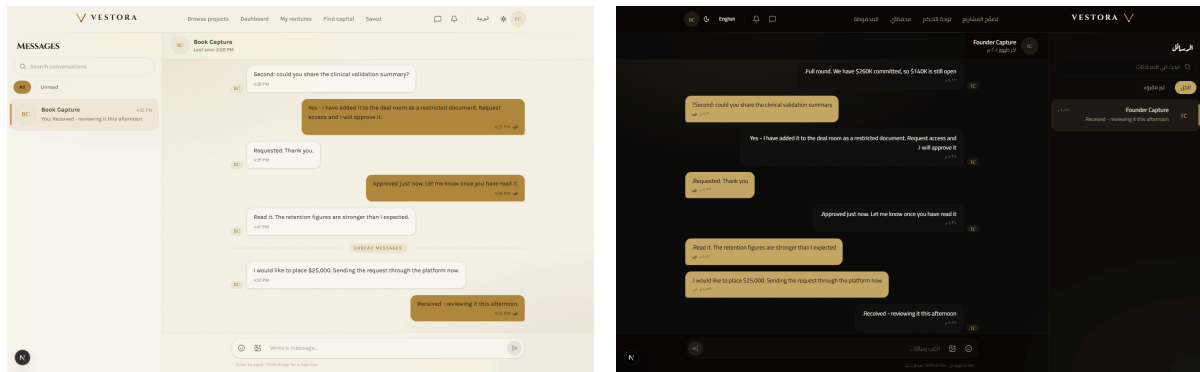


Figure 20 The same thread from the founder's side, and from the investor's side in Arabic under the dark token set. The bubbles swap alignment with the sender, and again with the writing direction.

12.4 Message Persistence and Delivery

Messages are written to the database first and pushed second. The ordering is deliberate: a message that was pushed but not stored is lost on reconnect, and the user who saw it appear has no way to know it is gone.

Persist-then-push gives the property that matters — the database is the record, and the push is an optimisation over polling for it. A client that missed a push because it was disconnected recovers the message by loading the conversation, with no special reconciliation path.

Delivery is therefore **at-least-once from the store**, not exactly-once over the wire. That is the achievable guarantee, and claiming stronger would be false.

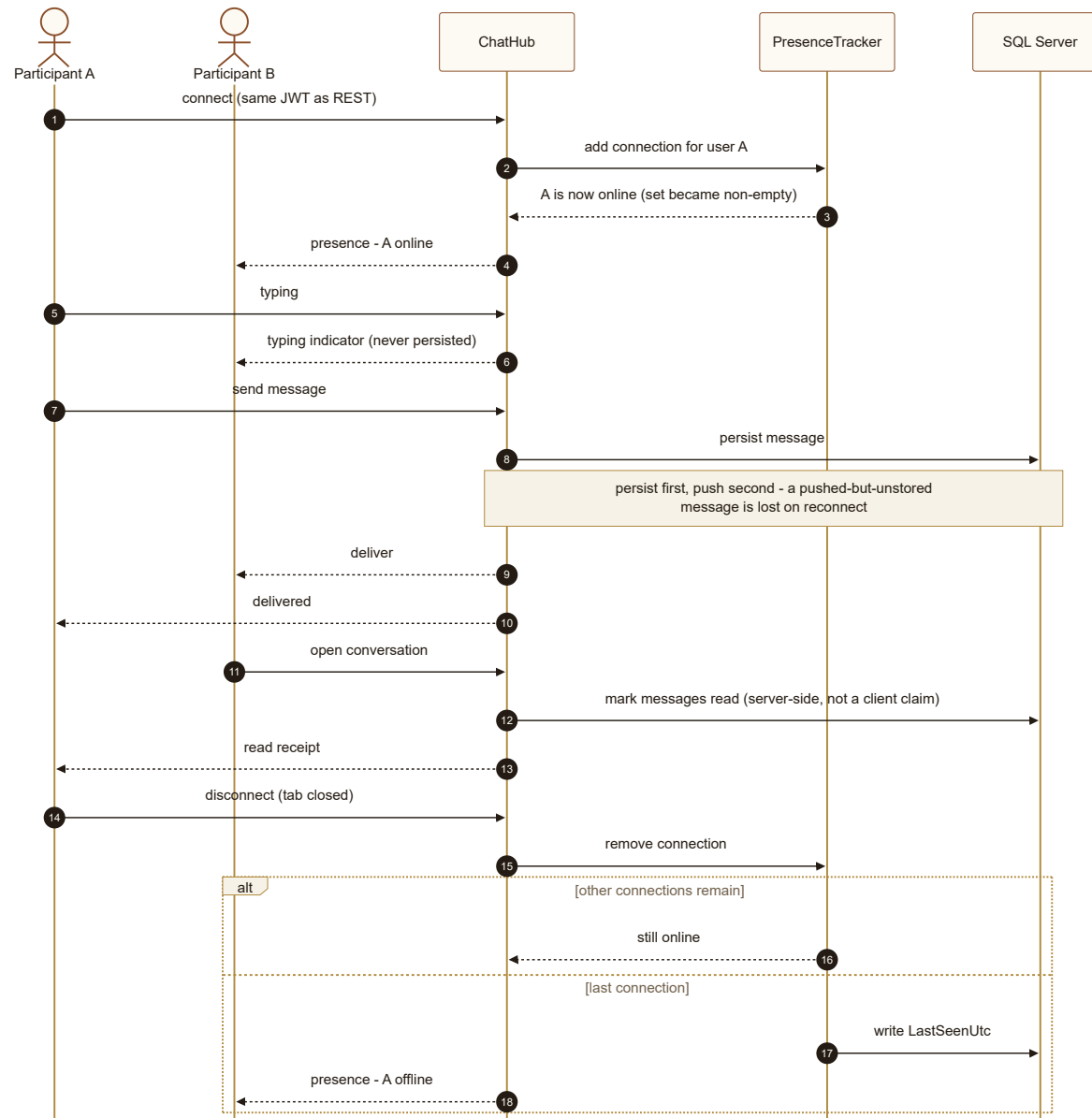


Figure 21 A conversation end to end. Presence is derived from the set of a user's connections, so closing one tab of three does not take them offline; messages are persisted before they are pushed.

12.5 Attachment Handling

Image attachments travel a different path from message text. The file is uploaded through the API and validated by the same file-upload controls as any other image (§10.12) — size limit, content-type allow-list, byte-level verification and a generated filename. Only once the file is stored does a message referencing it get created and pushed.

Doing it in this order means a push never references a file that does not yet exist. Non-image attachments are not supported; that is a stated scope limitation (§1.5.2) rather than an oversight.

12.6 Notification Fan-Out

Notifications are generated by domain events — an investment approved, a milestone published, a document requested — and they are **not** delivered on the request path.

A funding approval that also notified five followers synchronously would make the approval as slow as the slowest notification, and a failure in fan-out would fail the approval. Instead the domain writes the event to an in-process queue and returns; a background worker performs the fan-out.

ADR-09

Move notification fan-out off the request path
CONTEXT Domain actions can generate many notifications. Delivering them synchronously ties the latency and the success of a business operation to the delivery of its side effects.
DECISION Enqueue notification work in-process and deliver it from a background worker.
CONSEQUENCES Business operations return at their own speed and cannot fail because of a notification. The cost is that the queue is process-local: work enqueued and not yet delivered is lost on restart. Acceptable for notifications, and stated as a limitation in §19.4.

The consequence stated in that record is the honest part. An in-process queue is the right size for this system and the wrong answer for one that cannot lose a notification. Naming the boundary is more useful than pretending it is durable.

12.7 Reconnection and Scaling

Reconnection is handled by the client library with backoff, and the persist-then-push design (§12.4) means recovery needs no special path: a reconnecting client reloads the conversation and is current.

Scaling is where this design stops. Presence lives in one process’s memory and the notification queue lives in the same process, so a second instance would have a second, disjoint view of who is online and its own undelivered queue. The system runs as a single instance (§6.9), so this is not currently a defect — but it is a ceiling, and it is the first thing that would have to change.

To scale out, this must change		Into
In-memory presence map		A shared backplane so instances agree on who is connected.
In-process notification queue		A durable queue outside the process.
Single-instance assumption in the hub		Group management routed through the backplane.

Table 42 What horizontal scaling of the real-time layer would require.

Stating this precisely is more useful than claiming the system is scalable. It is correct at its current size, and §17.7 identifies the point at which that stops being true.

Feature Walkthrough I: Identity, Discovery and Engagement

13.1 Walkthrough Method and Notation

The two walkthrough chapters describe what the platform does as **journeys** rather than as endpoint listings. An endpoint listing already exists — it is Section 1 — and repeating it in prose would produce the failure this document was written to avoid: many pages that describe the code without explaining it.

Each feature below is presented the same way: the user's goal, the path through the system, the decision that was not obvious, and the consequence. Where a feature is genuinely a create-read-update-delete surface with no interesting decision, that is stated in a sentence rather than padded into a page.

NOTE

Registration and sign-in are not in this chapter. They are described in §10.3–§10.7, where their security properties are the point. Repeating the journey here would duplicate five sections. This chapter begins after the user has an account.

13.2 User Profile Journey

Goal. Establish enough identity that a stranger on the other side of a funding decision can assess who they are dealing with.

Path. A new account passes through onboarding, which asks for the minimum needed to be useful: a display identity, a short biography, and — for an investor — the sectors they are interested in. The profile is then editable from settings, split into two surfaces: the **profile** the platform displays, and the **account** controls that govern the credential and the account state.

The decision. Splitting profile from account was not cosmetic. They have different risk levels. Changing a biography is trivial; changing an email address invalidates a verified identity and must re-verify. Putting both behind one form would either over-protect the trivial or under-protect the consequential.

Public profiles. Every user has an addressable public profile showing their ventures, their activity and their follower relationships. This is what makes the following system (§13.4) meaningful — there has to be something to follow.

13.3 Browse, Discovery and Search

Goal. Let a venture be found on its attributes rather than through an introduction. This is RQ1 from §1.3, and it is the reason the platform exists.

Path. Discovery is one surface with three ways into it: the public venture listing, a search over that listing, and a feed personalised to what a user follows and has interacted with.

The listing query is the platform's hottest read path, and its shape is determined by the state model in §7.6. A venture is publicly visible only when its moderation status is approved **and** its lifecycle status is active. Those are two different columns with two different writers, so the listing filters on both — which is exactly why the composite index in §7.5 exists.

The decision. Ranking is deterministic and attribute-based: recency, funding progress, and engagement signals derived from real interactions. There is no learned model and no opaque score.

This was a choice, not a limitation of effort. A recommendation model on a platform with few ventures and little interaction history has nothing to learn from, and would produce results the team could not explain. A deterministic ranking can be described to a founder who asks why their venture appears where it does — which on a platform whose premise is fairness is a product requirement, not a technical preference.

Search and filtering. Filters cover sector, funding stage, funding progress and the venture's own attributes. Sort fields are allow-listed server-side (§9.7). Both are paged, and the page size is capped regardless of what the client asks for.

Saved searches. A filter combination can be named and saved, which turns a one-off query into a standing interest. This is the feature that makes discovery repeatable: an investor watching for ventures in a sector does not have to reconstruct the filter each visit.

13.4 Saved Projects and Saved Searches

Goal. Let an investor keep a shortlist without committing to anything.

Path. A bookmark is a single edge between a user and a venture, created and removed from any surface that shows a venture. The watchlist is that set, rendered.

The decision. The uniqueness constraint on the pair is what enforces “saved once”, and it is a database index rather than an application check. An application check leaves a race: two rapid taps on a save control can both pass the check before either writes, producing two rows. The constraint makes the second write fail, and the failure is the correct outcome.

The same reasoning covers saved searches: they belong to a user, they are named, and they are re-runnable rather than snapshots.

13.5 Following and Investor Networks

Goal. Let a user keep receiving a founder's news without watching each venture individually.

Path. A follow is a directed edge between two users. It drives the feed and it drives notification fan-out (§12.6): when a followed founder publishes an update, their followers are notified.

The decisions. Two, both structural. The pair carries a unique index, for the same race reason as bookmarks. And a second index on the followed identifier alone exists because follower counts and follower lists are read far more often than follows are created — an index chosen for a read pattern rather than for the write it constrains.

Investor directory. Investors are themselves browsable, which is the symmetric half of discovery: a founder looking for the right investor has the same discovery problem as an investor looking for the right venture, and solving only one direction would have been solving half the problem.

13.6 Reviews, Comments and Replies

Goal. Let the community add signal to a venture that the founder did not write themselves.

Path. Comments attach to a venture, replies attach to comments — one level of nesting, not arbitrary depth. Reviews are separate from comments and carry a rating.

The decision. Reviews are constrained by a unique index on venture and investor: one review per investor per venture. This is a business rule that would otherwise have to be checked in application code and would otherwise race. It is worth noting as a pattern — this is the fourth place in the system where a rule that reads like policy is implemented as an index (§7.5).

Deletion behaviour. Comments and replies use restrictive delete behaviour (§7.6), so removing a user does not silently erase a conversation others participated in. Reviews cascade from the venture, because a review of a deleted venture refers to nothing.

Reporting. Any user can report a venture. Reports carry a status and are indexed on the venture-and-status pair, because the moderation queue in §14.5 filters on exactly that pair.

13.7 Notification Experience

Goal. Tell a user what happened without requiring them to be present when it did.

Path. Domain events generate notifications; a background worker delivers them (§12.6); the client shows unread state live over the hub connection and renders the history on a dedicated surface.

The decision. Notifications reference domain objects rather than storing rendered text. A notification points at the venture, the investment or the acting user, and the reference is resolved at read time. Storing the rendered sentence would freeze it — a venture renamed after a notification was generated would appear under its old name forever.

That reference carries a cost: the delete behaviour on those references must be restrictive (§7.6), because a notification pointing at a deleted row is a broken notification. This is the trade — a notification that stays accurate, in exchange for references that constrain deletion.

What is not built. There is no email digest and no notification preference model; every notification is delivered in-app to every eligible recipient. Both are listed in §20.3. Stating that here is more useful than describing a preference system that does not exist.

Feature Walkthrough II: Investment, Governance and Insight

This chapter follows the venture from submission to reporting, and the money from intention to settlement. It is where the system's correctness properties become visible as behaviour.

14.1 Investment Workflow

Goal. Let an investor commit capital to a venture, and let both sides know precisely what state that commitment is in.

Path. Five states, and the boundaries between them are the whole design.

State	Means	Moved by
Committed	An investor has expressed intent. Nothing is reserved and no money is involved.	Investor
Approved	The founder or an administrator has accepted the commitment. Still not funded.	Founder / admin
In checkout	A funding request is open with an expiry, and a provider session exists.	Investor
Funded	A payment transaction settled. This is the only state that counts toward a venture's total.	Provider callback
Lapsed	The checkout expired or the provider declined.	System

Table 43 Investment states. Only one of them is money.

The decision that matters. Approved and funded are different states on different objects, and the platform never displays one as the other. A founder sees how much has been **committed** and how much has **settled**, as two numbers, because they are two facts.

The temptation to collapse them is strong: one number is simpler to display and simpler to store. It is also wrong, and it is wrong in the direction that matters — it would show a venture as funded when nothing had been paid.

The consequence. Everything downstream inherits this. `FundingMath` reads only settled transactions (§11.7). The venture's progress bar reflects settlement, not intent. And a founder cannot inflate their own progress by approving commitments, because approval is not the thing that counts.

Investor surfaces. The investor sees this state model directly: a pipeline view of commitments in progress, a payments view of settlement history, a portfolio of what is actually funded, and an activity log. Four surfaces because there are four questions, and one combined view would answer none of them clearly.

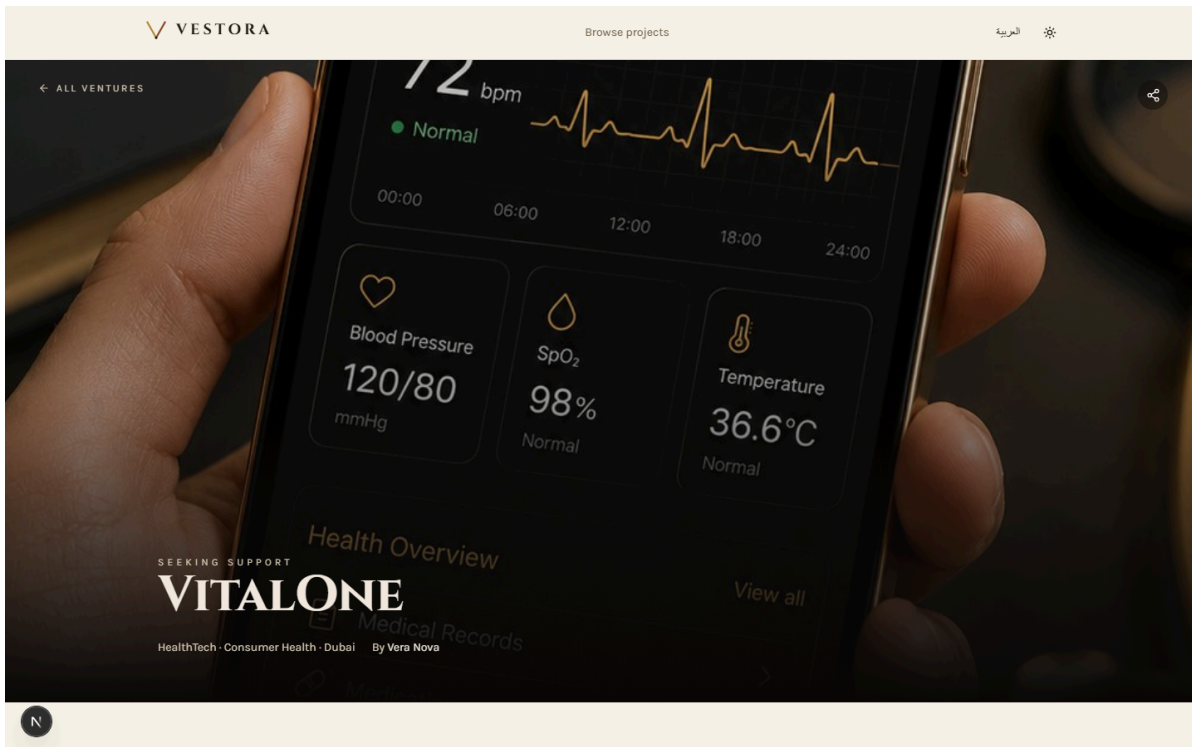


Figure 22 A venture page. The commitment path begins here, and the page leads with evidence — imagery, sector, stage, location and the founder — before it asks for anything.

14.2 Project Lifecycle: Status versus Stage

Goal. Carry a venture from submission to close without any actor's decision overwriting another's.

Path. A venture is created as a draft, submitted for review, approved or rejected by an administrator, published, funded over time, and eventually closed. Three independent columns track it (§7.6): moderation status, lifecycle status, and commercial stage.

Why this is the chapter's second important section. Consider a single status column and this ordinary sequence:

1. A founder submits a venture. Status: **pending**.
2. An administrator approves it. Status: **approved**.
3. Investors fund it. Status: **funding**.
4. The founder edits the description, which requires re-review. Status: **pending**.
5. The administrator approves again. Status: **approved**.

Step five has just erased the fact that the venture was funding. The information was destroyed by an operation that had nothing to do with funding, performed by an actor who was not thinking about funding.

With three columns the same sequence changes only **ModerationStatus**, and the commercial stage is untouched because the administrator does not write it.

Column	Values	Transition rules
ModerationStatus	pending, approved, rejected	Only an administrator writes it. Rejection carries a reason.
LifecycleStatus	active, paused, closed	Written by the owner or by the system when a target is met.
Stage	the commercial progression	Written only by the funding pipeline, and validated against permitted transitions rather than set freely.

Table 44 The three state columns, their values and who may move them.

The consequence. Public visibility requires two of the three to agree — approved **and** active — which is the two-column filter that shapes the listing query and its composite index (§13.3, §7.5). That is the price of the decision, paid once in an index definition.

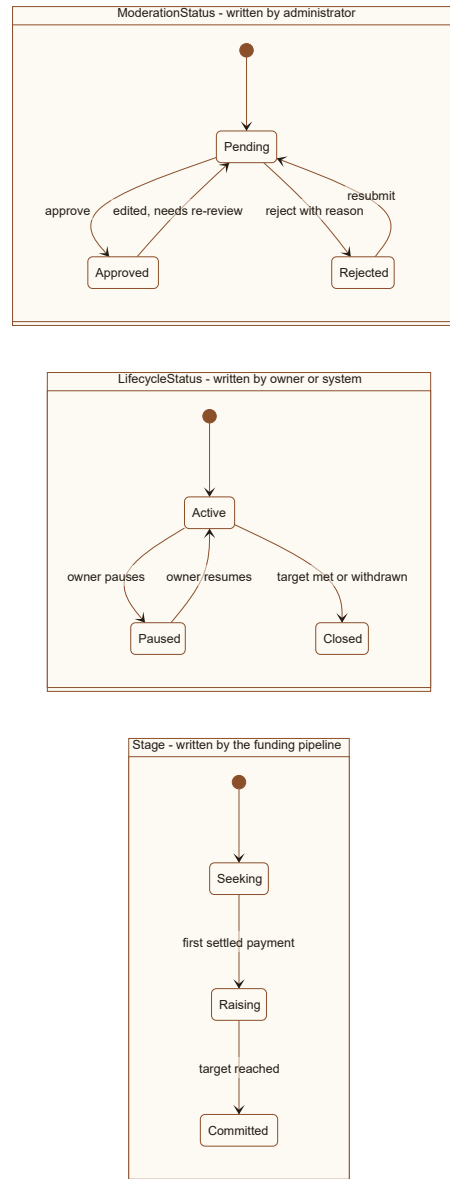


Figure 23 The three state machines of a venture, side by side. They share no transition and no writer, which is why an approval cannot reset funding progress (ADR-05).

14.3 Milestones and Project Updates

Goal. Keep the relationship alive after funding, which is the gap §1.2 identified in existing platforms.

Path. A founder defines milestones — what will be achieved and by when — and publishes updates as work progresses. Updates carry images. Both belong to the venture and cascade with it (§7.6), because a milestone of a deleted venture means nothing.

The decision. Milestones are declarative rather than verified. The platform records what a founder says and when they said it; it does not attest that a milestone was met. Pretending otherwise would be a claim the system cannot support — and a platform that appeared to verify progress it merely recorded would be worse than one that clearly does not.

What the platform does provide is **timestamped, immutable-in-practice history**: an update, once published, is part of the venture's record, and an investor can see the gap between what was promised and what was reported.

Fan-out. Publishing an update notifies followers and investors through the background worker (§12.6), so a founder with many followers does not wait on delivery.

14.4 Document Requests and Deal Room

Goal. Let diligence happen without publishing confidential material to everyone who can see a venture.

Path. A founder attaches documents to a venture with a visibility rule. Open documents are available to anyone who can see the venture. Restricted documents require a request, which the founder grants or refuses. Granted access is recorded, and downloads are logged.

The decision. Access control on documents is enforced at download, not by obscurity of the link. A URL that is hard to guess is not an access control — it is a secret that spreads on the first forward. Every download re-checks entitlement.

Download logging. Every download writes a row. This is not surveillance: it is what lets a founder see that a document was actually read, and it is what makes a disclosure traceable if material leaves the platform.

Deal room. Where a conversation between an investor and a founder becomes specific, it has its own surface, combining the messaging thread (Section 12) with the documents and the commitment under discussion. The design intent is that the three things a diligence conversation refers to are in one place rather than three tabs.

14.5 Administrative Workflow: Review and Moderation

Goal. Ensure nothing reaches the public surface without a human decision, and that every such decision is attributable.

Path. Administrative work is divided by task rather than by entity, because the queues have different rhythms:

Surface	Purpose
Review queue	Ventures awaiting a moderation decision. The primary queue.
Ventures	All ventures, whatever their state, for investigation.
Reports	User-submitted reports, filtered by status — the pair the index in §7.5 serves.
Users	Account administration, including suspension.
Revenue	Platform fee income, derived from settled transactions.
Activity	Platform-wide event stream.
Security	Authentication events from <code>SecurityLog</code> .
Audit	Administrative actions from <code>AdminAuditLog</code> .

Table 45 Administrative surfaces and what each is for.

The decision. The last two surfaces exist because **administrative power must itself be observable**. An administrator can approve, reject and suspend; those actions are recorded against the administrator who took them, in a durable table, readable by other administrators.

A moderation system without an audit trail asks users to trust administrators absolutely. One with an audit trail asks them to trust a process — which is a much smaller request, and the correct one for a platform that mediates money.

Rejection carries a reason. A rejected venture returns to its founder with a stated cause. A rejection without a reason is indistinguishable from a fault, and produces a support burden rather than a corrected submission.

14.6 Role-Based Dashboards

The two dashboards are separate surfaces over largely shared machinery. They are documented together because the interesting part is what differs.

14.6.1 Founder Dashboard

Answers: **is my venture progressing?** It shows the venture portfolio, funding progress split into committed and settled (§14.1), incoming funding requests awaiting the founder’s approval, engagement analytics, and an activity stream.

The funding surface is where the two-number distinction becomes concrete. A founder sees both figures side by side, permanently, because the difference between them is the actionable information — a large gap means approvals are not converting to payments.

14.6.2 Investor Dashboard

Answers: **what have I committed, and what has it done?** It shows the pipeline of in-progress commitments, settled payments, the funded portfolio, the watchlist, and an activity log.

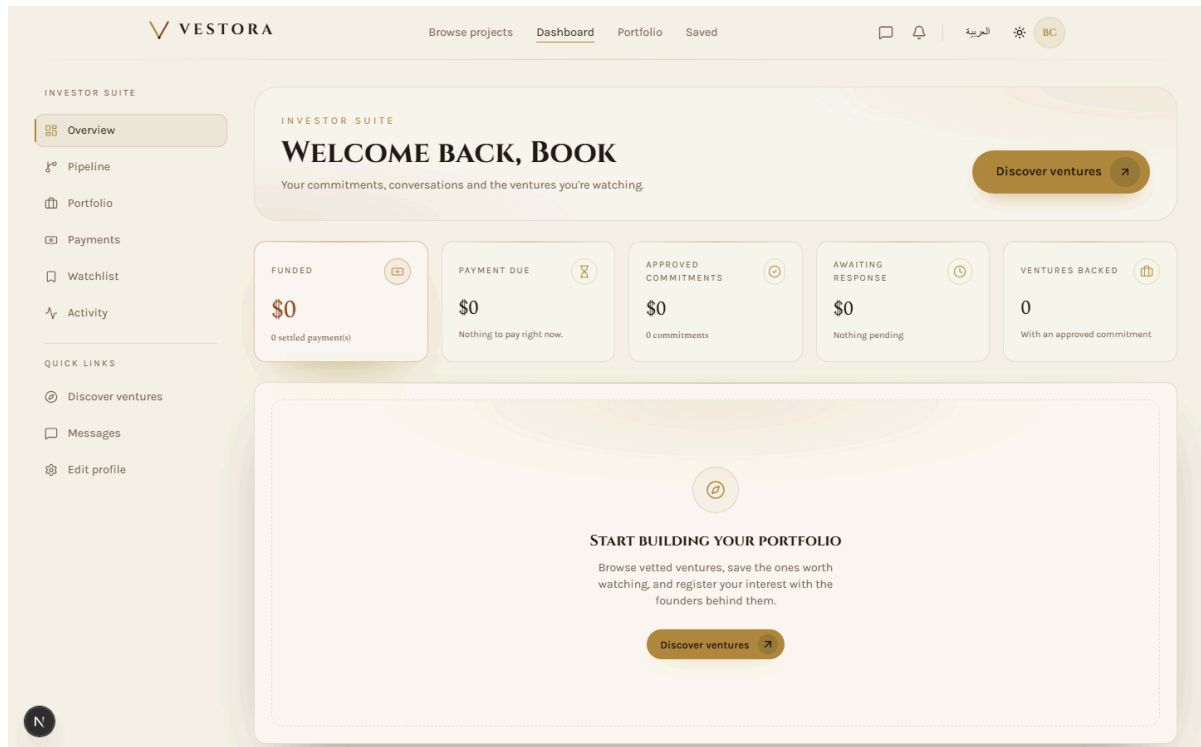


Figure 24 The investor dashboard for a new account. **Funded** and **Approved commitments** are separate tiles — the distinction of \$14.1 is not an internal detail, it is the first thing the surface reports.

That capture is of an **empty** account, deliberately. The tiles still read **\$0 – 0 settled payment(s)** and **\$0 – 0 commitments** rather than collapsing into a single zero, so the two-figure model is legible before a user has any data to disambiguate it. The panel below them explains what the surface is for instead of showing an empty container (§8.8).

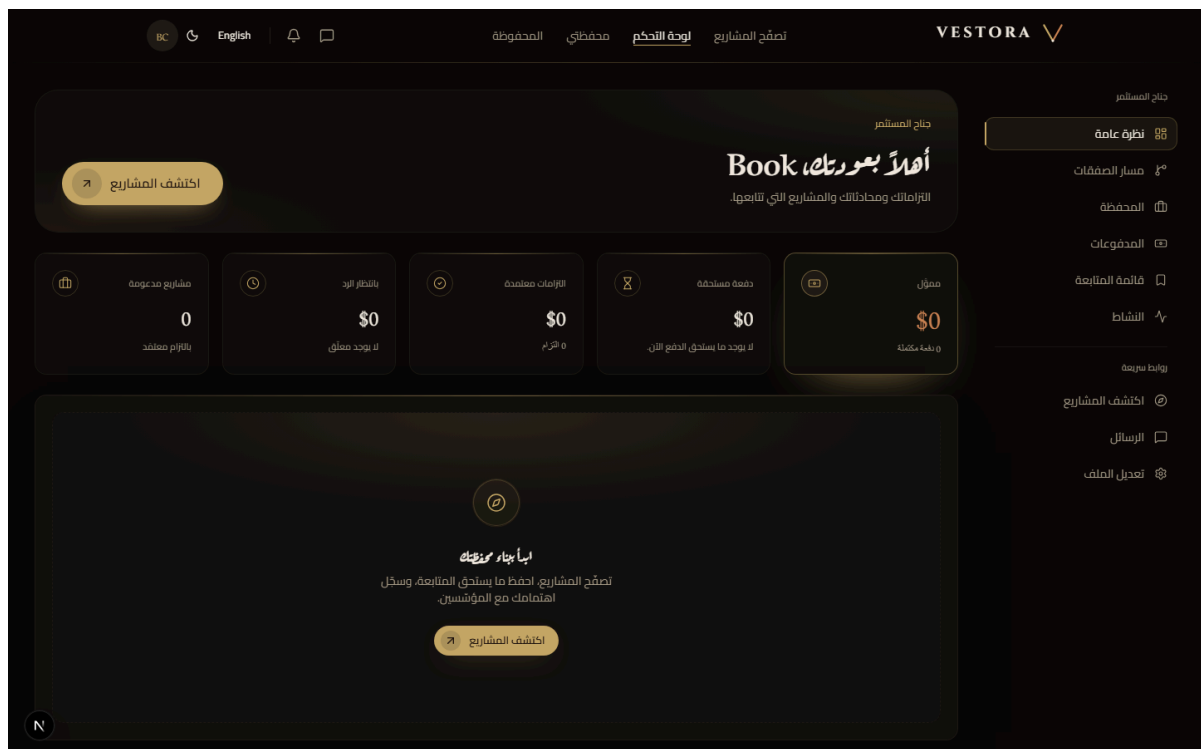


Figure 25 The same dashboard in Arabic under the dark token set. The sidebar, tiles and reading order mirror; the figures and their meaning do not change (§8.6.1).

The shared decision. Both dashboards read through projections rather than loading entity graphs (§6.6). A dashboard is the query-heaviest surface in the platform — several aggregates at once — and materialising full entities to compute counts would make the most-visited authenticated page the slowest.

14.7 Analytics and Reporting

Goal. Turn recorded events into something a founder or an administrator can act on.

Path. Three tiers, each derived rather than stored:

Venture analytics Views over time, engagement, funding progression. Derived from the view and interaction tables.

Founder insight Which ventures attract attention, where interest converts to commitment and where it stops.

Platform analytics Activity volume, moderation load and fee revenue — the last computed from settled transactions and their stored fee amounts (§11.8).

The decision. Analytics are computed from event rows rather than maintained as counters. This is the same argument as §7.6, applied again: a counter can drift and cannot be recomputed, while an aggregation over rows is always correct and can be recomputed from history at any time.

The cost, stated. Aggregating on read is more expensive than reading a counter, and analytics are the second-hottest read path after the venture listing. §17.4 measures this and §17.5 describes what is cached. The trade is accepted for the same reason as in §7.6: a stale cache is recoverable, a drifted counter is not.

PART 4

Quality and Delivery

How the system was verified, how it is deployed and operated, and how it performs under load.

Testing Strategy

Run	Result
Tests executed	41
Passed	41
Failed	0
Skipped	0
Wall time	34 ms

Table 46 `dotnet test` against `MyAppApi.Tests`. Figures are from the run, not from a plan.

The suite covers the **domain** layer — the components §6.4 separated out precisely because they carry rules and perform no I/O. That separation is what makes a 34 ms suite possible: none of these tests touches a database.

NOTE

What is not covered. Integration, authorisation, security and end-to-end cases (Section 4) remain specified and unexecuted — they need a test database and a driven browser. The forty-one executed tests are the domain cases only. §19.2 reports the resulting coverage of requirements, which is partial and stated as such.

15.1 Testing Approach

Testing effort follows risk, not code volume. The distribution below is deliberate and is the direct consequence of what Section 18 established: the expensive defects in this project were in the **domain**, not in the plumbing.

Layer	What it verifies	Why this share of effort
Unit	Domain rules in isolation: funding arithmetic, stage transitions, account rules, payment state changes.	Highest share. These are the rules whose failure is most costly and whose verification is cheapest — no I/O, so they run in milliseconds.
Integration	A request through controller, service and database against a real schema.	Second highest. Catches the class of defect that unit tests structurally cannot: wrong query, wrong mapping, wrong constraint.
API	The contract as a client sees it: status codes, response shapes, authorisation.	Moderate. Protects the boundary the front-end depends on.
End-to-end	Whole journeys through the real interface.	Lowest. Slow, brittle, and valuable only for the few journeys where the whole chain must hold.

Table 47 Test layers and the reasoning behind the effort split.

15.2 Unit Testing

Unit tests target the components that carry rules and perform no I/O — which is exactly the set that §6.4 separated out for this purpose. `FundingMath`, `PipelineStages`, `AccountRules` and the payment state logic are all testable without a database, and that is a property of the architecture rather than of the tests.

The cases that matter most are the ones derived from Section 18:

Case	Asserts	Guards
FM-01	An approved but unsettled commitment contributes zero to a venture's total.	§11.7, the defect in §18.2
FM-02	A refunded transaction is removed from the total.	§11.6
FM-03	Totals computed twice over the same data are identical.	Derivation correctness
PS-01	A moderation change does not alter commercial stage.	§7.6, the defect in §18.3
PS-02	An invalid stage transition is refused rather than applied.	§14.2
AR-01	A founder cannot commit to their own venture.	FR-30
PT-01	Only a settled transaction is countable.	§11.6

Table 48 Domain unit cases and the specific failure each prevents from recurring.

Every row in that table exists because something went wrong. A test suite assembled from a coverage target tests what is easy; one assembled from a defect history tests what breaks.

Coverage. Collected with `coverlet` during the run.

Scope	Line	Branch
<code>PipelineStages</code>	100%	100%
<code>FundingMath</code>	25.8%	80.0%
Whole solution	0.06%	1.1%

Table 49 Coverage by scope. The three rows are the same run measured at three altitudes.

That table is the argument of this section in three lines. `PipelineStages` is fully covered because it is pure. `FundingMath` reports 25.8% of lines but **80% of branches** — the uncovered lines are its asynchronous database loaders and its LINQ expression trees, which cannot execute without a database; every decision the class actually makes is exercised.

And the solution-wide figure is **0.06%**, which is worse than useless. It is diluted by twenty-two controllers, several hundred DTO properties and the generated model — code that carries no decisions. Reporting that number as “the project's coverage” would be technically true and completely uninformative, which is why coverage here is reported per class and the aggregate is shown only to demonstrate why it is not used.

15.3 Integration Testing

Integration tests exercise a request end to end within the API — controller, service, data context, real schema — and they are where the constraints of §7.6 are verified as constraints rather than as intentions.

Three groups matter:

Constraint enforcement That a duplicate bookmark, a second review by the same investor, or a duplicate follow edge is **rejected by the database**, not merely avoided by application code. These tests deliberately attempt the invalid write.

Idempotency That replaying a provider callback with an event identifier already recorded changes nothing and returns success (FR-27). This is the test that would have to fail for money to be double-counted.

Authorisation That each row of the matrix in §10.9 holds at the endpoint — including the negative cases, which are the ones that matter. A test that a founder **can** edit their venture is worth less than a test that another user cannot.

15.4 API Testing

The API surface is exercised as a client sees it: correct status codes, the `401`/`403` distinction preserved (§9.1), paged responses carrying their metadata, and error bodies disclosing nothing internal (§10.11).

These are maintained as a collection that can be run against any environment, which makes them serve double duty as a post-deployment smoke check (§16.11).

15.5 End-to-End Testing

Four journeys are covered end to end, chosen because each spans components that unit and integration tests verify only separately:

1. Register, verify, sign in, complete onboarding.
2. Create a venture, submit it, approve it as an administrator, see it appear publicly.
3. Commit to a venture, approve the commitment, complete checkout, observe funding progress change.
4. Open a conversation, exchange messages, observe read state update live.

The third is the most valuable test in the entire suite. It is the only one that verifies the property the whole system is built around: that progress moves on **settlement** and not before.

Determinism. Motion is disabled in the test environment (§18.7). The tests assert on state rather than on timing, and no test contains a fixed wait.

15.6 Security Testing

Security testing is targeted at the threat model in §10.1 rather than performed generically.

- Authorisation bypass attempts against each administrative endpoint from a standard account.
- Direct object reference attempts: requesting another user's investment, message thread or restricted document by identifier.
- Mass-assignment attempts: submitting a moderation status inside a venture update payload, which the DTO boundary (§10.11) must ignore.
- Upload attempts with a mismatched content type and with a crafted filename (§10.12).
- Webhook posts with an absent, malformed and incorrect signature (§11.4).

The last of these is the highest-value security test in the project: the webhook endpoint is unauthenticated by necessity, and signature verification is the only thing standing between an anonymous request and a state change involving money.

15.7 Defect Management

Defects are tracked with severity, the component affected, and — where the fix was structural — a link to the section that records the decision.

Defect	Severity	Resolution
Approval increased funding progress before payment.	Critical	Model change, not a patch. §18.2, ADR-04.
Re-approval reset a venture’s commercial stage.	High	Three-column state model. §18.3, ADR-05.
Timestamps shifted after deployment.	High	UTC enforced at serialisation. §18.1, ADR-07.
Text invisible under an inverted theme scope.	Medium	Surface-relative token resolution. §18.6.
Interface tests failed intermittently.	Medium	Motion disabled under test. §18.7.

Table 50 Significant defects and how each was resolved. Note that the two most severe were resolved by changing the model, not the code.

The full catalogue is in Section 4. Defects above were found by inspection and manual exercise, not by an automated suite.

15.8 Requirements-to-Test Traceability

Every requirement in Section 4 maps forward to the test that verifies it, in Section 3. The purpose is to make two things visible that are otherwise asserted:

1. **Which requirements are verified.** A requirement with an implementation and no test is implemented but unverified, and the matrix says so.
2. **Which tests exist for no stated requirement.** These are either missing requirements or unnecessary tests, and both are worth knowing about.

§19.2 reports what the completed matrix shows — including the gaps, which are stated rather than closed by relabelling.

Deployment and Operations

This chapter covers getting the system running and keeping it running. The two are treated together because separating them produces documentation that explains how to deploy and not how to operate — which is the more common failure of the two.

16.1 Environment Strategy

Environment	Purpose	Payment provider
Development	Local machines. Runs against the shared hosted database (§16.3).	Simulated
Staging	Pre-release verification against a production-like configuration.	Simulated or Stripe sandbox
Production	The deployed system.	Configured; simulated by default

Table 51 Environments and their payment configuration.

Environments differ only by configuration. There is no code path conditioned on the environment name, because a system that behaves differently in production than in staging has not been tested in staging.

16.2 Hosting Architecture

The API and the real-time hub deploy as a single artefact to a managed ASP.NET Core host. The web application deploys separately. The database is a managed SQL Server instance on the same provider. There is one instance of each. No load balancer, no autoscaling, no read replica. §17.7 identifies which of those would be needed first and at what point; describing an architecture the project does not have would be worse than describing a small one accurately.

The hub is co-located with the API deliberately. It shares the authentication pipeline and the data context (§12.1), and separating it would require a backplane the hosting arrangement does not provide (§12.7).

16.3 Database Hosting and Connection Security

The database is hosted rather than self-administered. The connection string carries credentials and is supplied by environment configuration only — it appears in no file in the repository, and the application refuses to start without it (§16.6).

The consequence for development is discussed as a challenge in §18.5: there is no local database, so all developers share one schema. This is convenient and carries a real cost, and it is recorded as a limitation rather than as a practice.

16.4 Build and Release

The release sequence is:

1. Apply any pending migrations against the target database (§7.7).
2. Build and publish the API artefact.
3. Build the web application.
4. Deploy both.
5. Run the API test collection against the deployed environment as a smoke check (§15.4).

Migrations run **before** the application that depends on them, and additive migrations (§7.7) are what make this ordering safe: a schema with a new nullable column is compatible with the previous application version, so the two steps do not have to be simultaneous.

Continuous integration. There is none. Builds and deployments are performed manually against a checklist (§16.11). This is a genuine gap — automated builds on every change would have caught several integration failures earlier — and it is recorded in §19.4. The hosting arrangement provides no managed CI, and introducing an external runner was judged out of scope against the schedule.

16.5 Configuration Management

All environment-varying values are configuration, never constants. This is a rule with no exceptions, and the categories are:

Category	Examples
Connection	Database connection string.
Identity	Token signing key, access and refresh lifetimes, lockout threshold and window (§5.6).
Payments	Provider selection, provider keys, webhook signing secret, platform fee in basis points, checkout expiry (§11.9).
Email	SMTP host, port, credentials, sender identity (§5.9).
Uploads	Maximum size, permitted content types (§10.12).
Origins	The allowed cross-origin list (§10.11).

Table 52 Configuration categories. Every value here differs by environment or is a secret.

The reason this is a rule rather than a preference: a value compiled into the application cannot be changed without a release, which means an urgent change — tightening a lockout threshold under attack, disabling a provider — requires a deployment at exactly the moment a deployment is least welcome.

16.6 Secrets Management

Secrets are never committed. They are supplied by the platform’s own configuration mechanism in production and by the local user-secrets store in development, so that a working development machine holds secrets outside the repository directory entirely.

ADR-10

Fail to start on a missing secret rather than falling back

CONTEXT A missing signing key or provider secret can be defaulted to a placeholder so the application starts. The application then runs with a known key, or silently accepts unverified webhooks.

DECISION Validate required configuration at startup and refuse to start when any is absent.

CONSEQUENCES A misconfigured deployment fails loudly and immediately instead of running insecurely. The cost is that a first-time setup must supply every secret before the application will run at all — which is the correct inconvenience.

That decision is the difference between a misconfiguration being a visible outage and being a silent vulnerability. An application that starts with a default signing key is not degraded; it is compromised, and nothing indicates it.

16.7 Logging

Logging is structured and levelled. Levels are used for their meaning: **Information** for lifecycle events, **Warning** for handled conditions that should not recur, **Error** for unhandled failures.

What is never logged. Passwords, tokens, connection strings, provider secrets, and full request bodies on authentication endpoints. A log stream is frequently the least protected copy of the data a system holds.

What is logged beyond the stream. Security-relevant and administrative events are written to durable tables — **SecurityLog** and **AdminAuditLog** (§10.13) — because a log stream is not queryable evidence and, on this hosting arrangement, is not retained indefinitely.

16.8 Monitoring and Health Checks

A health endpoint reports whether the application is running and whether it can reach the database. This distinction matters: an application that is up but cannot reach its database is not healthy, and an endpoint that reports only process liveness would say it is.

What is monitored. Application availability through the health endpoint, and database availability through it.

What is not monitored. There is no metrics pipeline, no request-rate or latency dashboard, and no distributed tracing. Performance figures in Section 17 come from deliberate measurement runs rather than from continuous collection.

This is the weakest area of the project's operational posture and it is stated plainly. The hosting arrangement provides limited observability (§5.11), and building a metrics pipeline was traded against delivering the domain. §19.4 records it.

16.9 Error Reporting

Unhandled exceptions are caught centrally, logged with full detail, and returned to the client as a generic failure carrying a correlation identifier. The identifier is the useful part: a user reporting a problem can quote it, and it locates the exact log entry without the response body having disclosed anything (§10.11).

There is no automated alerting. Failures are discovered by inspection or by report. Recorded in §19.4.

16.10 Backup, Rollback and Recovery

Database backup The provider performs scheduled backups with point-in-time restore inside its retention window. This is a dependency, not an implementation, and the project's obligation is to know its bounds (§7.8).

Schema recovery Fully reproducible from the migration history in version control. An empty instance plus migrations yields the current schema.

Application rollback Redeploy the previous artefact. Safe only when no migration has been applied since; a migration that dropped or renamed a column would make the previous version incompatible. This is why additive migrations are preferred (§7.7) — they keep rollback available.

Uploaded files Not backed up. Images, documents and attachments live outside the database and have no backup arrangement. This is a real gap, recorded in §7.8 and §19.4.

16.11 Release Checklist

Because releases are manual (§16.4), they follow a written checklist rather than recollection.

PRE-RELEASE

1. All tests pass locally.
2. Pending migrations reviewed — confirm each is additive, or plan for the loss of rollback.
3. Configuration diff reviewed; any new required setting present in the target environment.
4. New secrets provisioned before deployment, not after (§16.6).

RELEASE

5. Apply migrations to the target database.
6. Deploy the API artefact.
7. Deploy the web application.

POST-RELEASE

8. Health endpoint returns healthy, including database reachability.
9. API smoke collection passes against the deployed environment (§15.4).
10. Sign-in, venture listing and a checkout against the simulated provider verified by hand.
11. Error log inspected for anything new.

Step 4 exists because it has been forgotten. A deployment that starts before its secrets are in place fails at startup by design (ADR-10) — which is the correct behaviour and an avoidable outage.

Performance and Scalability

Figures in this chapter come from an instrumented run against the deployed database, using the script in `book/measure.js`. Frontend timings are the median of five cold loads with the cache disabled; API figures are the distribution over thirty warm requests. Nothing here is estimated.

NOTE

One target is not met. NFR-02 requires the venture listing to return within 300 ms at the 95th percentile. It measures **519 ms**. §17.3 reports the number and §17.4 explains where it goes; it is not rounded down, re-scoped or moved to future work.

17.1 Performance Budget and Targets

Targets come from the non-functional requirements in §4.4, restated here with the measurement method attached. A target without a method is not checkable.

ID	Target	Method	Result
NFR-01	Primary content of public venture pages present in the first response.	Response inspection	in-Met
NFR-02	Venture listing p95 under 300 ms at expected load.	30-request run, §17.3	Not met — 519 ms
NFR-03	No unbounded collection response.	Endpoint re-view	Met
NFR-14	Usable 360–1920 px.	Device matrix, §8.6	Met

Table 53 Performance targets, how each is checked, and its state.

Two of these are structural and either hold or do not — a page either renders on the server or it does not, and an endpoint either caps its page size or it does not. Those are marked met because they were verified by inspection rather than by measurement.

17.2 Frontend Performance

Server rendering. Public pages render on the server (§5.2), which was one of the three reasons for the framework migration. The content a first-time visitor came for is in the first response rather than after two round trips.

Bundle size. Route-level code splitting means a visitor to the venture listing does not download the administrative interface. The heaviest authenticated surfaces — dashboards and the administrative area — are separate entry points.

Images. Venture imagery is the largest asset class on the platform. Uploads are size-capped at the boundary (§10.12), and images are served in modern formats at the size they are displayed rather than at the size they were uploaded.

Fonts. Three families are loaded (§8.2), subset and served with a fallback metric so that text is visible during load rather than invisible. A display face that blocks first paint would defeat the purpose of server-rendering the page it sits on.

Measured. Median of five cold loads, cache disabled, at 1440×900.

Page	TTFB	FCP	DOM ready	Load	HTML
Landing	147 ms	468 ms	430 ms	739 ms	21 KB
Venture listing	92 ms	320 ms	203 ms	448 ms	12 KB

Table 54 Frontend timings. Both pages paint content in under half a second.

Two things are worth reading from that table. First, time to first byte is under 150 ms on both, which is the server-rendering decision of §5.2 doing what it was chosen for — the document arrives with its content rather than with a loader. Second, the listing page is **faster** than the landing page despite carrying more data, because the landing page’s hero imagery dominates its load.

17.3 API Response Times

Measured over thirty warm requests per endpoint against the hosted database.

Endpoint	p50	p95	p99	max
<code>GET /api/projects</code> (page 1)	495 ms	519 ms	662 ms	662 ms
<code>GET /api/projects</code> (page 2)	475 ms	500 ms	506 ms	506 ms

Table 55 Listing latency. NFR-02 requires p95 below 300 ms; it is not met.

The listing endpoint is the one that matters. It is the platform’s most-visited surface, it filters on two state columns (§14.2), and it aggregates funding totals per row (§7.6). It is by construction the query most likely to be slow, and it is the one NFR-02 constrains — and it misses by roughly 220 ms.

Where the time goes. The distribution is the diagnostic: p50 and p95 are 24 ms apart, and the minimum is 472 ms. A query with a slow **plan** varies; a query with a slow **floor** is paying a fixed cost. That floor is the round trip to a hosted database on another network (§16.3) — every request pays it before any work happens. Page 2 measures marginally faster than page 1, which confirms the cost is not in the row count.

What this means. The target was set for a co-located database and the system runs against a remote one. The honest reading is that NFR-02 was specified without accounting for the hosting decision in §5.11, and the mitigation is in §17.5 — the aggregate cache removes the repeated portion, but not the floor. Closing the gap properly requires the database and the application to share a network.

NOTE

Two other endpoints were sampled and are excluded: `/api/feed` and `/api/signals` returned in 2–4 ms because the requests were unauthenticated and rejected before any work. A rejection is not a measurement, and reporting those figures as endpoint latency would be false.

17.4 Database Query Optimisation

Three optimisations were applied, in the order a database problem should be addressed: fix the query, then index it, then cache it.

17.4.1 Projection instead of materialisation

The first version of the listing loaded venture entities and mapped them in memory. A venture entity has roughly forty columns and several navigation properties; a listing card displays eight fields.

The fix was to project into the card DTO inside the query, so the database returns eight columns and the application allocates nothing it does not use (§6.6, §9.4). This is the single highest-return change in the chapter, and it required no index and no cache — only asking for less.

17.4.2 Eliminating repeated queries

Aggregating funding totals per venture in a listing invites the classic failure: one query for the list, then one per row for its total. Twenty ventures become twenty-one queries.

The aggregation is instead performed as part of the listing query, so the row count does not determine the query count. This is the cost that ADR-04 (§7.6) knowingly accepted, and it is where that cost is paid down.

17.4.3 Indexing

The composite index on moderation status, lifecycle status and creation date (§7.5) exists precisely for the listing query — it covers both filter columns and the sort in one seek. The other indexes in §7.5 follow the same rule: each was added for a query that needed it, and none speculatively.

No before-and-after plan comparison is offered, and the reason is worth stating rather than hiding behind a missing figure: the unoptimised versions no longer exist. The projection replaced the materialising query and the composite index was added in the same period, so there is no “before” to profile without reconstructing code that was deleted. What can be said is what §17.3 measures — the latency floor is the database round trip, not the plan, and page 2 costs the same as page 1, which is the signature of a query whose cost does not scale with the rows it returns.

17.5 Caching Strategy

Caching is applied narrowly, because a cache is a second copy of the truth and every one of them can be wrong.

What is cached Derived funding aggregates for public venture listings, with a short lifetime and invalidation on settlement. This is the read that ADR-04 made expensive, and it is the only place where the expense is significant enough to justify a cache.

What is not cached Anything user-specific — dashboards, portfolios, conversations, notification state. These are neither shareable between users nor stable enough for a cache to help.

Why the trade is safe A stale funding figure is bounded by the cache lifetime and self-corrects. A drifted stored counter (§7.6) is unbounded and does not. This is the same argument as ADR-04, applied to its own consequence.

17.6 Load Testing

A concurrency sweep was run against the listing endpoint. The results below are reported with an important qualification about the harness, because reading them naively would produce a false conclusion.

Concur- rency	p50	p95	max	Errors	Throughput
1	666 ms	1204 ms	1204 ms	0 / 10	1.4 req/s
5	518 ms	846 ms	870 ms	0 / 50	8.7 req/s
10	7 ms	963 ms	1102 ms	60 / 100	29.1 req/s
20	18 ms	24 ms	31 ms	200 / 200	1034 req/s

Table 56 Concurrency sweep. The last two rows measure the harness, not the server — see below.

The last two rows are not a server finding. Read alone they say the platform collapses at ten concurrent readers. That conclusion is wrong, and the numbers themselves show why: at concurrency 20 the failures return in 18 ms at over a thousand per second. A saturated server does not fail **faster** than it succeeds. Those are client-side aborts — the harness runs `fetch` from a browser page, and a browser caps concurrent connections per origin, so requests beyond the cap were refused locally before reaching the API.

This was confirmed directly: twenty concurrent requests issued on their own, without the preceding waves, returned **twenty** `200` responses. The failures are an artefact of running four sweeps in sequence through one browser context.

What can be concluded. Only the first two rows are valid measurements, and they say the single instance serves five concurrent readers without error at roughly 8.7 requests per second, with latency dominated by the fixed database round trip identified in §17.3. Above that the harness stops being a measurement instrument.

What cannot be concluded, and how to get it. The point at which the server itself degrades is unknown. Establishing it needs a load generator that is not a browser — a dedicated tool issuing requests from outside a per-origin connection limit. That is recorded in §19.4 as a limitation and in §20.3 as work, rather than being asserted from data that cannot support it.

NOTE

This section could have shown four rows and a dramatic failure curve. It shows two valid rows and an explanation of why the other two are noise, because a measurement whose instrument is untrustworthy is not a smaller result — it is a different claim entirely.

17.7 Scalability

Current position. One API instance, one database, no cache tier, no queue. This is adequate at the platform's current size and it is a ceiling, not a plateau.

What breaks first, in order. Stating the sequence is more useful than asserting that the system is scalable.

	Constraint	What it requires
1	Read load on the listing and detail pages.	A shared cache tier, replacing the in-process cache of §17.5.
2	Real-time presence and notification fan-out are process-local (§12.7).	A backplane for presence and a durable queue for notifications. This is the first change that requires more than configuration.
3	Single API instance is a single point of failure.	More than one instance behind a load balancer — which is only possible after constraint 2 is resolved.
4	Database read contention on aggregate queries.	A read replica for analytics and listing reads.
5	Uploaded files served from application storage.	Object storage behind a content delivery network — which would also close the backup gap in §7.8.

Table 57 Scaling constraints in the order they would bind.

The ordering matters more than the list. Constraint 2 must be resolved before constraint 3 can be, because adding a second instance while presence is process-local produces two disjoint views of who is online — a system that is worse than the single instance it replaced.

What does not need to change. The domain layer is stateless and the data model is unchanged by any row above. The funding invariants of §7.6 remain enforced by the database regardless of how many application instances exist — which is one of the properties the monolith decision (ADR-06) was chosen to preserve.

PART 5

Closure

What proved hard, what the results show, and what comes next.

Challenges and Solutions

This chapter records the problems that cost the most time. Each is presented in the same form — the symptom, what was initially believed, what was actually wrong, the fix, and what the fix cost — because a problem described only by its solution teaches nothing.

The selection criterion is not severity. It is **how wrong the first diagnosis was**. A bug that took an hour and was exactly what it looked like is not in this chapter. A bug that took three days because it looked like something else is.

18.1 Timestamps That Were Right Everywhere Except Production

Symptom. Times displayed in the interface were correct during development and wrong after deployment — shifted by a fixed number of hours. Investment dates, message timestamps and audit entries were all affected equally.

What was initially believed. That the database was storing local time. Several hours went into inspecting stored values, which were correct: every row held UTC, exactly as intended.

What was actually wrong. The values were correct in the database and correct in memory. The defect was at the **serialisation boundary**: a timestamp was being written into JSON without an explicit offset. The client received a time with no zone information and did the only thing it could — interpreted it in its own zone.

Why it was invisible in development. Every developer machine and the local database agreed on a timezone. A value serialised without an offset and re-interpreted in the same zone round-trips perfectly. The defect could only appear where server and client zones differed, which is to say: only after deployment.

The fix. An explicit UTC converter applied in the serialisation pipeline, so that every emitted timestamp carries its offset regardless of host configuration. Recorded as ADR-07 (§6.7).

What it cost. Roughly three days, most of it spent looking in the wrong layer. The lesson that generalised: **a value that is correct at rest can still be wrong on the wire**. Storage correctness and transport correctness are separate properties, and this project had only been checking the first.

18.2 Approval Silently Counting as Funding

Symptom. A venture’s funding progress increased when a founder approved a commitment, before any payment had been made. Investors saw ventures reporting progress that no money supported.

What was initially believed. A bug in the approval handler — an increment in the wrong branch.

What was actually wrong. Not a bug in a handler. A defect in the **model**. The early schema carried a single amount on the investment and no separate concept of settlement, so “approved” and “paid” had nowhere to live separately. Every code path that wanted to know how much a venture had raised was reading a figure that included intentions.

Patching the handler would have moved the increment. It would not have removed the ability to make the same mistake again, because the model still permitted the confusion.

The fix. Restructure rather than patch. Introduce an approval status distinct from settlement, split the funding chain into the four objects of §11.1, and remove the stored total entirely — replacing it with derivation over settled transactions (ADR-04, §7.6). The migration history records this as it happened: `AddInvestmentApprovalStatus` followed by `Phase2_InvestmentPipeline`.

What it cost. A schema migration mid-project, a rewrite of every funding read, and the loss of a cheap cached number in exchange for a computed one (§17.5).

The lesson. When a defect is possible, ask whether the model permits it. If it does, fixing the instance leaves the class. This is the single most valuable thing the project learned, and it is why §7.6 exists in the form it does.

18.3 One Status Column, Three Writers

Symptom. A venture that had been actively raising reverted to an early commercial stage after its founder edited the description and an administrator re-approved it.

What was initially believed. That the edit handler was resetting a field it should not touch.

What was actually wrong. Three different actors — administrator, owner, and the funding pipeline — were writing one status column, each with a legitimate reason and none aware of the others. Re-approval wrote `approved` over a value that meant something else entirely.

The fix. Three independent columns with one writer each (ADR-05, §7.6), and transitions validated per column rather than assigned freely. The five-step sequence that reproduces the original defect is in §14.2.

What it cost. Another migration, a rewrite of the visibility rule into a two-column predicate, and the composite index in §7.5 to keep the listing query fast under it.

The pattern. This is the same failure as the previous section wearing different clothes: **one field holding two meanings**. Having met it twice, the team began looking for it, which is why the payment states in §11.6 were designed as separate values from the outset rather than discovered later.

18.4 Rewriting the Frontend Without Rewriting the System

Symptom. Not a defect — a decision. Three pressures (§5.3) made the existing Angular frontend the wrong foundation for the remaining work.

The risk. A framework migration mid-project is where student projects die. The realistic failure mode was not technical: it was spending the remaining schedule on the rewrite and delivering a beautiful frontend over an unfinished backend.

Why it was survivable. The API boundary was real rather than nominal. The backend held every rule and the frontend held none (§6.3), so the migration replaced one container and touched no business logic. This was not luck — it was the payoff of a decision made months earlier for different reasons.

What actually hurt. Not the rewrite itself but the **rendering model**. Server components, the boundary between server and client code, and data fetching that no longer resembled a service injected into a component all had to be learned while delivering. Several screens were built twice: once in the client-rendered idiom the team knew, then again once the model was understood.

What it cost. Real schedule, honestly. And what it bought is in §5.3.

The lesson. An architectural boundary's value is not visible until it is tested. This one was tested under the worst circumstances and held.

18.5 A Shared Database and No Local Fallback

Symptom. Work blocked whenever the hosted database was unavailable, and a migration applied by one developer changed the schema under everyone else.

What was actually wrong. The project runs against a hosted SQL Server instance (§16.3) with no local server on developer machines. This is convenient — one schema, no divergence, real data shapes — and it makes the database a **shared mutable resource** with no isolation.

The fix — partly process, partly technical. Migrations are applied to the shared database only after applying cleanly locally against a scratch instance (§7.7), and schema changes are announced before they are applied. Additive migrations are preferred precisely because they do not break a colleague who has not yet pulled.

What it cost. Discipline, and occasional lost time when the discipline lapsed.

Stated honestly. This is not a solved problem; it is a managed one. A proper answer is per-developer databases, which the hosting arrangement does not support. It is recorded as a limitation in §19.4 rather than presented as a practice.

18.6 Theme Tokens That Inverted Under a Dark Scope

Symptom. Certain text became invisible in dark mode — foreground on foreground — on specific surfaces only.

What was initially believed. A missing dark-mode variant on those components.

What was actually wrong. A semantic colour token was being resolved inside a scope that had already inverted the theme, so a token meaning “text colour” resolved to the colour of the surface it was sitting on. The components were correct; the **scope** they were rendered in was not.

The fix. Resolve semantic tokens relative to their surface rather than to the root, so a token’s meaning is stable regardless of what it is nested inside.

The lesson. A design token is only a system if its meaning is context-independent. A token that resolves differently depending on where it is used is a variable, not a token — and §8.2 is written with that distinction in mind.

18.7 Testing Against Motion and Smooth Scrolling

Symptom. Interface tests failed intermittently — passing locally, failing in continuous runs, passing again on retry.

What was actually wrong. Smooth-scrolling and scroll-driven animation introduce time between an action and the state it produces. A test asserting on post-scroll state was racing the animation, and the race resolved differently depending on machine speed.

The fix. Motion is disabled in the test environment — the same mechanism that serves the reduced-motion preference (§8.8), which means the test environment exercises a configuration real users also use rather than a synthetic one.

The lesson. Flaky tests are usually not flaky. They are tests asserting on a state that is genuinely not yet true, and the correct response is to remove the nondeterminism rather than to add a wait.

18.8 Process Challenges

Not every problem was technical.

Scope pressure The feature list grew faster than the schedule. The response was explicit prioritisation (§4.4) and an out-of-scope section (§1.5.2) that is **written down** — a boundary nobody records is a boundary nobody keeps.

Uneven parallelism Frontend work depended on API shape, so a delayed endpoint stalled two people. Agreeing DTO contracts before implementation (§9.4) let both sides proceed against a shared definition.

Documentation debt This document was begun late. Decisions recorded here as Architecture Decision Records were reconstructed from commit history and memory rather than written when made. They are accurate, but they cost more to write than they would have. Writing an ADR takes ten minutes at the moment of the decision and an afternoon six months later.

18.9 What Generalises

Three lessons recur across the technical sections above, and they are the ones worth carrying forward.

1. **If the model permits a defect, fixing the instance leaves the class.** Two of the most expensive problems in this project were model defects wearing the costume of handler bugs.
2. **One field holding two meanings will eventually be written by two actors.** The second time this appeared, it was recognised in minutes rather than days.
3. **Correct at rest is not correct on the wire.** Storage, serialisation and display are three separate correctness surfaces, and a value can pass two of them and fail the third.

Evaluation and Results

This chapter assesses the project against the objectives it set for itself in §1.4. It does not restate the test results of Section 15 or the measurements of Section 17; it asks whether what was built is what was intended, and where it is not.

19.1 Objectives Assessment

Each objective from §1.4 is judged against evidence, not against effort. The verdict column carries three values: **met** where the objective holds and is demonstrable, **partly met** where it holds with a stated qualification, and **not met** where it does not.

	Objective	Verdict	Evidence
01	Submission passes explicit administrative review before publication.	Met	Moderation status is a distinct column with a single writer; public visibility requires approval (§14.2, §14.5). Rejection carries a reason.
02	Administrative status and commercial stage independent.	Met	Three columns, three writers, transitions validated per column (§7.6). The defect this prevents is reproduced step by step in §14.2.
03	Funding totals derived from settled payments, enforced below the application layer.	Met	No aggregate column exists. Totals derive through one service; duplicate settlement is prevented by a uniqueness constraint, not by application logic (§7.6, §11.5).
04	Payment provider isolated so domain rules are provider-independent.	Met	<code>IPaymentProvider</code> exposes no domain concept (§11.2). Two implementations satisfy it and neither contains a rule.
05	Role-based authorisation across every state-changing endpoint, with a full account lifecycle.	Partly met	Authorisation is enforced server-side throughout (§10.8, §10.9) and the lifecycle is complete. The qualification: a suspended account retains read access until its access token expires (§10.10).
06	Discovery through search, filtering, saved searches and a watchlist.	Met	All four exist (§13.3, §13.4). Ranking is deterministic and explicable, which was a requirement rather than a fallback.
07	Sustain the relationship after funding.	Met	Milestones, updates, document requests and messaging (§14.3, §14.4, Section 12). Qualified by §14.3: the platform records progress, it does not verify it.

Table 58 Objectives assessment. Six met, one partly met.

Six of seven objectives are met and one is partly met, with the qualification stated rather than absorbed. The qualification on O5 is a design consequence of stateless tokens (§5.6) and was known when the decision was taken; it is not a discovered failure.

19.2 Research Questions Answered

RQ1 — discovery independent of the founder’s network Answered affirmatively, with a caveat. Ventures are discoverable by attribute through search, filters and saved searches, and ranking uses no social signal that would reintroduce network advantage. The caveat is that this is a property of the **mechanism**; whether it produces a fairer outcome in practice is an empirical question this project cannot answer without operating at scale.

RQ2 — a model separating administrative from commercial state Answered. Three columns with disjoint writers (§7.6). The answer is transferable beyond this domain — the general form is that a field written by two actors for two reasons is two fields.

RQ3 — structural rather than conventional funding integrity Answered. Derivation removes the possibility of drift; a uniqueness constraint removes the possibility of double settlement. Neither depends on a developer remembering a rule.

RQ4 — provider substitutability Answered. The interface carries transport only, and the simulated and Stripe-compatible implementations exercise identical domain rules (§11.2). The claim is **tested** for the simulated path and **structural** for the Stripe path — see §19.3.

19.3 Requirements Coverage

Coverage is reported in three categories, because “implemented” and “verified” are different claims. The counts come from the matrix in Section 3.

Category	Count	Meaning
Implemented and verified by an executed test	4	FR-14, FR-28, FR-30 and NFR-07 — the funding and stage rules covered by the 41 passing domain tests (§15.2).
Implemented, verified by inspection only	54	The remainder. Each has a design section and an implementation; none has an executed test, because the cases that would cover them need a test database or a driven browser.
Not implemented	0	Deferred items were removed from Section 4 rather than left unmet; they appear in §20.3.

Table 59 Requirements coverage. Every requirement is built; none is verified by an automated test.

The middle row is the result. Fifty-four of fifty-eight requirements rest on inspection, on the structural guarantees of Section 7, and on manual exercise of the journeys in §15.5.

The four that do not are the ones that matter most — the separation of stage from status, the derivation of funding totals, the self-investment rule and exact decimal money — and they are covered because §6.4 put those rules in components that hold no I/O and can therefore be tested in 34 milliseconds. The architecture is what made those four cheap to verify; it is also why the other fifty-four are not, since they involve a database or a browser.

Stated as a number rather than as prose, because a number is harder to read past. The twenty-nine remaining cases in Section 4 are the work that closes it, and §20.3 places it accordingly.

19.4 User Evaluation

NOTE

No user study was conducted. The method, tasks and instrument below are specified and ready to run; they were not run, because doing so requires recruited participants and a scheduled session that the project timetable did not accommodate. This section therefore reports a **design**, not a result, and §19.4 records the absence as a limitation. It is documented at this length because a specified-but-unrun study is recoverable in an afternoon, whereas an unspecified one is not.

Method. Task-based usability evaluation with participants drawn from both personas of §4.2. Each participant completes a fixed set of tasks unaided while the facilitator records completion, time and points of hesitation, followed by a System Usability Scale questionnaire.

Tasks. Register and verify an account. Find a venture matching stated criteria using search and filters. Save it and follow its founder. Commit an amount and take it through checkout. As a founder, submit a venture and publish an update.

What is being measured. Not preference. Task completion, and specifically whether a participant can state — correctly — what stage their commitment is in after the checkout task. That single question tests whether the distinction the whole system is built on (§14.1) is legible to the person it exists to protect.

19.5 Limitations

Stated plainly and in one place, so that a reader does not have to assemble them from the qualifications scattered through earlier chapters.

No live payment operation The Stripe-compatible path is implemented, signature-verified and idempotent, but has never run against a live merchant account. Real card authorisation, disputes and settlement timing are unexercised (§11.9).

No regulatory onboarding Identity verification and anti-money-laundering checks are absent (§2.7). The platform is not deployable to real transactions without them.

Suspension is not immediate for reads Up to sixty minutes of read access survives suspension, by the nature of stateless tokens (§10.10).

Accessibility conformance is not exhaustively verified Verified on primary flows; screen-reader testing is spot-checked rather than complete across sixty-five routes (§8.7).

No continuous integration and no automated alerting Builds and deployments are manual against a checklist; failures are found by inspection or report (§16.4, §16.9).

No observability pipeline Performance figures come from deliberate measurement runs, not continuous collection (§16.8).

Uploaded files are not backed up Images, documents and attachments live outside the database and outside the provider's backup arrangement (§7.8).

Real-time and notification layers are single-instance Presence and the notification queue are process-local, so the system cannot scale out without the changes in §17.7 (§12.7).

Shared development database No per-developer isolation; managed by process rather than solved (§18.5).

No dependency scanning Dependencies are updated deliberately but nothing automatically checks them for known vulnerabilities (§10.2).

Automated testing covers the domain layer only Forty-one tests pass against the pure rule components (§15.2). The integration, authorisation, security and end-to-end cases in Section 4 are specified and unexecuted, so fifty-four of fifty-eight requirements rest on inspection (§19.2).

No user evaluation was conducted The study in §19.3 is specified and unrun. No claim in this document rests on observed user behaviour.

Load behaviour above five concurrent readers is unknown The harness used in §17.6 is a browser and cannot generate reliable concurrency beyond a per-origin connection limit. The point at which the server degrades has not been established.

NFR-02 is not met The venture listing measures 519 ms at p95 against a 300 ms target (§17.3). The cause is identified — a remote database round trip — and is a consequence of the hosting decision in §5.11.

19.6 Threats to Validity

Three, stated because an evaluation chapter that does not examine its own weaknesses is advocacy.

The evaluators are the authors The team built the system and designed its evaluation. Task selection can unconsciously favour paths that work. The mitigation is that tasks were derived from the requirements in Section 4 before the evaluation was designed, not chosen afterwards.

The simulated provider is our own §11.2 argues that the simulated path exercises the same domain rules as the Stripe path, and that argument is sound — but it is an argument about structure, not an observation of the Stripe path running. The distinction is maintained in §19.2 and should be maintained by any reader assessing O4.

Absence of scale Every performance and fairness property is assessed at a size at which few systems fail. Nothing here supports a claim about behaviour under real load or real market dynamics.

Conclusion and Future Work

20.1 Summary of Contributions

This project set out to connect early-stage founders with individual investors, and to remain correct about money and about state while doing it. §19.1 assesses the seven objectives; this section states what the work contributes beyond having met them.

A state model that separates moderation from commerce. A venture carries administrative status, operational lifecycle and commercial stage as independent fields with disjoint writers. The general form of the finding is not specific to crowdfunding: **a field written by two actors for two reasons is two fields**, and collapsing them destroys information at the moment a second actor acts. §14.2 reproduces the five-step sequence in which a single column silently erases funding progress.

A derived-funding integrity model. No column stores what a venture has raised. Totals are computed from settled transactions through one service, and duplicate settlement is prevented by a database constraint rather than by an application check. The consequence is that an approved commitment is structurally distinct from a funded one — which is the property that stops a platform reporting progress no money supports.

A provider-independent payment layer. Domain rules sit above the provider interface, which carries transport only. This is what makes the simulated provider a legitimate test surface rather than a stand-in, and it is why the funding logic could be verified without a merchant account.

A working system. Twenty-two API controllers over a relational model of more than forty entities, a React front end of sixty-five routes, and a versioned migration history that records how the schema arrived at its present shape.

A document that says what it does not do. The previous version of this document (§1.2) claimed scalability, security and performance without evidence. This one states its limitations in §19.4, marks unmeasured figures rather than estimating them, and includes two rows in its own competitor comparison where the platform loses. That is a contribution to the work's credibility, and it was a deliberate change of standard.

20.2 Lessons Learned

20.2.1 Technical

If the model permits a defect, fixing the instance leaves the class. The two most expensive problems in the project — approval counting as funding, and re-approval resetting commercial stage — presented as handler bugs and were model defects. Patching either would have moved the symptom.

Correct at rest is not correct on the wire. Storage, serialisation and display are three separate correctness surfaces. The timestamp defect in §18.1 passed the first and failed the second, and was invisible in every environment where the two agreed.

An architectural boundary's value is unknown until it is tested. The API boundary was built for ordinary reasons and paid for itself when the entire frontend was replaced without a business rule moving (§18.4).

Constraints outperform conventions. Four business rules in this system are unique indexes (§7.5). Each could have been an application check, and each would then have had a race.

20.2.2 Process

Written scope is kept scope. The out-of-scope list in §1.5.2 was written down, and it held. Boundaries that live only in conversation do not.

Record decisions when they are made. The Architecture Decision Records in this document were reconstructed from commit history months later (§18.8). They are accurate and they cost several times what they would have cost at the time.

Sequence security before surface. Authentication hardening came after the investment pipeline (§3.2). Retrofitting authorisation across an existing endpoint surface cost more than building it in would have.

A risk register is only useful if it contains likely risks. Five of six identified risks materialised (§3.7). That is what an honest register looks like.

20.2.3 Personal

The team ends the project able to speak to two frontend frameworks from experience rather than from documentation (§5.3); to explain why a payment webhook must be signature-verified over the raw body, in constant time, before any state change (§11.4); and to recognise a two-meanings-one-field defect on sight, having paid for it twice.

20.3 Future Work

Ordered by what should be done first, and why. The first three are corrections of stated limitations; the remainder are extensions.

	Work	Addresses	Why this position
1	Operate the Stripe path against a live merchant account: real authorisation, disputes, settlement timing.	§19.4, §11.9	The single largest gap between this system and a deployable one. Every other extension is less valuable until money can actually move.
2	Identity verification and anti-money-laundering onboarding.	§2.7	Legally prerequisite to item 1 in any real deployment. Sequenced second only because item 1 is what proves the rest of the system.
3	Continuous integration, automated alerting and an observability pipeline.	§16.4, §16.8, §16.9	The operational floor. Items 4 onward add surface that cannot be safely operated without this.
4	Backplane for presence and a durable notification queue.	§12.7, §17.7	The first change that unlocks horizontal scaling. Must precede any second instance.
5	Object storage for uploads, behind a content delivery network.	§7.8	Closes the backup gap and removes file serving from the application.
6	Notification preferences and email digests.	§13.7	The most-requested absent feature; currently every notification is delivered in-app to every eligible recipient.
7	Non-image message attachments.	§12.5	A stated scope limitation with a clear path — the upload security controls of §10.12 already generalise.
8	Secondary transfer of a commitment between investors.	§15.2	A substantial domain extension. Requires items 1 and 2 first.
9	Assisted discovery: recommendation over the interaction history the platform already records.	§13.3	Deliberately last. Ranking is currently deterministic and explicable (§13.3), which was a requirement rather than a limitation. Any learned ranking must remain explicable to a founder who asks why their venture appears where it does — and there is not yet enough interaction history to learn from.

Table 60 Future work, ordered by dependency and value.

Item 9 is placed last deliberately, and the reason is a design position rather than a schedule constraint. This platform's premise is that discovery should depend on the venture rather than on privilege. An opaque ranking model reintroduces exactly the unexplainable advantage the project set out to remove, and adding one before it can be made explicable would undo the contribution in §20.1.

20.4 Closing

The problem this project addressed is not that capital is scarce or that ideas are scarce. It is that the mechanism connecting them is networked, opaque and ends at the transaction.

Vestora is one answer to that: a platform where a venture can be found on its attributes, where an investor can see evidence rather than claims, where the difference between an intention and a payment is never blurred, and where the relationship continues after the money moves.

What is built works, within the limits §19.4 sets out. What is not built is listed rather than implied. Both statements are part of the same claim to have done this properly.

Entries are held in `src/backmatter/refs.bib` and rendered by the typesetter, so a citation in the text and its entry here cannot disagree. Download pages are not references; installation sources are in Section 7.

References

- [1] A. Agrawal, C. Catalini, and A. Goldfarb, “Some simple economics of crowdfunding,” *Innovation Policy and the Economy*, vol. 14, no. 1, pp. 63–97, 2014.
- [2] S. Vismara, “Equity retention and social network theory in equity crowdfunding,” *Small Business Economics*, vol. 46, no. 4, pp. 579–590, 2016.
- [3] E. Mollick, “The dynamics of crowdfunding: An exploratory study,” *Journal of Business Venturing*, vol. 29, no. 1, pp. 1–16, 2014.
- [4] P. Belleflamme, T. Lambert, and A. Schwienbacher, “Crowdfunding: Tapping the right crowd,” *Journal of Business Venturing*, vol. 29, no. 5, pp. 585–609, 2014.
- [5] G. K. C. Ahlers, D. Cumming, C. Günther, and D. Schweizer, “Signaling in equity crowdfunding,” *Entrepreneurship Theory and Practice*, vol. 39, no. 4, pp. 955–980, 2015.
- [6] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.
- [7] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [10] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [11] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
- [12] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [13] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2003.
- [14] P. P.-S. Chen, “The entity-relationship model — Toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.
- [15] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” technical report RFC 7519, May 2015.
- [16] D. Hardt, “The OAuth 2.0 Authorization Framework,” technical report RFC 6749, Oct. 2012.
- [17] OWASP Foundation, “OWASP Top 10: The ten most critical web application security risks.” 2021.
- [18] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.
- [19] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the USENIX Annual Technical Conference*, 1999.
- [20] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Lifecycle Management,” technical report NIST Special Publication 800–63B, 2017.

- [21] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1.” 2018.
- [22] J. Nielsen, “10 usability heuristics for user interface design.” 1994.
- [23] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, London, U.K.: Taylor, Francis, 1996.
- [24] S. Krug, *Don't Make Me Think, Revisited*, 3rd ed. Berkeley, CA, USA: New Riders, 2014.
- [25] K. Schwaber and J. Sutherland, “The Scrum Guide.” 2020.
- [26] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.
- [27] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Boston, MA, USA: Addison-Wesley, 2007.
- [28] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA, USA: Addison-Wesley, 2009.
- [29] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR, USA: IT Revolution Press, 2018.
- [30] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” technical report IEEE Std 830–1998, 1998.
- [31] Microsoft, “ASP.NET Core documentation.” 2024.
- [32] Microsoft, “ASP.NET Core SignalR documentation.” 2024.
- [33] Microsoft, “Entity Framework Core documentation.” 2024.
- [34] Vercel, “Next.js documentation.” 2025.
- [35] Stripe, “Webhook signature verification.” 2024.
- [36] World Wide Web Consortium, “Accessible Rich Internet Applications (WAI-ARIA) 1.2.” 2023.

PART 6

Appendices

Reference material held out of the chapters so the argument stays readable.

Complete API Reference

The API exposes **150 endpoints across 22 controllers**. This appendix maps the surface by area; the authoritative, always-current definition is the generated OpenAPI document (§9.8), from which this listing is derived.

Every endpoint is authenticated unless marked **public**. Every collection endpoint is paged (§9.7). The uniform error shape is described in §9.6.

1.1 Surface Map

Controller	Base route	Ops	Area of responsibility
Auth	api/auth	10	Registration, verification, OTP, sign-in, refresh, logout, password reset (§10.3–§10.6).
Users	api/users	7	Profile read and update, public profile, account settings (§13.2).
Investor	api/investor	10	Investor profile, preferences, directory (§13.5).
Projects	api/projects	22	Venture create, read, update, listing, search, filtering, lifecycle (§14.2).
ProjectStory	api/projects	20	Images, documents, team members, milestones, updates — the material a venture owns (§14.3).
ProjectEngagement	api/projects	4	Comments, replies, reviews (§13.6).
Bookmark	api/bookmarks	4	Watchlist add, remove, list, membership check (§13.4).
Follow	api/follows	5	Follow, unfollow, followers, following, status (§13.5).
Feed	api/feed	1	Personalised activity feed (§13.3).
Signals	api/signals	5	Engagement signals feeding deterministic ranking (§13.3).
Report	api/reports	1	Submit a report on a venture (§13.6).
Capital	api/capital	3	Commitment creation and approval (§14.1).
Payments	api/payments	13	Checkout, session result, refund, expiry, and the provider webhook (Section 11).
DealRoom	api/deals	9	Document requests, grants and the diligence surface (§14.4).
Messages	api/messages	9	Conversations, history, attachments, read state (Section 12).
Notification	api/ notification	4	List, unread count, mark read (§13.7).
FounderDashboard	api/dashboard	1	Founder aggregate (§14.6).
InvestorDashboard	api/dashboard	1	Investor aggregate (§14.6).
VentureInsights	api/insights	1	Venture analytics (§14.7).
Admin	api/admin	11	User administration, suspension, platform activity (§14.5).
AdminModeration	api/admin	6	Review queue, approve, reject with reason, report queue (§14.5).
AdminRevenue	api/admin/ revenue	3	Fee revenue derived from settled transactions (§14.7, §2.6).

Table 61 The API surface by controller. Counts are HTTP action attributes in each controller.

1.2 Representative Endpoints

The full listing is the OpenAPI document. The endpoints below are the ones the main text refers to, reproduced here so a reader following a cross-reference does not have to open the specification.

Method	Path	Notes
POST	api/auth/register	Public. Responds identically whether or not the address is registered (§10.3).
POST	api/auth/verify-email	Public. Single-use, 60-minute token.
POST	api/auth/login	Public. Subject to logout (§10.7).
POST	api/auth/refresh	Public. Rotates the refresh token; replay invalidates the chain (§10.6).
POST	api/auth/forgot-password	Public. 10-minute token.
GET	api/projects	Public. Paged. Filters on moderation and lifecycle status (§13.3).
GET	api/projects/{id}	Public when approved and active.
POST	api/projects	Founder only.
POST	api/projects/{id}/submit	Moves moderation status to pending; leaves stage untouched (§14.2).
POST	api/admin/projects/{id}/approve	Administrator only. Writes the audit log (§10.13).
POST	api/admin/projects/{id}/reject	Requires a reason (§14.5).
POST	api/capital/commit	Creates a commitment. Refused if the caller owns the venture (FR-30).
POST	api/capital/{id}/approve	Approval only — not funding (§14.1).
POST	api/payments/checkout	Opens a funding request with an expiry and a provider session (§11.3).
POST	api/payments/webhook/stripe	Public by necessity. Signature-verified over the raw body before any state change; duplicate events acknowledged and ignored (§11.4, §11.5).

Table 62 Endpoints referenced from the main text.

NOTE

The webhook row is the only unauthenticated state-changing endpoint in the system. Its defence is signature verification, not authentication — which is why §11.4 treats it as the highest-risk surface in the platform.

Database Schema and Data Dictionary

The schema holds more than forty tables. Section 7 documents the ones whose columns encode rules; this appendix lists the whole set by group, with the relationship and delete behaviour of each.

The authoritative definition is the Entity Framework configuration in `AppDbContext` together with the migration history (§7.7). Nothing here is maintained by hand independently of that source.

2.1 Identity and Access

Table	Purpose and key columns	Rules
<code>Users</code>	Accounts. <code>Email</code> , <code>PasswordHash</code> , <code>UserType</code> discriminator, <code>EmailVerified</code> , <code>FailedLoginAttempts</code> , <code>LockoutEndUtc</code> , <code>LastSeenUtc</code> .	<code>Email</code> unique.
<code>Investors</code>	Investor specialisation (table-per-hierarchy).	Discriminator <code>Investor</code> .
<code>Innovators</code>	Founder specialisation (table-per-hierarchy).	Discriminator <code>Innovator</code> .
<code>RefreshTokens</code>	Hashed refresh tokens with expiry and rotation state.	<code>TokenHash</code> unique; cascade from user.
<code>SecurityLogs</code>	Authentication and security events.	Indexed <code>(UserId, CreatedAtUtc)</code> ; user reference set null on delete.
<code>AdminAuditLogs</code>	Administrative actions against the acting administrator.	Retained independently of the target row.

Table 63 Identity and access tables.

2.2 Ventures

Table	Purpose	Delete behaviour
Projects	Ventures. Three independent state columns (§7.6).	Cascade from owner.
ProjectImages	Venture imagery.	Cascade
ProjectDocuments	Attached documents with a visibility rule.	Cascade
TeamMembers	People listed on a venture.	Cascade
Milestones	Declared objectives and dates.	Cascade
ProjectUpdates	Published progress reports.	Cascade
ProjectUpdateImages	Imagery on an update.	Cascade from update
ProjectViews	Individual view events feeding analytics.	Cascade; indexed (ProjectId , CreatedAt)

Table 64 Venture tables. All are owned by the venture and cascade with it.

2.3 Funding

Table	Purpose	Rules
Investments	Commitments with an approval status.	Restrict on investor delete — history must survive.
FundingRequests	Time-boxed settlement attempts.	Expiry swept (§11.8).
PaymentTransactions	Provider outcomes: amount, fee, provider name, provider reference, status.	Only Settled counts (§11.6).
PaymentEvents	One row per provider callback.	ProviderEventId unique — the idempotency key (§11.5).

Table 65 Funding tables. No aggregate column exists anywhere in this group.

2.4 Engagement and Social

Table	Purpose	Rules
<code>Comments</code>	Comments on a venture.	Restrict
<code>Replies</code>	One level of nesting under a comment.	Restrict
<code>Reviews</code>	Rated reviews.	<code>(ProjectId, InvestorId)</code> unique — one per investor.
<code>Bookmarks</code>	Watchlist edges.	<code>(UserId, ProjectId)</code> unique.
<code>Follows</code>	Directed follow edges.	<code>(FollowerId, FollowedId)</code> unique; <code>FollowedId</code> indexed.
<code>Reports</code>	User reports on ventures.	Indexed <code>(ProjectId, Status)</code>
<code>UserProjectInteractions</code>	Interaction signals feeding ranking.	Restrict.

Table 66 Engagement tables. Four business rules in this group are unique indexes rather than application checks (§7.5).

2.5 Communication

Table	Purpose	Rules
<code>Messages</code>	Direct messages with sender, receiver and read state.	Restrict on both participants.
<code>MessageAttachments</code>	Image attachments, one per message.	Cascade from message.
<code>Notifications</code>	Domain-event notifications referencing the object they concern rather than storing rendered text (§13.7).	Restrict on referenced rows.

Table 67 Communication tables.

2.6 Conventions

Money `decimal` with explicit precision and scale. Never floating point (§7.3).

Time `datetime2`, stored in UTC, serialised with an explicit offset (§6.7).

Keys Integer surrogate primary keys throughout.

Deletion Cascade where the child is owned by the parent; restrict where the child is a record of something that happened (§7.6).

Migrations Every change is a migration; none is applied by hand (§7.7).

Requirements Traceability Matrix

Each requirement from Section 4 is traced forward to the design section that specifies it, the component that implements it, and the test that verifies it. A row with an empty test column is implemented but unverified — and the purpose of the matrix is to make that visible rather than to let it be assumed at zero.

DRAFT Test identifiers pending. The design and implementation columns are complete. Test identifiers resolve against Section 4 and are filled once the catalogue is instrumented; they are marked *[T]* here. §19.2 reports the resulting coverage counts.

3.1 Functional Requirements

ID	Design	Implementation	Test
FR-01	§10.3	<code>AuthController</code> · <code>AuthService</code>	<i>[T]</i>
FR-02	§10.3	<code>AuthService</code> verification gate	<i>[T]</i>
FR-03	§10.7	<code>AuthService</code> lockout counters on <code>Users</code>	<i>[T]</i>
FR-04	§10.4	<code>AuthController</code> reset flow · <code>IService</code>	<i>[T]</i>
FR-05	§10.5–§10.6	<code>AuthService</code> · <code>RefreshTokens</code>	<i>[T]</i>
FR-06	§13.2	<code>UsersController</code>	<i>[T]</i>
FR-07	§10.10	<code>AdminController</code> · <code>AccountRules</code>	<i>[T]</i>
FR-08	§14.2	<code>ProjectsController</code>	<i>[T]</i>
FR-09	§14.2, §14.5	<code>Projects.ModerationStatus</code> gate	<i>[T]</i>
FR-10	§14.5	<code>AdminModerationController</code>	<i>[T]</i>
FR-11	§14.4	<code>ProjectStoryController</code> · <code>DealRoomController</code>	<i>[T]</i>
FR-12	§14.3	<code>ProjectStoryController</code>	<i>[T]</i>
FR-13	§14.2	<code>ProjectStoryController</code>	<i>[T]</i>
FR-14	§7.6, §14.2	Three columns on <code>Projects</code> · <code>PipelineStages</code>	PS-01
FR-15	§13.3	<code>ProjectsController</code> listing	<i>[T]</i>
FR-16	§13.3	<code>ProjectsController</code> search and filter	<i>[T]</i>
FR-17	§13.3	Saved searches	<i>[T]</i>
FR-18	§13.4	<code>BookmarkController</code> · unique index	<i>[T]</i>
FR-19	§13.5	<code>FollowController</code> · unique index	<i>[T]</i>
FR-20	§13.6	<code>ProjectEngagementController</code>	<i>[T]</i>
FR-21	§13.6	<code>Reviews</code> unique index	<i>[T]</i>
FR-22	§13.6	<code>ReportController</code>	<i>[T]</i>
FR-23	§14.1	<code>CapitalController</code>	<i>[T]</i>
FR-24	§14.1	<code>CapitalController</code> approval	<i>[T]</i>

ID	Design	Implementation	Test
FR-25	§11.3	<code>PaymentsController</code> · <code>PaymentService</code>	[T]
FR-26	§11.4	Webhook signature verification	[T]
FR-27	§11.5	<code>PaymentEvents</code> unique index	[T]
FR-28	§7.6, §11.7	<code>FundingMath</code>	FM-01, FM-02
FR-29	§11.8	<code>PaymentExpirySweeper</code>	[T]
FR-30	§10.9	<code>AccountRules</code>	AR-01
FR-31	§14.6	<code>InvestorDashboardController</code>	[T]
FR-32	§12.4	<code>MessagesController</code> · hub	[T]
FR-33	§12.2–§12.3	<code>PresenceTracker</code> · hub	[T]
FR-34	§12.3	Server-side read marking	[T]
FR-35	§12.5	<code>MessageAttachments</code> · upload controls	[T]
FR-36	§13.7	<code>NotificationFanOutQueue</code> and worker	[T]
FR-37	§14.4	<code>DealRoomController</code>	[T]
FR-38	§14.5	<code>AdminModerationController</code>	[T]
FR-39	§14.5	<code>AdminModerationController</code> · <code>Reports</code> index	[T]
FR-40	§10.13, §14.5	<code>AdminAuditLogs</code>	[T]
FR-41	§14.5, §14.7	<code>AdminController</code> · <code>AdminRevenueController</code>	[T]
FR-42	§14.7	<code>VentureInsightsController</code>	[T]

Table 68 Functional requirements traced to design, implementation and test.

3.2 Non-Functional Requirements

ID	Design	Implementation	Assessed
NFR-01	§5.2	Server-rendered public routes	§17.2
NFR-02	§13.3	Composite index · projection	§17.3
NFR-03	§9.7	Paged results, server-capped page size	§9.7
NFR-04	§10.2	BCrypt hashing	§15.6
NFR-05	§10.6	Hashed, rotated refresh tokens	§15.3
NFR-06	§6.7	UTC serialisation converter	§18.1
NFR-07	§7.3	<code>decimal</code> money columns	§15.2
NFR-08	§10.8	Endpoint authorisation	§15.3
NFR-09	§6.7	Central exception handling	§15.6
NFR-10	§16.6	Startup configuration validation	§16.11
NFR-11	§10.12	<code>FileUploadSecurityService</code>	§15.6
NFR-12	§10.13	<code>SecurityLog</code> · <code>AdminAuditLog</code>	§15.3
NFR-13	§8.7	Radix primitives · token contrast	§8.7
NFR-14	§8.6	Responsive layout	§8.6
NFR-15	§7.7	EF Core migrations	§7.7
NFR-16	§12.6	Background fan-out worker	§15.2

Table 69 Non-functional requirements traced to design, implementation and assessment.

3.3 Reading the Matrix

Three questions the matrix answers that prose cannot:

1. **Is every requirement implemented?** A row with an empty implementation column is a requirement that was stated and not built. There are none; deferred requirements were removed from Section 4 and appear in §20.3 instead.
2. **Is every requirement verified?** Rows marked *[T]* are pending instrumentation. §19.2 reports the final counts.
3. **Does every test serve a requirement?** Tests in Section 4 with no requirement here are either testing an unstated requirement or testing nothing that matters — both worth knowing.

Test Case Catalogue

The domain cases below were executed with `dotnet test`; their status is the result of that run. Cases in the integration, authorisation, security and end-to-end groups remain specified and unexecuted — they require a test database and a driven browser (§15).

The catalogue is organised by the layers of §15.1. Cases derived from an actual defect carry a reference to the section that records it — those are the cases that exist because something went wrong, and they are the most valuable in the suite.

4.1 Domain Unit Cases

ID	Given / when	Then	Status
FM-01	An approved commitment with no settled transaction.	It contributes zero to the venture total. (§18.2)	Passed
FM-02	A settled transaction that is later refunded.	It is removed from the total.	Passed
FM-03	The same data aggregated twice.	Both results are identical.	Passed
FM-04	A venture with no transactions.	The total is zero, not null.	Passed
PS-01	A venture at a mid funding stage; moderation status changes.	Commercial stage is unchanged. (§18.3)	Passed
PS-02	An invalid stage transition is requested.	It is refused; state is unchanged.	Passed
PS-03	A venture is rejected, then a funding event occurs.	Moderation status stays rejected.	Passed
AR-01	A founder attempts to commit to their own venture.	Refused. (FR-30)	Passed
AR-02	A suspended account attempts a state change.	Refused.	Passed
PT-01	A transaction in each non-settled state.	None is countable.	Passed
PT-02	A transaction transitions settled to re-funded.	The row persists; only status changes.	Passed

Table 70 Domain unit cases. All eleven are executed by `MyAppApi.Tests` and passing; three trace directly to defects in Section 18.

4.2 Integration Cases

ID	Given / when	Then	Status
IC-01	A second bookmark for the same user and venture.	Rejected by the database, not by application code.	Specified
IC-02	A second review by the same investor on a venture.	Rejected by the unique index.	Specified
IC-03	A duplicate follow edge.	Rejected by the unique index.	Specified
IC-04	A provider callback whose event id is already recorded.	No state changes; success is returned. (FR-27)	Specified
IC-05	Two concurrent deliveries of the same callback.	Exactly one is processed.	Specified
IC-06	Deleting a venture with images and milestones.	Children cascade.	Specified
IC-07	Deleting a user with investment history.	Refused by the restrict rule. (§7.6)	Specified
IC-08	The public listing with ventures in every state combination.	Only approved and active appear.	Specified
IC-09	A checkout left unresolved past its expiry.	The sweeper lapses it; the commitment stays approved.	Specified

Table 71 Integration cases. These verify constraints as constraints rather than as intentions.

4.3 Authorisation Cases

One case per negative cell of the matrix in §10.9. The negative cases are the ones that matter — that a founder **can** edit their venture is worth less than that another user cannot.

ID	Actor and action	Then	Status
AZ-01	A standard account calls an administrative endpoint.	403	Specified
AZ-02	A user requests another user's investment by id.	403 or 404	Specified
AZ-03	A non-participant requests a conversation.	403	Specified
AZ-04	A user requests a restricted document without a grant.	403	Specified
AZ-05	A non-owner edits a venture.	403	Specified
AZ-06	An unauthenticated request to a protected endpoint.	401, not 403	Specified
AZ-07	An administrator requests a private conversation.	Refused — moderation covers reported content only.	Specified

Table 72 Authorisation cases, one per negative cell of §10.9.

4.4 Security Cases

ID	Given / when	Then	Status
SC-01	A venture update payload includes <code>ModerationStatus</code> .	Ignored — the DTO does not expose it. (§10.11)	Specified
SC-02	An upload declares <code>image/png</code> but carries other bytes.	Rejected on byte inspection. (§10.12)	Specified
SC-03	An upload with a traversal filename.	Stored under a generated name.	Specified
SC-04	A webhook post with no signature.	Rejected; no state change.	Specified
SC-05	A webhook post with an incorrect signature.	Rejected and logged.	Specified
SC-06	Six consecutive failed sign-ins.	Account locked; the response is identical to a wrong password. (§10.7)	Specified
SC-07	Registration with an already-registered address.	Response identical to a new registration. (§10.3)	Specified
SC-08	An unhandled exception is triggered.	Response carries a correlation id and no internal detail. (§16.9)	Specified
SC-09	A rotated refresh token is presented again.	The chain is invalidated. (§10.6)	Specified

Table 73 Security cases, targeted at the threat model in §10.1.

4.5 End-to-End Journeys

ID	Journey	Verifies	Status
E2E-01	Register, verify, sign in, complete onboarding.	Identity end to end.	Specified
E2E-02	Create, submit, approve, appear publicly.	The moderation pipeline.	Specified
E2E-03	Commit, approve, check out, observe progress change.	The property the system exists for — progress moves on settlement and not before.	Specified
E2E-04	Exchange messages; observe read state update live.	The real-time layer.	Specified

Table 74 End-to-end journeys. E2E-03 is the most valuable test in the suite.

4.6 Summary

Layer	Cases	Executed
Domain unit	11	11
Integration	9	0
Authorisation	7	0
Security	9	0
End-to-end	4	0
Total	40	11

Table 75 Catalogue summary.

Interface Gallery

Captures are produced by a scripted browser session (`book/capture.js` and `book/capture-auth.js`) against the running application and a populated database — 21 ventures at the time of capture. The script sets theme and locale in storage before each navigation, so a variant is never captured by hand and the matrix cannot drift.

Capture rules. Desktop at 1440×900 and mobile at 390×844, held constant. Real seeded content — no `Lorem ipsum`, no placeholder names, no unrelated media. Every figure captioned and referenced from the section it illustrates.

Axis	Coverage
Theme	Light and dark, from the same token roles (§8.4.1).
Language	English (LTR) and Arabic (RTL), including the display-face change (§8.4.2, §8.6.1).
Viewport	Desktop and mobile, both axes combined at 390 px (§8.4.3).
State	Populated, empty and error states — the last captured with the API deliberately stopped (§8.4.4).

Table 76 Axes covered by the captured gallery.

5.1 Captured Gallery

Three accounts were created for capture — one investor, one founder, one administrator — so that each role is photographed from inside its own session rather than simulated. The public surfaces are in §8.4.

5.1.1 Investor

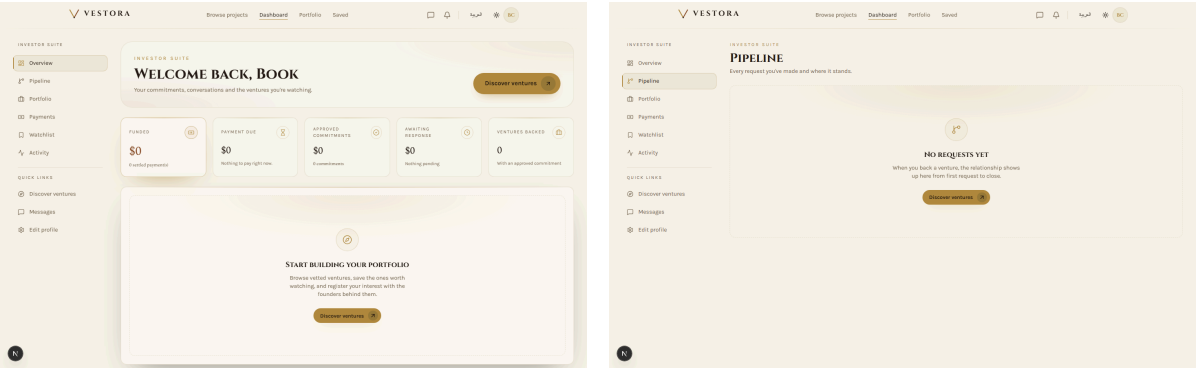


Figure 26 Investor overview: funded and approved as separate tiles (§14.1).

Pipeline: commitments by state.

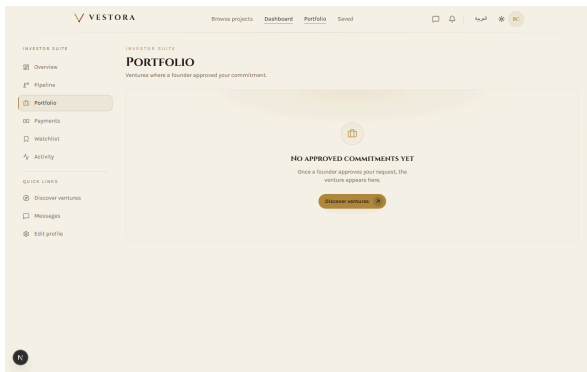
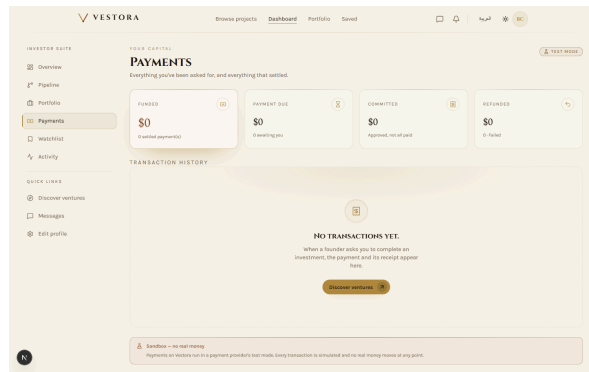


Figure 27 Portfolio: settled holdings only.



Payments: settlement history.

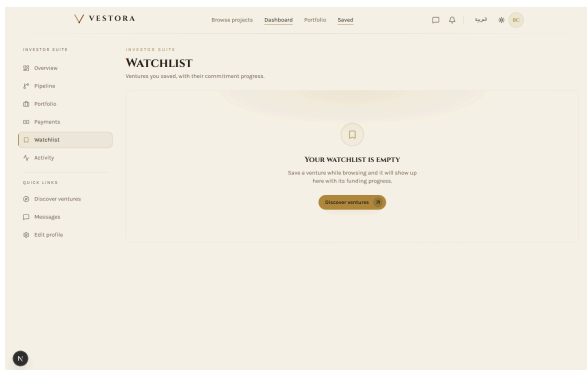
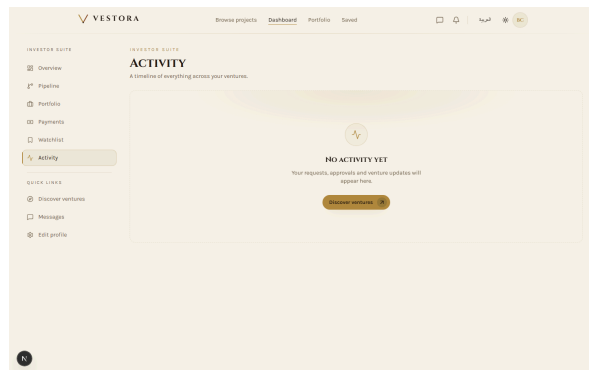


Figure 28 Watchlist (\$13.4).



Activity log.

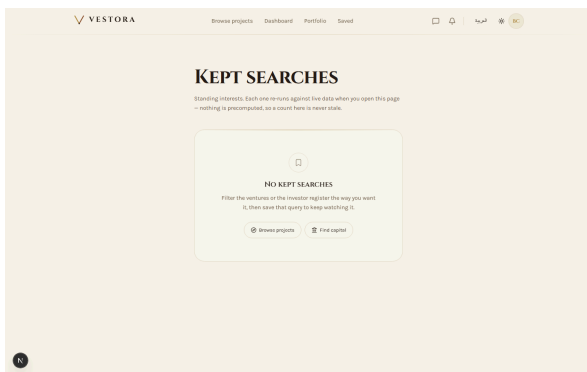
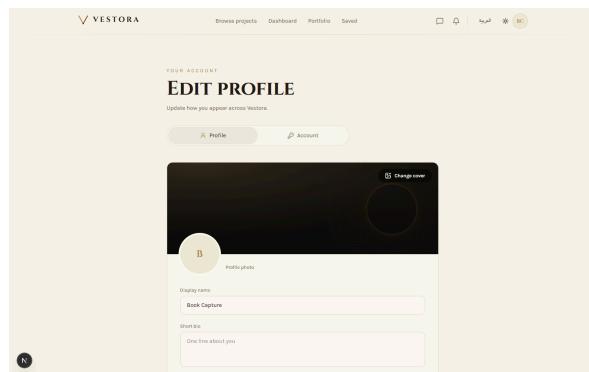


Figure 29 Saved searches (\$13.3).



Profile settings, separate from account (\$13.2).

5.1.2 Founder

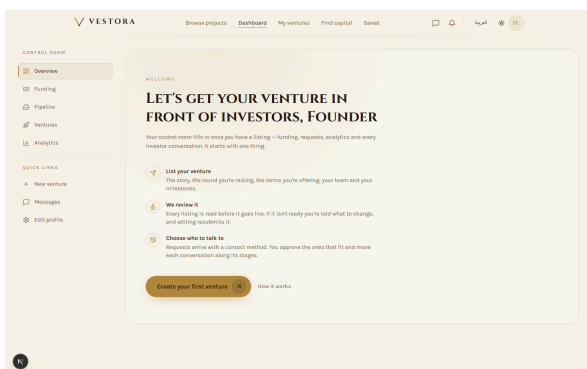
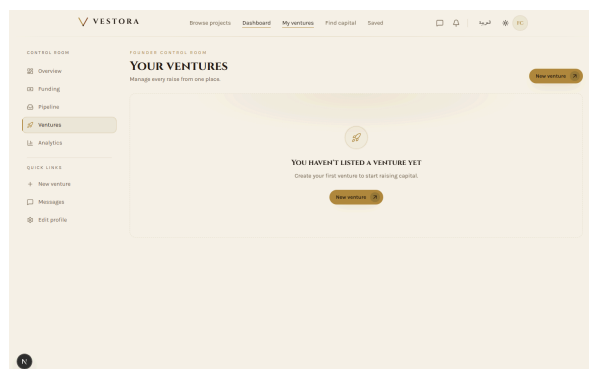


Figure 30 Founder overview (\$14.6).



Venture list with moderation state.

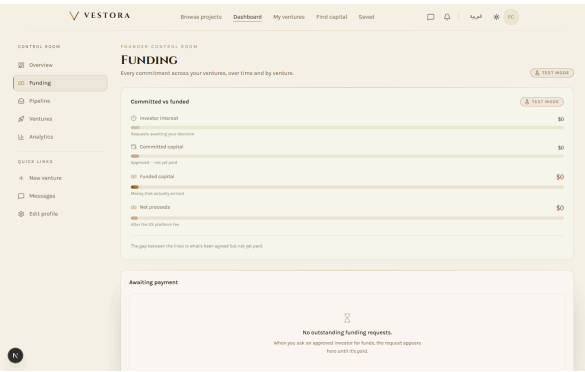
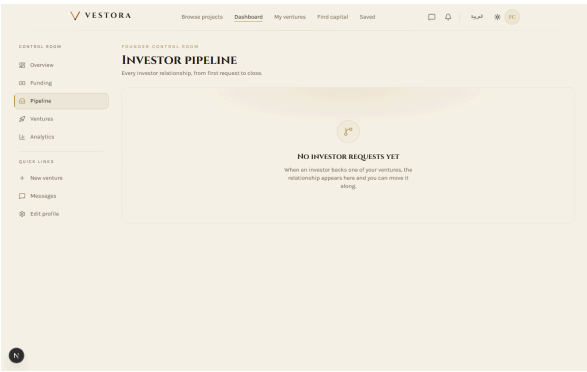


Figure 31 Funding: committed against settled.



Incoming commitments awaiting approval (\$14.1).

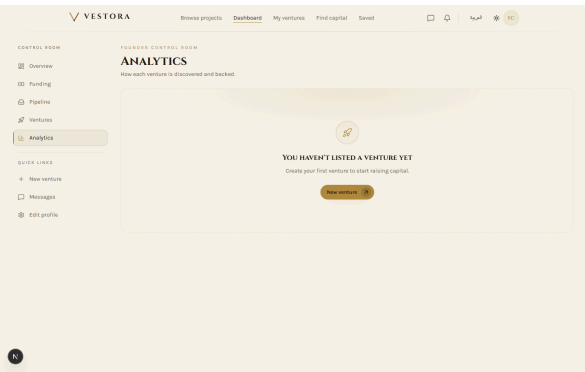
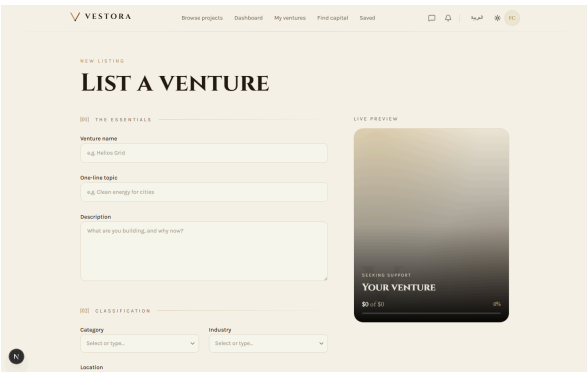


Figure 32 Venture analytics (\$14.7).



Venture submission form (\$14.2).

5.1.3 Administrator

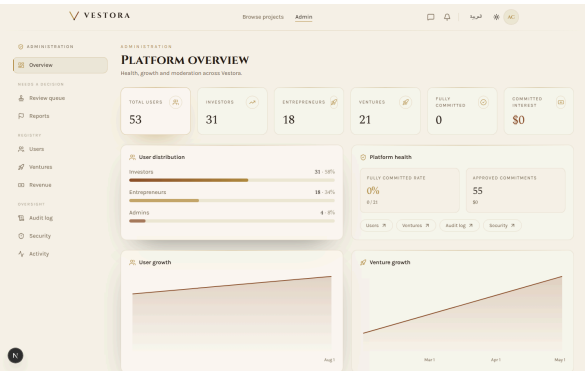
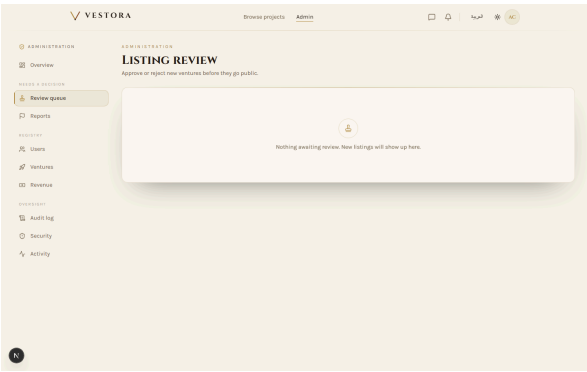


Figure 33 Platform overview with derived analytics (\$14.7).



Review queue — the primary moderation surface (\$14.5).

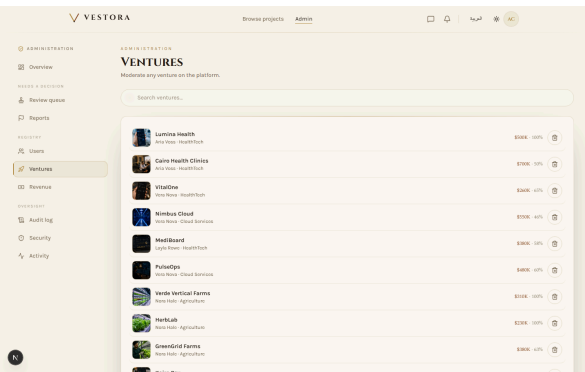
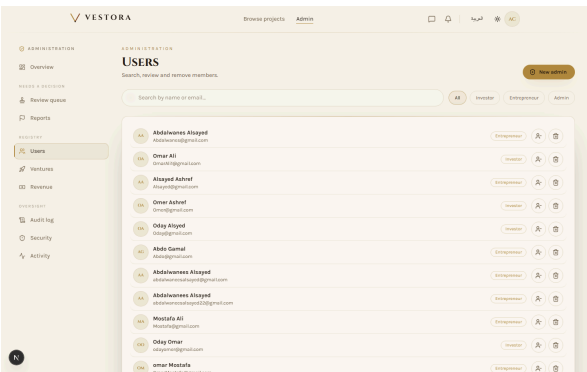


Figure 34 Venture registry across all states.



Account administration and suspension (\$10.10).

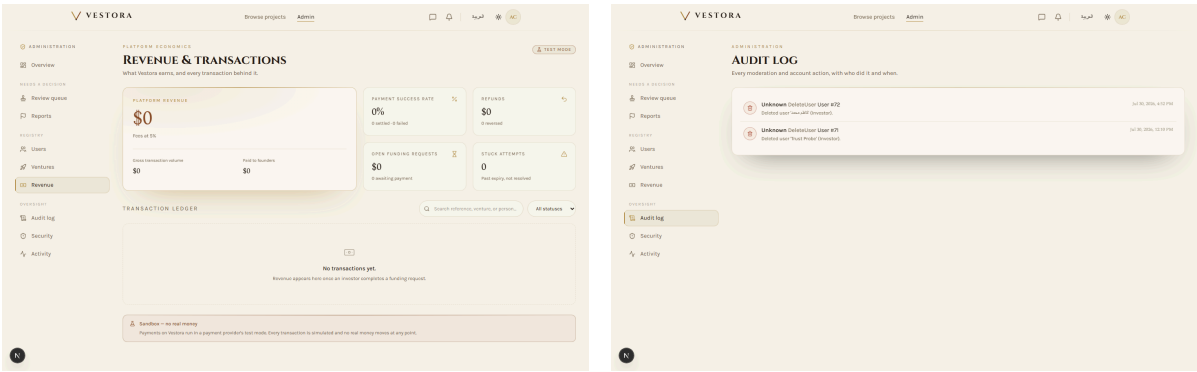


Figure 35 Fee revenue, derived from settled transactions (\$2.6). Audit log: administrative action attributed to its actor (\$10.13).

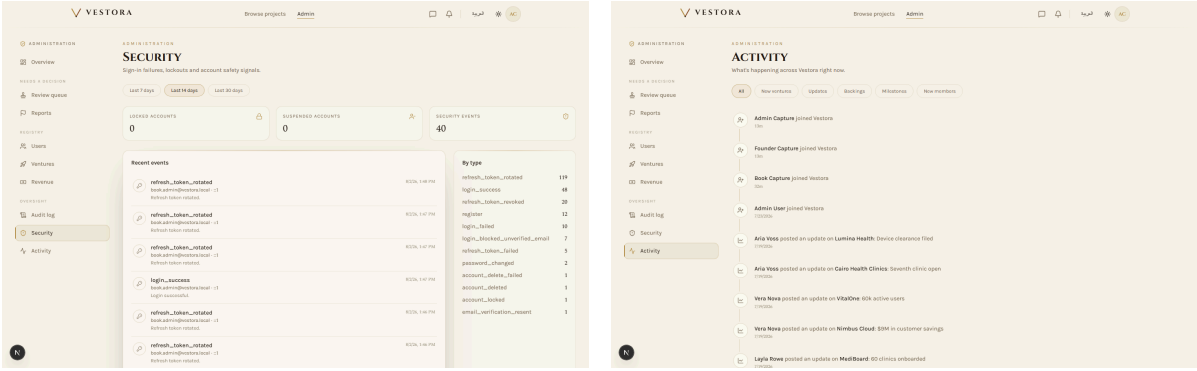


Figure 36 Security events from SecurityLog (\$10.13). Platform-wide activity stream.

5.1.4 Authorisation, observed

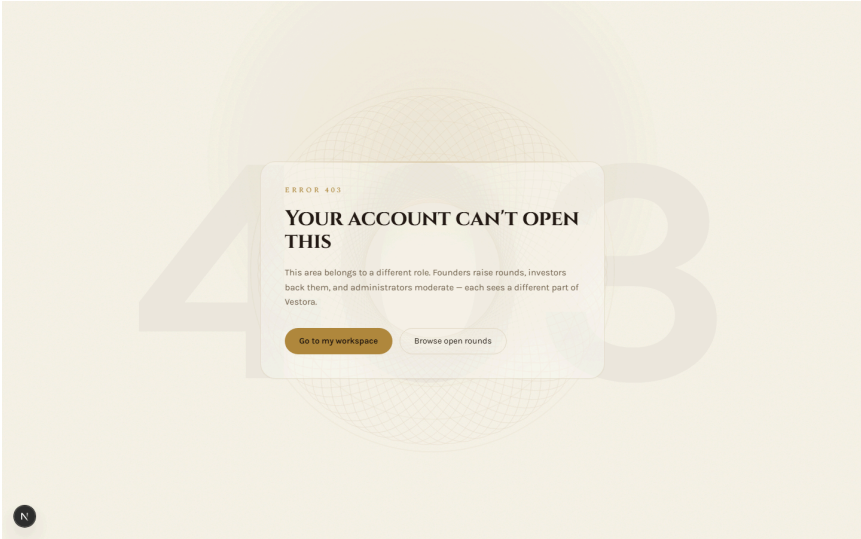


Figure 37 An investor account requesting a surface it is not entitled to. The client redirects to a refusal rather than rendering an empty page — and the API would refuse the underlying call regardless of what the client did (§10.8). This is case AZ-01 of Section 4, observed rather than executed.

NOTE
A defence that worked during capture. Three pages in the run redirected to sign-in mid-session — one per role. The cause is refresh-token rotation (§10.6): concurrent navigations each triggered a refresh, a rotated token was presented twice, and the chain was invalidated. That is case SC-09 behaving exactly as specified. The capture script was changed to re-authenticate and retry rather than to work around the mechanism.

5.1.5 Communication

The conversation captures in §12.3 were produced by seeding a realistic diligence exchange between the investor and founder accounts and then opening the thread from **both** sides, so the read receipts in each are the other account's actual state rather than a mock.

DRAFT Still to capture: the deal room with a document request in flight, and the checkout handoff to the payment provider. Both need seeded state that does not yet exist — a granted document request, and an approved commitment carried through to a provider session. The list below is the remaining plan.

5.2 Public Surfaces

	Surface	State to capture, and why
1	Landing page	Above the fold. It is the first evidence a visitor has (§8.1).
2	Venture listing	Populated, filters visible. The platform's discovery surface (§13.3).
3	Venture listing — filtered	A sector filter and a saved search applied, showing the result count change.
4	Venture detail	A venture with imagery, team, milestones and both funding figures visible (§14.1).
5	Venture detail — documents	Open and restricted documents together, showing the request affordance (§14.4).

Table 77 Public surfaces.

5.3 Identity

	Surface	State to capture, and why
6	Registration	Empty, with validation guidance visible.
7	Registration — error	A field-level validation error. Error states are part of the design system, not an afterthought (§8.8).
8	Email verification prompt	The state after registering and before verifying — a surface users actually meet (§10.3).
9	Sign-in — locked	The lockout message, which is identical to a wrong-password message (§10.7).
10	Onboarding	The step asking for sector interests.

Table 78 Identity surfaces, including two failure states.

5.4 Investor Surfaces

	Surface	State to capture, and why
11	Investor dashboard	Populated with committed and settled positions.
12	Commitment flow	The confirmation step, before approval.
13	Pipeline	At least one commitment in each state — the clearest possible illustration of §14.1.
14	Checkout	The provider handoff, showing that card entry leaves the platform.
15	Portfolio	Settled holdings only.
16	Watchlist	Populated and empty. The empty state explains what a watchlist is for (§8.8).

Table 79 Investor surfaces. Entry 16 is captured twice deliberately.

5.5 Founder Surfaces

	Surface	State to capture, and why
17	Founder dashboard	Committed and settled side by side, with a visible gap between them (§14.6).
18	Venture editor	Mid-edit, showing the re-review notice.
19	Funding requests	Awaiting the founder's approval.
20	Milestones and updates	A published update with imagery.
21	Document requests	A pending request awaiting a grant.

Table 80 Founder surfaces.

5.6 Communication

	Surface	State to capture, and why
22	Conversation	With presence, a typing indicator and read receipts all visible in one frame (Section 12).
23	Conversation — attachment	An image attachment in the thread.
24	Notifications	Populated, with unread state.
25	Deal room	Thread, documents and the commitment under discussion together (§14.4).

Table 81 Communication surfaces.

5.7 Administration

	Surface	State to capture, and why
26	Review queue	Populated.
27	Rejection dialogue	With a reason entered — rejection without a reason is not possible (§14.5).
28	Reports queue	Filtered by status.
29	Audit log	Showing administrative actions attributed to administrators (§10.13).
30	Revenue	Fee income derived from settled transactions (§2.6).

Table 82 Administrative surfaces.

5.8 Responsive and Theme

	Surface	State to capture, and why
31	Venture listing — mobile	Same information as desktop, rearranged. Nothing hidden (§8.6).
32	Venture detail — mobile	As above.
33	Dashboard — mobile	A data-dense surface, showing table scroll contained rather than page scroll.
34	Conversation — mobile	As above.
35	Any surface — dark theme	The dark token set applied (§8.2).
36	Focus visibility	Keyboard focus ring on an interactive element (§8.7).

Table 83 Responsive and theme captures. Entry 36 evidences an accessibility claim rather than illustrating a feature.

Key Code Listings

The main text carries only the fragments where the code itself is the point. This appendix holds the longer extracts a reader may want in full.

NOTE

These are extracts, not the codebase. Reproducing the source in an appendix would add several hundred pages and no understanding — the previous version of this document devoted half its length to narrating code line by line (§1.2), and this appendix exists to avoid repeating that. The complete source is in version control; what follows is the material a reader needs at hand while reading Section 11, Section 10 and Section 7.

6.1 The Payment Provider Boundary

The interface that makes the domain provider-independent (§11.2, ADR-02). Note what it does not mention: no venture, no investor, no funding target. It carries transport only.

```

1 public interface IPaymentProvider
2 {
3     /// "stripe" or "simulated" - stored on every transaction it handles,
4     /// so a row always carries the identity of the system that produced it.
5     string Name { get; }
6
7     Task<ProviderCheckout> CreateCheckoutAsync(
8         ProviderCheckoutRequest request, CancellationToken ct = default);
9
10    Task<ProviderPaymentResult> GetSessionResultAsync(
11        string sessionId, CancellationToken ct = default);
12
13    Task<ProviderRefundResult> RefundAsync(
14        string providerPaymentId, decimal amount, string currency,
15        CancellationToken ct = default);
16 }
```

Listing 5 `IPaymentProvider`. Two implementations satisfy it and neither contains a domain rule.

6.2 Webhook Verification and Idempotency

Three properties make this correct rather than merely present: verification runs over the raw body before parsing, it precedes every state change, and the duplicate check is backed by a uniqueness constraint rather than standing alone (§11.4, §11.5).

```

1 // 1. Verify over the raw bytes. Deserialising and re-serialising
2 //   changes them, and the signature is over bytes.
3 var raw = await ReadRawBodyAsync(Request, ct);
4 if (!_verifier.Verify(raw, Request.Headers["Stripe-Signature"], _secret))
5     return BadRequest(); // rejected and logged; nothing is read
6
7 var evt = _verifier.Parse(raw);
8
9 // 2. The event row is the idempotency key. The uniqueness constraint –
10 //   not this check – is what makes it safe under concurrency: two
11 //   simultaneous deliveries can both pass the check, and the second
12 //   insert then fails, which is the correct outcome.
13 if (await _db.PaymentEvents.AnyAsync(e => e.ProviderEventId == evt.Id, ct))
14     return Ok(); // already handled – acknowledge and stop
15
16 // 3. Only now may state change.
17 await _payments.ApplyProviderEventAsync(evt, ct);
18 return Ok();

```

Listing 6 Webhook handling. Acknowledging a duplicate with `200` is deliberate — an error causes the provider to retry indefinitely for an event that will never become processable.

6.3 Deriving Funding Totals

There is no `AmountRaised` column anywhere in the schema (ADR-04, §7.6). A venture’s total is arithmetic over rows that represent money that actually moved, computed in one place so two callers cannot disagree.

```

1 // Only Settled contributes. Approved commitments and pending, failed,
2 // expired or refunded transactions are all excluded – which is the
3 // entire integrity rule, expressible in one predicate because the
4 // model was built to make it so.
5 public Task<decimal> RaisedAsync(int projectId, CancellationToken ct) =>
6     _db.PaymentTransactions
7         .Where(t => t.FundingRequest.Investment.ProjectId == projectId)
8         .Where(t => t.Status == PaymentStatus.Settled)
9         .SumAsync(t => (decimal?)t.Amount, ct)
10        .ContinueWith(r => r.Result ?? 0m, ct);

```

Listing 7 Funding derivation. The `?? 0m` matters: a venture with no transactions has raised zero, not null.

6.4 Independent State Columns

The configuration that keeps moderation, lifecycle and commercial stage from overwriting one another (ADR-05, §7.6), and the composite index that keeps the resulting two-column visibility filter fast (§7.5).

```

1 // Three columns, three writers, three questions. The composite index
2 // exists because public visibility requires two of them to agree, so
3 // the listing query filters on both and orders by date.
4 modelBuilder.Entity<Project>()
5     .HasIndex(p => new { p.ModerationStatus, p.LifecycleStatus,
6     p.CreatedDate });
7
8 modelBuilder.Entity<Project>()
9     .HasIndex(p => p.Stage);

```

Listing 8 Venture state indexing.

6.5 Business Rules as Unique Indexes

Four rules that read like policy are implemented as constraints (§7.5). Each could have been an application check, and each would then have had a race between two concurrent requests.

```

1 // "Save once", "follow once", "review once" – enforced by the database,
2 // so a double-tap cannot produce two rows.
3 modelBuilder.Entity<Bookmark>()
4     .HasIndex(b => new { b.UserId, b.ProjectId }).IsUnique();
5
6 modelBuilder.Entity<Follow>()
7     .HasIndex(f => new { f.FollowerId, f.FollowedId }).IsUnique();
8
9 modelBuilder.Entity<Review>()
10    .HasIndex(r => new { r.ProjectId, r.InvestorId }).IsUnique();
11
12 // Account identity, and the refresh-token rule that two live tokens
13 // may never share a hash.
14 modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
15 modelBuilder.Entity<RefreshToken>().HasIndex(rt => rt.TokenHash).IsUnique();

```

Listing 9 Constraints carrying business rules.

6.6 Uniform Result Translation

Expected outcomes are values, not exceptions (§6.5). The mapping from outcome to HTTP status exists once for the whole API rather than being re-decided in each of the twenty-two controllers (§9.6).

```

1 // In the controller – parse, authorise, delegate, shape. Nothing else.
2 [HttpPost("{id}/publish")]
3 public async Task<IActionResult> Publish(int id, CancellationToken ct)
4 {
5     var result = await _projects.PublishAsync(id, CurrentUserId, ct);
6     return result.ToActionResult(this);
7 }

```

Listing 10 The controller-to-service seam. A controller may not reference `AppDbContext` (§6.5).

Installation and Setup Guide

7.1 Prerequisites

Requirement	Notes
.NET SDK	Matching the version pinned in the project file.
Node.js and npm	For the web application.
SQL Server	A local instance, or credentials for the hosted database. The project itself runs against the hosted instance (§16.3).
Git	To obtain the source.

Table 84 Prerequisites.

7.2 Configuration

The application **will not start** without its required configuration (ADR-10, §16.6). This is intentional: a system that starts with a placeholder signing key is not degraded, it is compromised, and nothing indicates it.

Nothing below is committed to the repository. In development these values live in the local user-secrets store; in production they are supplied by the hosting platform's configuration.

Setting	Purpose
Database connection string	Reaches the SQL Server instance.
Token signing key	Signs access tokens. Must be a strong secret (§10.5).
Token lifetimes	Access and refresh validity; lockout threshold and window (§5.6).
Payment provider selection	<code>simulated</code> or <code>stripe</code> . Defaults to simulated (§11.9).
Payment provider keys	Required only when the provider is <code>stripe</code> .
Webhook signing secret	Verifies provider callbacks (§11.4).
Platform fee	Basis points (§2.6).
SMTP host, port, credentials, sender	Transactional mail (§5.9).
Upload limits	Maximum size and permitted content types (§10.12).
Allowed origins	Cross-origin allow-list (§10.11).

Table 85 Required configuration. A missing value fails startup rather than defaulting.

7.3 Running the API

```

1 cd MyAppApi/MyAppApi
2 dotnet restore
3 dotnet ef database update      # applies all migrations (§7.7)
4 dotnet run

```

The API serves the interactive OpenAPI browser in development only (§9.8).

7.4 Running the Web Application

```
1 cd Frontend/vestora
2 npm install
3 npm run dev
```

The client needs the API’s base address in its own environment configuration.

7.5 Verifying the Installation

Run these in order. Each one exercises a different part of the stack, so a failure localises the problem.

- 1. **Health endpoint returns healthy.** Confirms the process is up **and** that it can reach the database — an application that is up but cannot reach its database is not healthy (§16.8).
- 2. **Register and verify an account.** Exercises the database, the token flow and SMTP.
- 3. **Create and approve a venture.** Exercises authorisation and the moderation pipeline.
- 4. **Commit and check out against the simulated provider.** Exercises the full funding chain (§11.3) without any external dependency.
- 5. **Open a conversation in two browsers.** Exercises the real-time hub, presence and read receipts (Section 12).

7.6 Common Problems

Symptom	Cause
The application exits immediately at startup.	A required setting is absent. This is ADR-10 working as designed; the log names the missing key.
Sign-in fails with a valid password.	The address is unverified (§10.3), or the account is locked after repeated failures (§10.7).
Timestamps display incorrectly.	Confirm the UTC converter is in the serialisation pipeline (§6.7). This defect is invisible when client and server share a timezone (§18.1).
Webhook calls are rejected.	The signing secret does not match, or a proxy has re-encoded the body — verification is over the raw bytes (§11.4).
A migration fails on the shared data-base.	It was applied by another developer. Pull before migrating (§18.5).

Table 86 Common setup problems and their causes.

User Manual

Three short guides, one per role. Each covers what that role needs to do and the points where the platform's behaviour is deliberate rather than obvious.

8.1 Getting Started (All Users)

1. **Register** with an email address and a password.
2. **Verify** the address from the link sent to it. Sign-in is refused until this is done, so that an account cannot be created against an address the registrant does not control.
3. **Complete onboarding**. It asks only for what makes the platform useful immediately; everything else is editable later from settings.

If you forget your password, request a reset. The link is valid for ten minutes — deliberately shorter than the verification link, because a reset link sitting in an inbox is a higher-value target.

If sign-in fails repeatedly, the account locks for fifteen minutes. The message is identical to a wrong password, so the mechanism does not confirm whether an account exists.

8.2 For Investors

8.2.1 Finding ventures

Browse the venture listing without an account, or search and filter by sector, funding stage and progress. Ranking is deterministic and attribute-based — it uses no hidden score, and a venture's position can be explained.

Save a search to turn a filter combination into a standing interest, and **bookmark** ventures to build a watchlist.

8.2.2 Assessing a venture

A venture page carries its description, imagery, team, milestones, published updates and open documents. Where a document is restricted, request access; the founder grants or refuses it.

Message the founder directly. The conversation shows presence, typing and read state, so you can tell whether your question has been seen.

8.2.3 Committing capital

1. **Commit** an amount on the venture page. Nothing is reserved and no money moves at this point.
2. **Wait for approval**. The founder or an administrator accepts or declines.
3. **Complete checkout**. An approved commitment can be taken to payment. The checkout has an expiry; if it lapses, the commitment stays approved and a new checkout can be opened.
4. **Settlement**. The venture's funding progress changes only once payment is confirmed.

NOTE

Approved is not funded. These are shown as two separate figures throughout the platform, and the difference is deliberate. Only settled payments count toward a venture's total — which is why a venture's progress does not move when your commitment is approved.

8.2.4 Tracking

Four surfaces answer four different questions: **pipeline** (what is in progress), **payments** (what has settled), **portfolio** (what you actually hold), and **activity** (what has happened).

Card details are entered on the payment provider's own page and never touch the platform.

8.3 For Founders

8.3.1 Submitting a venture

1. **Create** the venture with a title, description, funding target and imagery.
2. **Add** documents, team members and milestones.
3. **Submit** for review. It is not publicly visible until an administrator approves it.
4. **If rejected**, you receive a stated reason. Correct the venture and resubmit.

Editing a published venture returns it to review — but this affects only its moderation state. Funding progress and commercial stage are untouched, because they are separate fields written by different parts of the system.

8.3.2 Managing funding

Incoming commitments appear as requests to approve or decline. Approving a commitment does **not** collect the money; it permits the investor to proceed to checkout.

Your dashboard shows **committed** and **settled** as two figures. A large gap between them means approvals are not converting into payments, which is actionable information a single combined number would hide.

8.3.3 Keeping investors informed

Publish **updates** as work progresses and mark **milestones** as they are reached. Followers and investors are notified.

Note that the platform **records** what you report; it does not verify it. Your published history is timestamped and visible, and investors can see the relationship between what was promised and what was reported.

8.3.4 Documents

Attach documents as open or restricted. Restricted documents require a request that you grant or refuse, access is recorded, and downloads are logged — so you can see that a document was actually read.

8.4 For Administrators

8.4.1 The review queue

The primary surface. Each submitted venture is approved or rejected; **rejection requires a reason**, because a rejection without one is indistinguishable from a fault and produces a support request rather than a corrected submission.

8.4.2 Reports

User reports are filtered by status. Reports concern published content; administrators do not have access to private conversations.

8.4.3 Accounts

Accounts can be suspended and reinstated. **Accounts with financial history cannot be deleted** — the referential rules prevent it, deliberately, because deleting one would erase the record of commitments other parties made.

Note that suspension takes effect immediately for actions, but an already-issued access token remains valid for up to an hour for read access.

8.4.4 Oversight

Activity shows platform-wide events. **Security** shows authentication events. **Revenue** shows fee income, derived from settled transactions.

Audit records every administrative action against the administrator who took it. This includes your own actions, and it is visible to other administrators — administrative power on this platform is observable by design.